Veena S. Chakravarthi
Shivananda R. Koteshwar

System on Chip (SOC) Architecture

A Practical Approach

Springer

System on Chip (SOC) Architecture

Veena S. Chakravarthi • Shivananda R. Koteshwar

System on Chip (SOC) Architecture

A Practical Approach

Springer

Veena S. Chakravarthi
Sensesemi Technologies Private Limited
Bangalore, India

Shivananda R. Koteshwar
Belakoo Trust
Bangalore, India

ISBN 978-3-031-36241-5 ISBN 978-3-031-36242-2 (eBook)
https://doi.org/10.1007/978-3-031-36242-2

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2023
This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

Dedicated to aspiring System Architects.

Foreword by Pradip K. Dutta

Semiconductors are an integral part of the electronics industry, and their significance cannot be overstated. They have transformed the way we work, play, live, and learn. From the simplest gadgets to complex systems, semiconductors are at the heart of it all. The constant appetite for information and ever-increasing thrust for improving lifestyle is driving this innovation.

The industry megatrends show the pervasive nature of semiconductor penetration as we move from computation to communication to smart devices coupled with data everywhere.

With the rapid advancement of cloud technology backed by AI, the technical innovation in electronics products starting from massive data centers to a growing sea of smartphones is increasing. With the increasing bill of materials for semiconductors each year in simple gadgets to complicated systems, it is estimated that by the end of this decade, semiconductors will become a one-trillion-dollar industry.

Complete solutions with more and more advanced SoCs are the answer to the industry's two most important goals: cost and power savings. Every product, big or small, consists of a CPU, GPU, memory, I/O devices, software and more, all intelligent than ever before can be found in tablets, smartphones, embedded systems, and countless other devices. More importantly, designing a system-on-chip (SOC) has become a critical skill for next-generation engineers and architecting them for the set design goals is an important aspect of product design. Till date, the art of defining system architecture is acquired only through on-the-job experience.

This book offers a comprehensive guide to define SoC architecture and aims to equip readers with the knowledge and skills required to thrive in this ever-changing industry. It covers everything from the basics of architectures to the intricacies of designing complex systems, making it an essential resource for students and professionals alike.

With its clear and concise explanations, practical examples, and expert insights, this book is a must-read for anyone interested in the fascinating world of SoC design. I have no doubt that it will become an invaluable resource for many in the industry.

The demand for intelligent, highly integrated chip design keeps growing globally. This means that any aspiring chip architect of a young startup or an established house of silicon needs to become fully versed in the art of architecting SOCs. There is no better teacher to learn from, than from the practicing system architects in the industry.

With 50 plus years of cumulative experience as SoC architect and VLSI designer, Dr. Veena S. Chakravarthi and Dr. Shivananda R. Koteshwar have distinguished themselves as both artists and engineers. Their abilities to design large, complex electronics systems on silicon have created a baseline, enabling technologies for several IOT and communications systems. Their depth of experience has allowed them to create a perfect primer for any designer wanting to arm themselves with the necessary mindset to understand the chip architecture and development cycle for SoCs. This practical approach contains straightforward applications of known techniques to create a structure which will help freshman engineers contribute effectively to the SoC design and development process.

With the chip act in the USA and semiconductor mission in India, I'm excited about the future of our industry and where SoCs can take us. They are at the heart of the advancements in medical, biotech, automotive, telecommunication, and countless other industries that will change our world in the future. This book is a thoughtful guide for any aspiring chip architect, and I am sure this will help create the next generation of innovators, inventors, and dreamers.

Pradip K. Dutta



Former Group VP & MD, Synopsys South Asia
Advisor to the Board, Simyog Technology Pvt. Ltd

Dr. Pradip K. Dutta, in his last corporate role, was a Group Vice President of Synopsys Inc., a leading provider of world's most advanced electronic design automation software for semiconductor chip design. He was also the Managing Director of its wholly owned subsidiaries in India and Sri Lanka. He led the growth of Synopsys South Asia from a little over 50 employees to more than 4000 highly skilled personnel spread across Bangalore, Colombo, Hyderabad, Mumbai, New Delhi, Noida, and Pune, representing every single business unit of the company.

Prior to joining Synopsys, Dr. Dutta had spent 12 years with Delphi Automotive Systems in the field of powertrain electronics and held a variety of positions in

engineering, business development, and management in the USA and in the Asia-Pacific region, including assignments in Japan, Singapore, and India.

Currently, he is actively involved with advising startups, angel investments, and Board positions. He was on the Board of Vayavya Labs for 9 years, a technology startup from Belgaum, Karnataka, where he started initially as a Nominee Director from the Department of Science and Technology (GoI). He is an Advisor to the Board of Simyog Technology, a deep tech startup in computational software from IISc, Bangalore. He is also an investor and Advisor to the Board of Schooglink, enabling affordable, accessible, and quality education for rural and semi-urban K-12 students.

Dr. Dutta earned his BTech (Hons) in Electronics Engineering from IIT Kharagpur followed by MS and Ph.D. in Electrical Engineering from the University of Maryland, College Park, USA. He was a recipient of NIST (National Institute of Standards and Technology) Fellowship under the auspices of the US Department of Commerce for his graduate studies. He was conferred the Distinguished Alumnus Award by IIT Kharagpur in 2011.

He has been a past chair of India Electronics and Semiconductor Association (IESA) and served on the boards of several industry bodies, including American Chamber of Commerce (AMCHAM) and Indo-American Chamber of Commerce (IACC). He advises on various university academic programs including those at IIT Kharagpur and IIIT Bangalore.

He takes keen interest in community relations and is a champion of Inclusion & Diversity in workplace.

Foreword by S. Janakiraman

System on Chip (SOC) that started as evolution has reached the stage of revolution. What started as integration of microprocessor, bit of memory, and IO interfaces as microcontrollers to design simpler systems have now moved into whole system on a chip with no other electronics in the box.

New-generation SoCs not only integrated processor, memory, IO, and network that had been traditionally digital but also analog/mixed-signal components like sensors, antennas, and many others. Now SoCs not only are restricted to transitional hardware arenas but also integration of software elements like multimedia, analytics at edge, and Artificial Intelligence elements too.

While the above building blocks get integrated into SoC, custom SoCs add a secret sauce of innovative applications for Industrial/Medical/Financial domains hardcoded to execute most of the real-life needs with speed and simplicity.

Though the visionary Gordon Moore is no more, his prophecy of the number of transistors doubling every two years seems to live forever. As the Semicon technology gets into finer nodes below 3.0 nm and as innovations in applications keep moving up, what can be achieved with SoC has no limits. With a recent buzz on AI and ChatGPT, there exists the potential for future devices not only mimicking human physical activities through robotics but the best of brains integrated through AI on Chip. There is no doubt that the world runs on small and powerful semiconductor devices. Designing a semiconductor device for any application is a complex activity which requires the correct definition of system architecture for its first-time success. For this reason, typically, system architecture is defined by the experienced and specialist in the domain. I am glad to see this book addressing the need to equip designers with the required knowledge and skills to architect the SOCs of good quality of results (QOR).

I have known Veena Chakravarthi since the formation of Mindtree in 1999. When we decided as an IT company, we will not limit ourselves to only IT applications but also start from hardcore Semicon and Embedded Electronic designs. She had been a major asset in the early years of our organization, willing to take challenges of designing complex silicon and SoCs of that time. It is very rare to find leaders who not only excel in industry but equally in academic teaching too. Veena is one such leader.

I got to know Dr. Shivananda Koteshwar recently. His accomplishments are not only limited to the field of semiconductor, but acts in theatres too. Again, another rare combination of skills that can unleash imagination for the future.

Veena and Dr. Shivananda, with their vast hands-on experience, have handcrafted this book that will be of high value not only to entry-level students but also for professionals who wish to prepare themselves for future challenges.

Best wishes to both of them for all success in the launch of this next-generation book *System on Chip (SoC) Architecture!*

S. Janakiraman



Founder & Chairman
Nuvepro Technologies

Janakiraman Srinivasan is currently the Chairman of Nuvepro Technologies, a startup in Cloud Technologies, and also President of Indo Japan Chamber of Commerce and Industries (IJCCI), Karnataka.

Prior to the above, Janakiraman was the President and CTO of Mindtree Ltd., as well as a Co-Founder and Board Member. He served Mindtree from 1999 to 2014.

Prior to Mindtree, Janakiraman served as the Chief Executive of the Global R&D Unit of Wipro where, he started his career as one of the first five employees in 1980.

Janakiraman was one of a Co-Founder of India Semiconductor Association (ISA) in 2002 and served for six years in its executive council and later as its Chairman and Advisor.

Janakiraman did his BE at Regional Engineering College, Trichy (1973–1978) and MTech from IIT-Chennai (1978–1980). He is a Distinguished Alumni of REC for his entrepreneurial excellence.

Janakiraman served on the board of Yokogawa India Ltd. for four years until March 2019. He is currently a Board Member in startups Nuvepro Technologies, Innohub Technologies, Sanohub Technologies, and Netra Accelerator Foundation. He is actively involved in promoting startups and entrepreneurial environment in Karnataka.

Foreword by Puneet Gupta

Moore's Law has enabled ever-tighter integration. This has allowed system components which used to sit on a package or a board to be brought into the chip itself. Such systems-on-chip or SoCs have higher performance, lower power, and typically lower cost than their disaggregated counterparts. The field of SoCs has seen remarkable advancements over the past few decades, transforming the way we design, develop, and deploy complex digital systems. From smartphones and tablets to wearables and IoT devices, SoCs have become the backbone of modern computing and communication systems.

An SoC contains a complex network of interconnected components, including processors, memory, peripherals, and interfaces. Designing and implementing such systems require a deep understanding of various disciplines, including computer architecture, digital design, software engineering, and verification.

This book aims to provide an accessible, broad but brief introduction to the field of systems on chip, focusing on practical aspects of SoC design. The book starts with an introduction to SoC abstractions and technology, defining the concept of SoC and its architecture and outlining the different types of systems and architectures. It then discusses system modeling and verification flows. It continues with a brief overview of different types of embedded processors, embedded system memory, and storage. It further covers the SoC design flow, including verification and IP integration. Later in the book, issues of architecture for performance, flexibility, and low power are outlined as well. Finally, the book also discusses software issues including system bootloader, system SW co-verification, and software on SoC architecture.

Overall, this book provides a comprehensive overview of SoC design. It is written in a clear and accessible style, making it suitable especially for students and professionals alike who want to get a capsule introduction to SoC design.

Dr. Puneet Gupta



Dr. Puneet Gupta
Professor of Electrical and Computer Engineering
UCLA Henry Samueli School of Engineering and Applied Science

Puneet Gupta received the BTech degree in Electrical Engineering from the Indian Institute of Technology Delhi, New Delhi, India, in 2000, and the PhD degree from the University of California at San Diego, San Diego, CA, USA, in 2007. He is currently a Faculty Member at the Electrical and Computer Engineering Department, University of California at Los Angeles. He co-founded Blaze DFM Inc., Sunnyvale, CA, USA, in 2004 and served as its Product Architect until 2007. He has authored over 200 papers, 18 US patents, a book, and two book chapters in the areas of design-technology co-optimization as well as variability/reliability-aware architectures. Dr. Gupta is an IEEE Fellow and was a recipient of the NSF CAREER Award, the ACM/SIGDA Outstanding New Faculty Award, SRC Inventor Recognition Award, and the IBM Faculty Award. He has led the multi-university IMPACT+ Center which focused on future semiconductor technologies. He currently leads the System Benchmarking theme within the SRC CHIMES JUMP 2.0 center.

Preface

The global semiconductor industry is experiencing positive demand across all regions post pandemic for two reasons: increased demand for consumer electronic devices and new opportunities because of emerging IoT, 5G, Machine learning (ML), and artificial intelligence (AI) technologies. Increasing demands for faster and advanced systems on a chip will drive the growth to a massive $1 trillion market by the end of this decade. With a large number of acquisitions happening in recent years, and start-ups playing a major role in the semiconductor market horizon, consolidations enable better product offerings that work more cohesively. For system optimization for a target application, it is getting complex integrating different subsystems into the same silicon when it in reality comes from different vendors and sometimes different technology nodes and even targeted different fabrication units. Overall, recent years have seen an increase in consolidation in the semiconductor industry, a trend that is expected to continue in the near future.

Another trend that has a tremendous impact on the semiconductor industry is the evolving nature of physics. The radical shift from the physics-based limits of Moore's era of scale to the complex high-level solution of new age systems.

While this was happening, multiple markets like automobiles, computing, and gaming were struggling to source chips and other related materials even as prices skyrocketed. Another compounding factor to the current semiconductor shortage is the dearth of tools and human resources tasked to designing extremely complex semiconductor chips of the future.

The book intends to present a comprehensive overview of the method of converting requirements to complex solutions with good architecture. Defining system-on-chip (SoC) architecture is historically done by experienced designers who have managed cross-functional development activities of chip design. Having worked in the semiconductor design and EDA industry cumulatively for over five decades, with experience in designing and architecting SoCs of various complexity, we decided to pass on the experiential knowledge to the next-generation designers, inventors, and innovators, and hence this book.

This book ensures that designers are aware and are able to understand the rationale of their role in the overall development of solutions with complex systems on chips.

While the book is targeted to be a valuable reference guide for professional designers who are part of semiconductor design, development, manufacturing, and fabrication houses and who aspire to be chip architects at some point in time, it is also targeted to practicing architects and researchers.

This book makes a ready candidate for curriculum offering during the advanced engineering courses at all levels to make them industry/research ready in SoC design.

It would not have been possible to realize this project without the support of many of our friends, colleagues, and family. First, we wish to thank our common instinct and desire to share the acquired knowledge with the coming generation. Our heartfelt thanks to our respective loving families, who encouraged us in this endeavor.

We are indebted to our reviewers, who patiently read each of the book chapters and offered line-by-line reviews.

And we are mighty thankful to Pradip Dutta, Advisor to the Board, Simyog Technology Private Limited, for taking time out of his busy schedule to write the foreword for this book.

We are also deeply indebted to Janakiraman S, Founder & Chairman, Nuvepro Technologies, for his valuable foreword for this book.

Our heartfelt thanks to Dr. Kavish Seth, Director, Solutions Engineering, Synopsys, for his valuable inputs in organizing chapters and Sashi Gundala for the editorial review.

This book is organized considering minimal knowledge on systems, structure, and architecture. Chapter 1 defines the system, its organization and evolution. It also discusses different types of system architectures. Chapter 2 deals with the System on Chips (SoC), its subsystems, and the role of processors in SoC subsystems. It introduces different processors and their use in System on Chips for different purposes. It covers different architectures for processor subsystems. It describes a variety of SoC architectures. Chapter 3 deals with the methodology of defining SoC architecture. It explains different system concepts, requirements capture, and approach to define architecture. Chapter 4 covers the choice of SoC architectures, hardware-software interfaces, systems modeling to derive system specification, and the selection of IP cores for SoC architecture. Chapter 5 deals with the role of embedded memories for SoC, different on-chip storage methods in SoCs, and their selection. Chapter 6 elaborates on the IoT SoC case study for environmental parameter measurements. Chapter 7 discusses the software and software architectures in SoC. Chapter 8 details the feasibility of design considerations in defining SoC architecture. Chapter 9 deals with converting architecture to the data path and

control path architectures and techniques to detail them as ready-to-design considerations. It also deals with the advanced architectures for complex SoCs. Chapter 10 is a self-assessment question bank to test your understanding of the different topics presented in this book classified chapter-wise.

We will be very happy if users find each chapter useful and subsequently turn out to be chip or solution architects. We are curious about your feedback and criticisms. We welcome them.



Dr. Veena S. Chakravarthi is a Bangalore-based technologist, system-on-chip architect, and educator. Over a career spanning three decades, she has spawned of several VLSI design and incubation centers and managed several high-performance tech-teams at ITI Limited and across various MNCs like Mindtree Consulting Pvt. Ltd., Centillium India Pvt. Ltd. Transwitch India Pvt. Ltd., Ikanos Communications Pvt. Ltd., Pereira Ventures, Asarva Chips and Technologies, Saankhya Labs, and Prodigy Technovations Pvt. Ltd. She has been Research Head and an Adjunct Professor at the Department of Electronics and Communication Engineering, BNM Institute of Technology, Bangalore. She holds a PhD from Bangalore University and a Management certification from IIM Bangalore. Veena was the co-founder and CTO of Sensesemi Technologies Private Limited, a healthcare company. She is the inventor and has filed five patents in VLSI and healthcare for the organizations she has worked with. She is now Staff Instructor at Synopsys India. She is the author of books *A Practical Approach to VLSI System on Chip (SoC) Design*, *Internet of Things and Machine to Machine (M2M) Communication Technologies*, and *System on Chip (SoC) Physical Design*, all of which are published by Springer Nature. She has been a blogger, mentor, and is active in promoting VLSI education and research. She is a senior IEEE member.



Dr. Shivananda R. Koteshwar has more than 27 years of experience in the semiconductor industry. He is currently the Vice President and Site Head of the EDA Group with Synopsys India. He is a founding trustee of the Belakoo Trust, which focuses on rural education, skills development, and experiential learning, and is a trustee of Aurinko Trust, whose flagship product is The Aurinko Academy, a progressive K12 school with career-focused programs in all streams. His start-up portfolio includes Your Philanthropy Story, Ultraadox, FabSkool, and KreedaLoka. Dr. Koteshwar received a doctorate in Education Management from the Indian Institute of Management Bangalore (IIMB), a Postgraduate Diploma in Innovation and Design Thinking from the Emeritus Institute of Management, a master's in Electrical Engineering from the OGI School of Science and Engineering, and BTech in Electronics and Communication Engineering from Mysore University. He is extremely active in mentoring start-ups catering to EdTech and the learning space and has served as a visiting faculty member in engineering design and management for several leading colleges and universities in India. He is currently an Adjunct Professor at Dayananda Sagar University and ISBR Business School. Dr. Koteshwar has written five books, including *SoC Physical Design: A Comprehensive Guide* (Springer, 2022).

Bangalore, India

Veena S. Chakravarthi
Shivananda R. Koteshwar

Contents

1 Introduction to Systems 1
- 1.1 Introduction 1
- 1.2 System Organization 2
- 1.3 System on Chip (SOC) 3
- 1.4 SOC Constituents 5
- 1.5 SOC Evolution 6
- 1.6 SOC Attributes 7
- 1.7 SOC Subsystems 8
- 1.8 SOC Architectures 9
- 1.9 Advanced Trends in SOCs 14

2 System on Chips (SOC) 17
- 2.1 Processor in SOC 17
- 2.2 Types of Processor Architectures 18
  - 2.2.1 Instruction-driven Processor Architecture 18
  - 2.2.2 Data-driven Architecture 19
  - 2.2.3 Platform-driven Architecture 19
  - 2.2.4 IP Core-driven SOC Processors 21
- 2.3 System-Level/Virtual Modeling Platforms 21
- 2.4 Physical Platforms 22
- 2.5 System Fabrication 23
- 2.6 Domain-specific SOC Architectures 25
  - 2.6.1 IOT Architecture 25
  - 2.6.2 Different Stages of IoT Architecture 28
  - 2.6.3 Digital Signal Processors 29

3 System on Chip (SOC) Architecture 37
- 3.1 System on Chip (SOC) Architecture 37
- 3.2 Architecture Processes and Their Dependencies 37
- 3.3 SOC Architecture and SOC Design 39
  - 3.3.1 Requirement Capture 40

3.3.2 Types of SOC Architectures . . . 40
3.4 Approach to Defining SOC Architecture . . . 44

4 Application-specific SOCs . . . 49
4.1 Application-specific SOCs . . . 49
4.2 Embedded Computers . . . 50
4.2.1 Embedded Processor Subsystems . . . 50
4.2.2 Hardware-Software Interface . . . 53
4.2.3 Exceptions and Interrupts . . . 56
4.3 System Modelling . . . 57
4.4 Capturing System Requirements . . . 58
4.4.1 Explicit and Implicit Requirements . . . 60
4.5 Deriving SOC Specifications . . . 60
4.5.1 Clock Frequency . . . 60
4.5.2 Choice of Processor Cores . . . 62
4.5.3 System Software . . . 62
4.6 Processor Subsystem IP selection . . . 63

5 Storage in SOCs . . . 65
5.1 Storage in SOCs . . . 65
5.1.1 On-chip Cache Memories . . . 66
5.1.2 Translation Lookaside Buffer (TLB) . . . 70
5.1.3 On-chip Data or Buffers . . . 71
5.2 Types of Memories . . . 71
5.2.1 Redundancy in Memory . . . 73
5.3 Memories in Advanced System Architectures . . . 73

6 SOC Architecture: A Case Study . . . 75
6.1 Introduction to SOC Requirements (Environment Parameters) . . . 75
6.2 Smart IoT SOC for Environment Monitoring—An Architectural Case Study . . . 76
6.2.1 Identifying SOC Requirements . . . 77
6.2.2 Proof of Concept System . . . 78
6.2.3 IoT Device . . . 81
6.3 IOT SOC Hardware Software Partition . . . 89
6.3.1 System Design Plan . . . 90
References . . . 90

7 IOT SOC Architecture Definition . . . 91
7.1 Chip Architecture Flow . . . 91
7.2 Chip Specification . . . 92
7.3 Hard IP Cores . . . 94
7.4 Soft IP Cores . . . 95
7.5 Firm IP Core . . . 99
7.6 Chip Data Flow Architecture . . . 100
7.7 Case Study of SOC for Environmental Monitoring . . . 102
7.7.1 SOC Architecture as Standard Data Path . . . 104

8 SOC Software 105
8.1 Software Development Life Cycle (SDLC) 105
8.1.1 Phase 1: Requirement Collection and Analysis 105
8.1.2 Phase 2: Feasibility Study 106
8.1.3 Phase 3: Software Design 107
8.2 High-Level Design (HLD) 107
8.3 Low-Level Design (LLD) 107
8.3.1 Phase 4: Coding 107
8.3.2 Phase 5: Testing 108
8.3.3 Phase 6: Installation/Deployment 108
8.3.4 Phase 7: Maintenance 108
8.4 Software Architecture Styles 108
8.5 Layered Architecture 109
8.6 Event-Driven Architecture 110
8.7 IOT Software Development for Environment Monitoring 110
8.7.1 Communication Technologies 111
8.7.2 OSI Model in Software Architecture 112
8.8 Application Software for IOT SOC 112
8.9 Prototype Design of IOT SOC-Based Solution 117
8.9.1 Cloud Storage and Analytics Setup 118
8.10 Registration to Cloud Server 119
8.11 Cloud Server Access 119
8.12 Product Design 121
8.12.1 Product Validation and Testing 121
8.13 Future Scope of IOT SOC Solution 125

9 SOC Advanced Architectures 127
9.1 SOC Advanced Architectures 127
9.2 SOC Accelerators 127
9.3 Scalable Platform Architectures 128
9.4 Multiple Clock SOC Architecture 128
9.5 Multiple Voltage Architecture 130
9.6 Requirements to Create a Multi-Voltage Design 131
9.6.1 Level Shifters 131
9.6.2 Power Gating 131
9.6.3 Clock Gating 133
9.7 Near Memory Processing Architectures 133
9.7.1 In-Memory Processing Architectures 135
9.8 Guidelines for a Good SOC Architecture 136
Reference 138

10 Self-Assessment Question Bank 139

Index 157

Abbreviations

AI	Artificial Intelligence
API	Application programming interface
AFE	Analog front end
BG	Background
BS	Boundary scan
CDO	Clock and datapath optimization
CTS	Clock tree synthesis
DRC	Design rule check
EM	Electromigration
FG	Front ground
GPIO	General purpose input-output
HAL	Hardware abstraction layer
HDR	High-definition range
HMD	Head-mounted devices
HMI	Human-machine interface
HTML	Hypertext markup language
HTTP	Hypertext transfer protocol
IR	Current-resistance
JTAG	Joint Test Action Group
LWM2M	Lightweight machine-to-machine
M2M	Machine to machine
MAN	Short form for manual
ML	Machine learning
OSI	Open systems interconnection
PDK	Process design kit
QoS	Quality of service
RAM	Random access memory
RPL	Routing
RT	receiver-transmitter
SG	Study group
SMS	Short message service

SOC	System on Chip
UDP	User datagram protocol
UE	User equipment
UI and ML	User interface and machine learning
UI	User interface
UPM	Useful packages and modules
URI	User resource identifier
URL	Uniform resource locator
URN	Uniform resource name
USB	Universal serial bus
USP	Unique selling proposition
VLSI	Very large-scale integration

Chapter 1
Introduction to Systems



1.1 Introduction

All electronic products we use have electronic systems in them. An electronic system forms the main part of any electronic product. A **system** is an organized set of functional blocks or components or subsystems that work for an application. A subsystem performs different functions and interacts with other subsystems in a coordinated way for an overall systemic function. The subsystems are made up of many smaller functional blocks which work in tandem for a subsystem function. Examples of electronic systems are television system, mobile system, air conditioning system, computer system, vacuum cleaners, and microwave system. An electronic system is implemented in **hardware** or **software** or **combination of both**. Hardware and software parts of the system closely interact in a predefined manner in a system solution as per the functional requirement of an application. Hardware consists of electronic components in the form of integrated circuits (ICs), discrete components which are mounted and interconnected to other external components on a **Printed Circuit Board (PCB)** and assembled with other parts like power supply connectors, battery, and user input-output devices such as keypads or keyboards and displays. The software functionalities exist in system as embedded software, layered software stored in memory components or storage devices in hardware and are typically controlled by the operating system software. The systems interact with the environment or user with the help of specific **Input-Output devices** such as keyboard, keypads, and displays. System components require power supplies which are connected from the source or generated using a dedicated electronic circuit from the main supply or a battery in the hardware. The systems are housed in a suitable mechanical package designed considering application requirement, ease of use, reliability, esthetics, safety, and security of the user, environment, and the system itself. These systems or system components are developed as per the appropriate applicable standard specifications, and they follow specifications enforced by

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

V. S. Chakravarthi, S. R. Koteshwar, *System on Chip (SOC) Architecture*,
https://doi.org/10.1007/978-3-031-36242-2_1

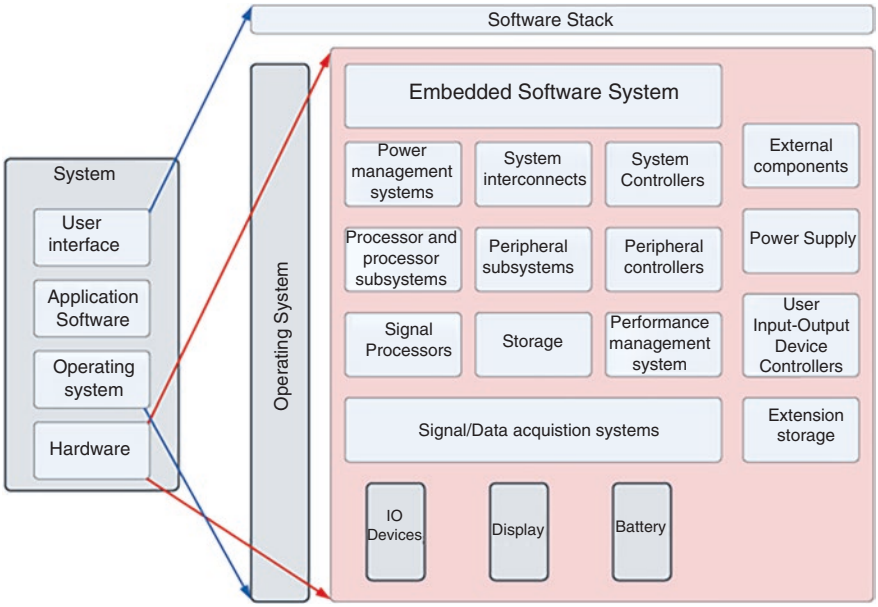


Fig. 1.1 System with its internal subsystems

regional or global regulatory or professional bodies such as IEEE, ITU, and IEC. This ensures safety, security, and interoperability with other systems developed by others. Sometimes one or more standards may be applicable to the systems. The electronic systems are certified by authorized certification bodies at component, subsystem, system, or product level. An example of a generic electronic system with its subsystems is shown in Fig. 1.1.

As shown in the figure, an electronic system consists of hardware and software which include operating system software, application software, and user interface. The hardware includes many system components for signal/data acquisition, data processing, storage, battery or power supply components, user input-output devices, and displays. Each of the components in a system consists of one or more processors, peripherals, memories, and software integrated or as discrete components. The software interacts with hardware components or user through the interface and the application software. This is coordinated by operating system software.

1.2 System Organization

Each system component has a dedicated functionality which is implemented to execute them in a systematic manner. If a dedicated functionality is executed by a set of system components such as processor, storage, and interface with other

system components or environment is referred as system. Most systems are application specific trying to integrate parts of the application-specific functionalities into a single entity. This makes it compact, reliable, and fast. This is accomplished by interconnecting different components and subsystems in a defined way so that they can coordinate in the correct sequence as dictated by the master control system. Electronic systems communicate with the external world through components like sensors and actuators or user input-output devices. Sensors sense physical parameters, and actuators control the different components so that desired physical parameters can be controlled. User can configure or interact with the system through input devices such as keyboards or keypads and get the response from the system through output devices such as displays. In a system the input data or signals captured are processed by the systems and the responses are generated. These are used by other system components in an application or user. The processed data from a system in some cases is used to interpret and derive meaningful information and communicated to the user or automatically used to control external devices. This process is carried out in an organized way. Based on the platform on which the system components are placed and interconnected, they are classified as **systems on printed circuit boards**, **multi-chip systems**, and **System on Chips (SOCs)**. Based on the input they process, the systems are classified as **Analog/Mixed signal systems, digital systems,** and **multimedia systems**. SOCs are electronic systems fabricated on a silicon wafer using advanced technologies such as CMOS process technologies. System components such as sensors and actuators are also fabricated using micro-electromechanical systems (MEMs) technology. All electronic system components such as SOCs, sensors, switches, and power supply circuits are processed and assembled on a glass epoxy substrate using **Printed Circuit Board** (PCB) technology. This book deals with the System on Chips (SOC) and its architecture definitions.

1.3 System on Chip (SOC)

A product function in part or full realized as VLSI system on a piece of silicon is called **System on Chip or SOC**. SOC with software system with real-time operating system is called **Embedded System**. Software system consists of power on initialization functionality called **booting.** Booting a system is referred to step by step sequence of operations which configures and initializes the system blocks to a stage which is ready to function for an intended operation. This part of software is called **boot loader** in a system.

Most of the electronic system functionality that is realized at low voltage can be integrated as System on Chip. The resources for this are data processors, memory, and other logic components. These resources are required to be conceptually and logically structured to perform a specified part of the functions in a system. The system architecture defines the conceptual models in terms of functions and their

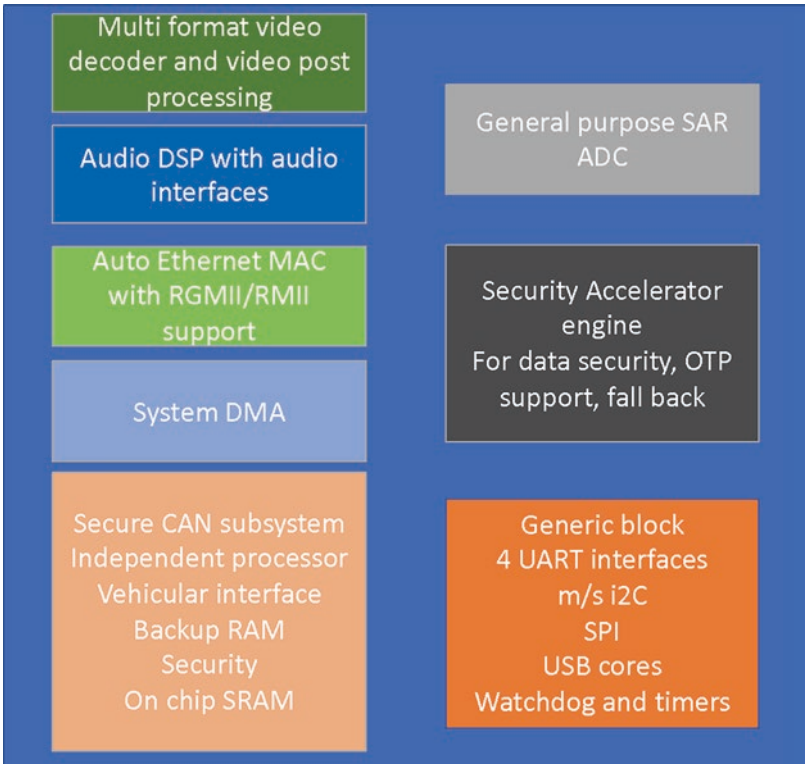


Fig. 1.2 Auto SOC block diagram

structure. System on Chip architecture is a process of defining the organization of subsystems, their functionalities, and the process by which each interacts with other subsystems and most importantly it can be realized as an integrated silicon chip. SOC architecture definition involves defining the method of interaction with the external world, partitioning the system functionality into smaller independent functions, which are sub-modules, their interactions, sequence of interactions, method of interactions, and timing. Figure 1.2 shows an example of an automotive system that can be implemented as System on Chip.

Figure 1.3 shows a smartphone System on Chip (SOC). It is evident in the figure shown that each of the SOC block is a complex functional block, with a processor core, random access memory (RAM), read only memory (ROM), and some organised proprietary functional logic to form a functional subsystem. It consists of cellular core, application processor core, and multimedia core as major system blocks in addition to regular general purpose functional modules. Cellular core is used to process the Subscriber Identity Module (SIM) data. Multimedia processor core is used to process all video, audio signals, and an application processor core is used for all other installed application functions. Some of the applications for which this core is used are retail consumer applications, online banking, gaming,

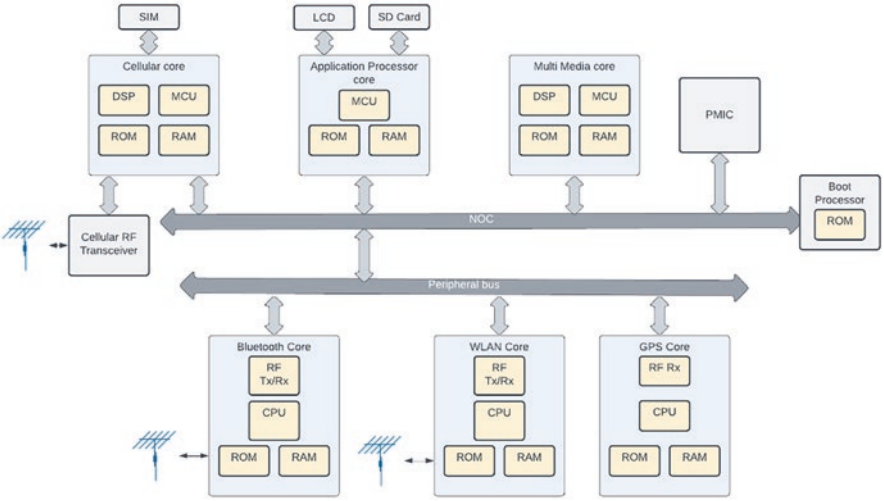


Fig. 1.3 Smartphone SOC

and data processing for document processing application software. In addition to these functionalities, the SOC, supports different interfacing functionalities using the interface cores. Some of these are Bluetooth, WIFI and geo positional system (GPS)-related applications. In addition, the SOC has a boot processor for starting the system and other support functions.

1.4 SOC Constituents

As can be seen in examples, an SOC subsystem consists of one or many processors and processor subsystems, on-chip memories, peripheral subsystems, standard communication cores, and peripheral device and memory controllers. All these are connected through a network on chip (NOC) or an interconnection bus. On-chip processor subsystem comprises of single or multiple processor cores, and standard peripheral bus bridges and interfaces. On-chip memory is of SRAMs and Read Only memory (ROM). Major constituents of SOC are application-specific functional blocks, protocol blocks, data processing blocks, physical layer functions in communication processors or high-efficiency signal processing cores in multimedia SOCs or a rule-based switching function in router SOCs. On-chip communication cores support communication with peer devices and make them interoperable. Some of the most used communication cores are USB, UART, I2C, and SPI. Present-day SOCs also consist of high-performance mixed-signal (analog and digital signal) processing blocks like ADCs/DACs, signal conditioning circuits, on-chip sensing functions for temperature and activity sensing and functional blocks with radio

frequency (RF) transceiver functions. Extra glue logic is added, which helps in housekeeping the data for processing or communication transfers, communication interface, and accelerators to support embedded firmware. Application-specific protocol functions and sensor/ actuator interfaces with signal conditioning circuits and other control path modules like clock–reset circuitry, debug logic, DMAC, memory controllers, interrupt controllers, bus conversions modules, network interconnect modules, and DFT logic are typically found in SOCs.

Complex SOCs use one or more instances of the above components depending on the application. Getting the SOC design right the first time with high QOR depends on the architecture of the System on Chip.

1.5 SOC Evolution

In the past, chip design technology could only process systems of very small complexity due to lack of electronic design automation (EDA) tools, high-end design computing systems, and advanced process technologies. But with the advent of cell-based design technologies, primarily synthesis tools facilitated designs of high complexity over the years. The invention of the synthesis tool is believed to be linchpin technology for earlier manual design methodology and advanced standard cell-based design methodology. Hardware description languages originally intended for hardware documents became hardware design languages. Figure 1.4 shows the evolution of chip development methodologies and major discoveries which accelerated the development process.

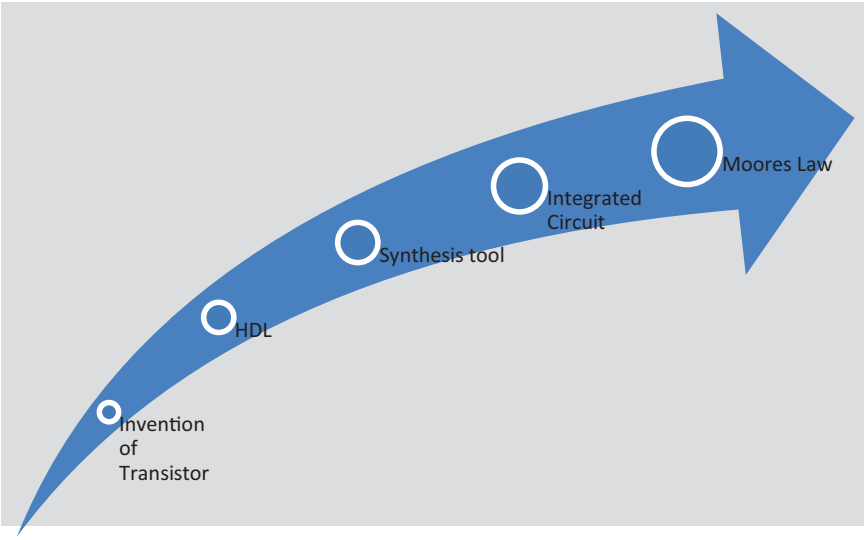


Fig. 1.4 Evolution of chip development

1.6 SOC Attributes

Many factors affect the figure of merit or quality of system architecture. Some of the important factors of a good system architecture, in addition to the intended system functionality and performance are:

- Reliability
- Availability
- Scalability
- Flexibility
- Maintainability

Reliability
A reliable system meets all its user requirements in application scenarios all the time. When you are designing a system, a set of features are planned and implemented, and an expectation on the behavior of system responses to the application requirements in terms of functions and performance is set. If your system performs all these features without failing, then the system is reliable.

Fault Tolerant System
Fault tolerant system is the one that reliably functions as intended, even in the presence of faults. Faults are the errors that arise in a particular component in the system. An occurrence of fault need not result in system failure. ***Failure*** is the state when the system is not able to perform as expected. It does not have an intended behavior for which it was designed in an application scenario.

Availability
System availability ensures the agreed level of operational performance as demanded by the user in the intended applications. A good system has high availability as per the application requirement. It is assessed in terms of response time or latency, speed, throughput, and power performance. There are different specialized design principles adopted to ensure high availability of systems. Some of which are to have a backup mechanism, a safe state for critical events, etc. Detecting possibility failure and resolving is another technique adopted to make the systems highly available.

Scalability
It refers to the ability of the system to deal with the increased load conditions. As a thumb rule, scalability is addressed by overdesigning the systems. If the system requirement is X, it is designed for 10X and tested for 100X. There can be situations where there can be a sudden increase in load unexpectedly, then the system must not only not fail but be smart enough to manage the additional load efficiently. This is the scalability of the system. For this, it is essential to assess the performance requirement of the system in real-time and adapt dynamically. There are different factors which describe the load on the system. Typical of them are:

- Data processing rate or throughput
- Speed of the system
- Power consumption
- Operating system variance

Apart from functional scalable factors such as number of users served, number of inputs, million instructions per second (MIPS) processed, and round-trip time (RTT), etc. are system-specific requirements of a good SOC.

For Systems on Chip (SOC)s, there are the additional **quality of result** (QOR)s such as **speed performance, power consumption,** and **silicon area** of the die. The three parameters power consumption, speed performance, and area of die (PPA) are considered system design goals which are the requirements in addition to the system functionality and reliability. A SOC architecture directly addresses all these parameters.

Flexibility
A good system architecture must be flexible enough to be able to configure to the variable environment. For example, when it is not used, it must be flexible enough to configure to power down or hibernate mode, so that very less power is consumed. The use of internal memory or external memory is another example of the flexibility required in systems.

Maintainability
Ease of maintenance is always preferred as system maintenance incurs costs and maintaining systems in the field is very expensive. Features such as remote maintenance, and periodic upgradation are requirements of systems.

1.7 SOC Subsystems

Most subsystems use processor that performs arithmetic and logic operations for data processing. Therefore, a typical System on Chip (SOC) consist of a processing unit, and data transfer unit, and a data path. The sequence and the data flow is controlled by the signals generated in control path, that sequences the processes along the data path. To interact with the external world, it consists of interface blocks which comply with standards defined by professional bodies such as IEEE, ATSE, and ITU. Compliance with standards guarantees the interoperability of the SOC developed with other SOCs with the same or peer interface subsystems. The interface to the physical world is through analog or interface cores working at radio frequency (RF) signals. Some of the examples of peripheral interface cores are UART, USB, SPI, SATA, and I2C.

The subsystems of SOCs are thus classified as follows:

- Protocol block
- Data converters
- Sensors
- Actuators or controllers
- Interface cores
- Computational cores
- Processor subsystems

1.8 SOC Architectures

Subsystems have processors that are used to perform major computing or data processing functions. Hence, a processor is the brain of the systems. Processors perform data processing functions using instructions which are commands to be executed issued by the software for any specific applications. Hence, they are developed using a set of instructions. The number of instructions supported depends on the computing requirement for an application. For example, a cloud server application requires large computations compared to a mobile smartphone. The processors support different instruction sets. The processors that support the instruction sets are designed for executing these instructions, which are arithmetic and logical instructions. Arithmetic instructions are addition, subtraction, multiplication, and division of different types of numbers. Logic instructions are logic operations such as AND, NAND, OR, SHIFT, XOR, etc. In addition, the processors also support data storage and data load instructions which are useful for part processing. The basic instructions are combined to define ***complex*** instructions to process multiple simple instructions in sequence without extra software commands depending on the application requirements. Instructions in processors are executed in a core functional block called a ***central processing unit or CPU***. The instructions are offered as a set of instructions for processors. The processors are thus designed using ***instruction set architecture (ISA)***. ISA in computer science is also called ***computer architecture***. ISA has become a common design technique of processors over the years, using specialized techniques for data handling to maximize performance. Processors used for high-end computing applications such as high-end computers and servers are developed using such many complex instructions in their instruction set, which on other computers take many simple instructions. Such instructions take multiple cycle steps, control multiple functional units, or otherwise appear on a larger scale than the bulk of simple instructions implemented by the given processor. The examples of complex instructions are:

- Complicated integer and floating-point arithmetic
- Direct memory access (DMA) transfers of a large amount of data from memory to registers
- Transferring multiple data to memories
- Performing read, modify, and write operations of the atomic data
- Arithmetic logic unit (ALU) executing arithmetic and logic operations on data in memory than local registers.

Many complex instructions are found in the instruction set popularly called complex instruction set computers (CISC) which belong to a family of x86 instruction sets also known as 80x86 instruction sets. This is because Intel initially developed these instructions to run on their 8086 microprocessors and 8088 varient. The most basic CISC instruction set consists of 81 instructions which, over the years, have many more instructions added to them depending on support in advanced processors. Most desktops and laptops to date use X86 ISA-based processors in them. At

the high end, x86 ISA continues to dominate computation-intensive workstation and cloud computing server segments. The advancement in the processor is the extension of the support for more and more complex instructions for specific data processing such as graphic data, real-time data, etc.

Computing needs exist in applications such as smartphones, IOT devices, and mobile devices. These applications do not require many complex instructions set. Such processors are developed using a reduced set of instructions called reduced instruction set computing or RISC architectures. A reduced instruction set computer (RISC) simplifies the processor by efficiently implementing only the instructions that are frequently used in programs, while the less common operations are implemented as subroutines, having their resulting additional processor execution time offset by infrequent use.

Classification of Instruction Set Architecture (ISAs)
ISAs, therefore, are mainly classified depending on the architectural complexity as follows:

- Reduced instruction set computers or RISC. Figure 1.5 shows the RISC architecture.
- Complex instruction computers or CISC. Figure 1.6 shows the conceptual CISC architecture.

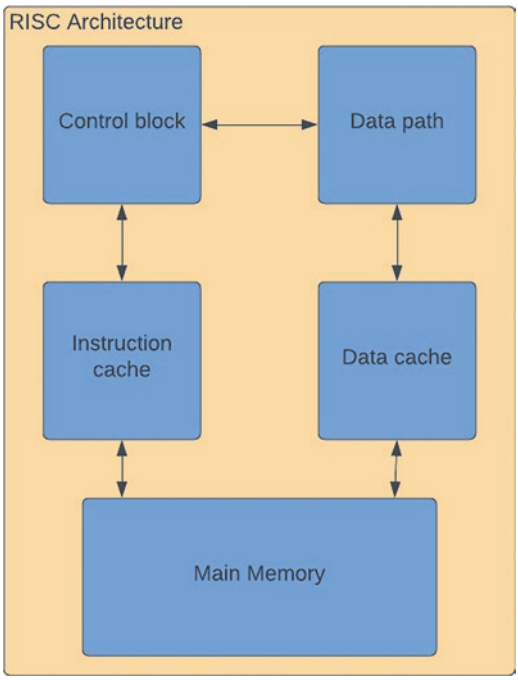


Fig. 1.5 RISC system architecture

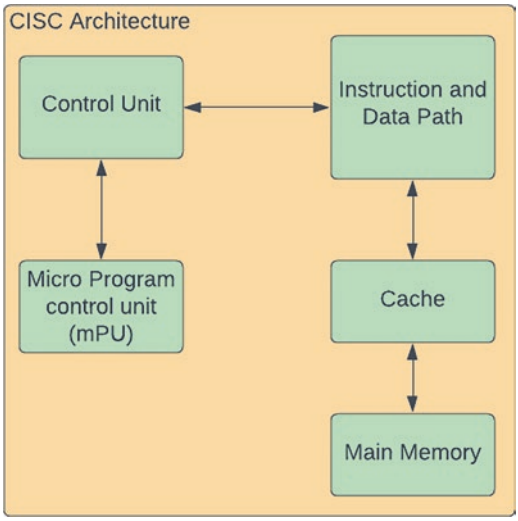


Fig. 1.6 CISC system architecture

There are other architectures, such as ***Very long instruction word (VLIW) and explicitly parallel instruction computing (EPIC)*** architectures, which exploit special hardware realizations of instruction-level parallel operations for executing special instructions. There are processors with bare minimal instructions called ***minimal instruction set computing (MISC)*** and ***one instruction set computer (OISC),*** which are not widely used architectures.

Types of Systems

Different types of systems use different processors based on CISC or RISC architectures depending on the applications needed for data processing. Some of the important classifications are discussed in this section. Depending on the number of processors integrated in a system, they are classified as:

- Uniprocessor architectures
- Multiprocessor architectures

Figure 1.7 shows uniprocessor and multiprocessor systems. Processors in these systems are referred to as cores.

Depending on the type of physical structure, systems are classified as

- Embedded systems
- System on Chip

Figure 1.8 shows the embedded system and System on Chips (SOCs). Embedded systems are developed using a System on Chips.

Depending on generic functions performed by the systems, they are classified as:

- Processing systems
- Computational systems

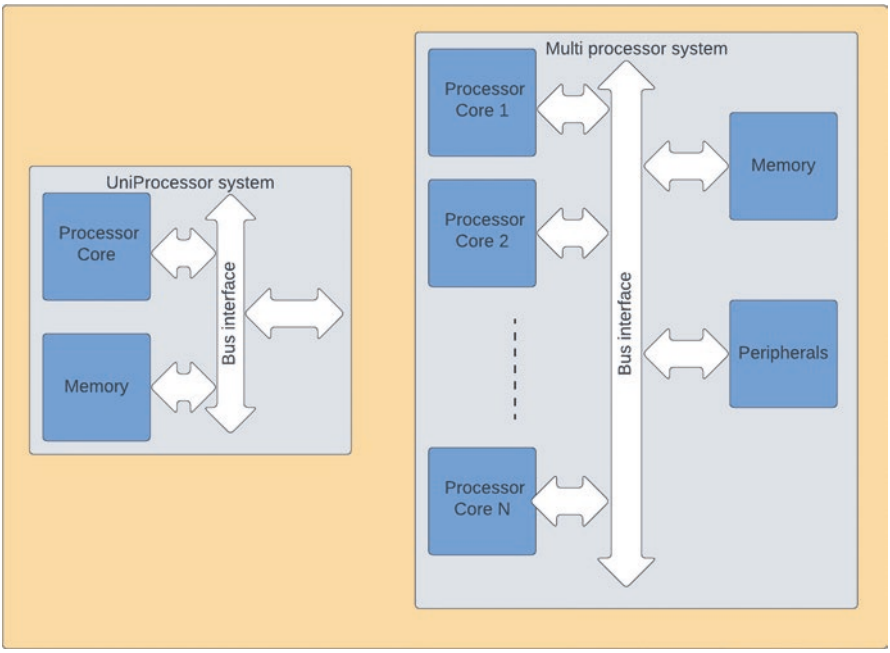


Fig. 1.7 Uniprocessor and Multiprocessor systems

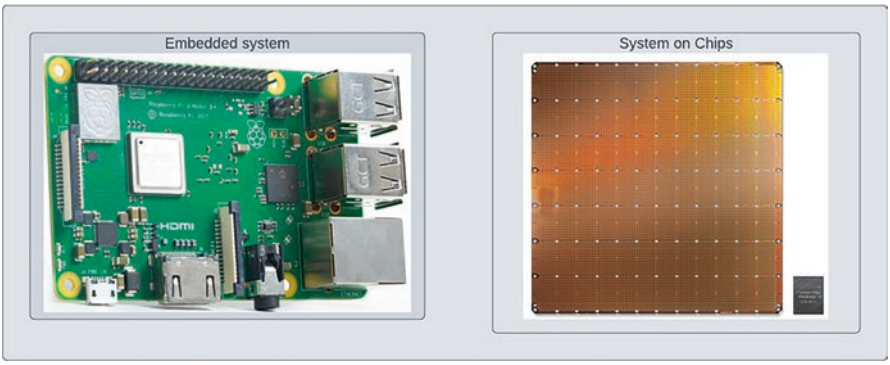


Fig. 1.8 Embedded system and System on Chips

Figure 1.9 shows the processing and computing systems.

- Control systems
- Signal processing systems

Figure 1.10 shows control and signal processing systems

In addition, there are application-specific systems and are classified as

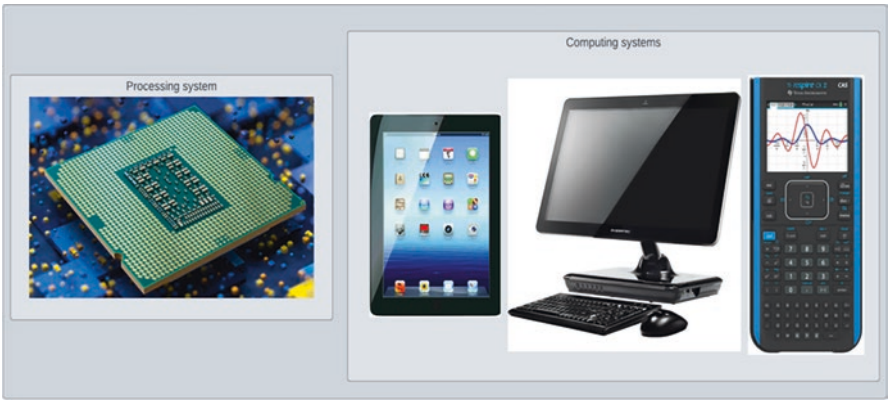


Fig. 1.9 Processing system and computing systems

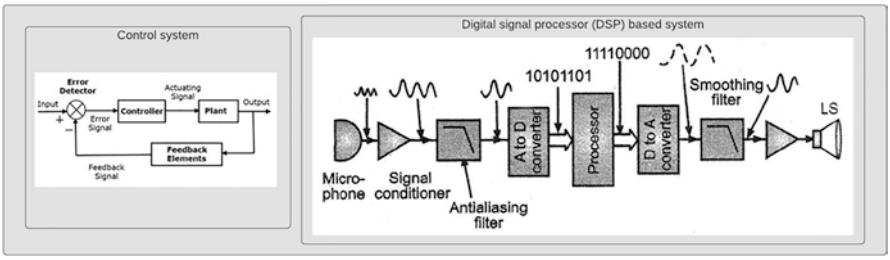


Fig. 1.10 Control system and digital signal processing (DSP)-based system

- General optical systems
- Communication systems
- Wireless systems
- Wired systems
- IOT systems

All electronic products use one type, or the other systems listed above. With advancements in semiconductor chip design and fabrication technologies over the decades, almost all electronic products use systems developed on semiconductor chips or SOCs. Since SOCs proliferating every product of any application, they have become an indispensable part of electronic products, big or small. All these systems have different architectures depending on the application requirements. System architecture defines a set of principles regarding the way the systems are designed and explains how each of the system components interacts with others to perform in predefined function as required by the application. System architecture defines the structure of submodules or system components and how they are organized or structured to perform a specific functionality. It also describes the relationships between components, levels of abstraction, and other aspects of the system. An architecture can be used to define the goals of a system development

project, which is used to guide the design and development of a proposed system for a targeted application.

The ***system architecture diagram*** is an abstract depiction of the system's component architecture. It is a visual representation of the system architecture. It shows the connections between the various subsystems of the system and indicates what functions each subsystem performs. The general system representation shows the major functions of the system and the relationships between the various system subsystems. Subsystems can be hardware or software.

System designers use system architecture as a reference document to design the systems. To define a system architecture, it is necessary to have a breadth of knowledge regarding applications, design principles, concepts and design flows and platforms, modeling requirements and techniques and verification strategies which are generally acquired by experience. This chapter, and subsequent ones, serves as a reference document for defining the system architecture in a systematic way which can be applied to define any System on Chip architecture definition.

1.9 Advanced Trends in SOCs

Phenomenal growth in the past couple of decades in technologies has enabled complex systems to be implemented on-chip by integrating ready-to-integrate subsystems, different types of subsystems such as analog, mixed-signal subsystems, communication cores, interface cores, and many hundreds of processor cores. To realize these kinds of complex systems on chip with highly demanding performance requirements for sophisticated and mission critical applications, a robust architecture is necessary. Also, the exorbitant cost of design and fabrication of these chips in deep submicron technology has always added pressure on system developers to do it right the first time This requires the right choice of subsystems; architectural decisions are required straight from a very early stage of design development. This poses many challenges throughout the development cycle of systems on chip.

To work on the challenges of architectures, it is good to understand the evolution of the integrated circuits itself. The journey of semiconductors started in the 1960s with the invention of integrating transistors by Texas Instruments. Eventually, the number of transistors getting integrated started growing so that the decade of the 60s is termed as the semiconductor era. **Gordon Moore,** co-founder of Intel announced that the number of transistors integrated will double every 2 years, which is known as Moore's law even today. Ever since the first microprocessor Intel's 4004 was introduced by Intel, the processors complexity grew to a large extent with increased processing power. The decade of the 1970s is known to be the era of personal computers which uses these processors. The following decade came to be known as networking eras in which the computers were seen networked and started communicating with each other, thus naming the decade of 1980s as the networking era. As the internet revolutionized the next decade of 1990s, it was aptly known to

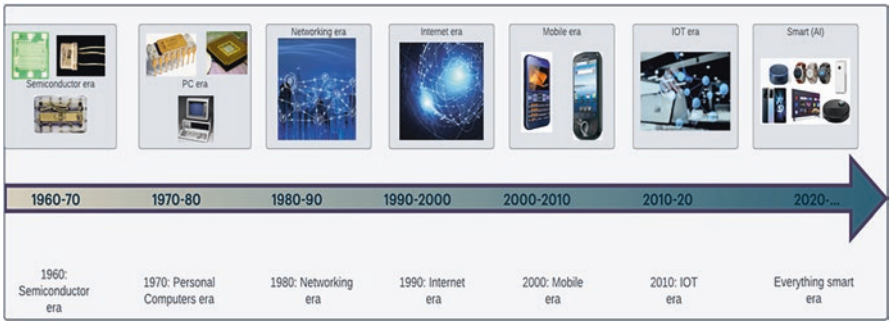


Fig. 1.11 Technological evolution

be the internet era. The year 2000 onwards is the era of mobile phones, and the recent decade with digital technologies infiltrating people's lives is seen as the IOT era. The technology in current years is making everything smarter with machine learning and artificial intelligence in every product we use. If you see, the key enabler for this advancement is the integrated circuit technology, which started by integrating few transistors in the 1960s has grown to integrate hundreds of millions of transistors on silicon chips today. This has enabled systems, networks, and solutions specific to applications being realized on chips in today's times.

Figure 1.11 shows the pronominal growth of these technologies over the decades and their evolution. Currently, as the integration of more and more components continue further, the trend is to make every system around us smart in terms of quality of results (QOR). Going further, the trend may continue to integrate the entire solution on a chip.

This requires doing system design right from the architecture level. Designing a system on a chip has its own advantages and disadvantages. Major advantages of System on Chip are the following:

- Lower chip area and hence product size
- Reduced product cost with reuse strategies
- High-speed operations
- Reliable product realization
- High-level design security

Major disadvantages of SOC are:

- High development cost
- Required to doing development right first time
- Long development cycles if required IPs are not available
- Need highly skilled resources to architect, design, and development

Chapter 2
System on Chips (SOC)



The subsystems and the components in a system are organized in an orderly manner and are fabricated in a single chip or assembled in a same package to form a System on Chip (SOC). Each subsystem has a dedicated function in the SOC. Each subsystem has its own set of functional blocks or work with other system components in a larger configuration. Most subsystems either use one or more processors or share processors in a SOC.

2.1 Processor in SOC

Processors are one of the most flexible components in an SOC. Depending on the requirements, SOC use one or many processors in its subsystems. Processor core is used for processing incoming signal or data. It is therefore necessary to understand some of the well-known processor system architectures. Processor cores are used for many functions in a system. It is used for booting the system in the power-on sequence. Booting is the process of bringing up the system and initializing the system components to default known state and be ready for processing the input data or signal. The processor used for this purpose is called **Boot processor**. Processor core is used for configuration and protocol implementation in peripheral interface cores. Some of the interface cores are USB, PCIexpress, SDIO, and SATA. Such a processor is called **Peripheral processor.** The processor used for processing the application-specific functions and processes is called **Application processor.** The processor which processes the captured input signals is a **Signal processor.**

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

V. S. Chakravarthi, S. R. Koteshwar, *System on Chip (SOC) Architecture*,
https://doi.org/10.1007/978-3-031-36242-2_2

2.2 Types of Processor Architectures

Processors are also classified depending on the way they process the input data as instruction-driven or data-driven processors.

2.2.1 Instruction-driven Processor Architecture

Instruction-driven processors are used in high-performance System on Chips. The reduced instruction set computer (RISC) is arguably one of the most commonly implemented processor architectures. In this type of processor, instructions are stored in the memory and the processor fetches the instructions one by one and executes it in sequential order. Instruction-driven processors are suitable for almost all data processing and protocol implementations and hence these processors are widely used in SOC architecture for general data processing and peripheral processor systems. The software development tools over a wide variety of standard operating systems are available which further boosts the processor usage. SOC software is developed using high-level programming languages such as System C, C++. This standardizes the software development for the system and offers enormous flexibility for SOC developers to use them in all subsystem functionalities. Popular examples of RISC-based processors include PowerPC™, ARM™, and MIPS™. Figure 2.1 shows the general instruction-driven architecture of processor cores in a SOC.

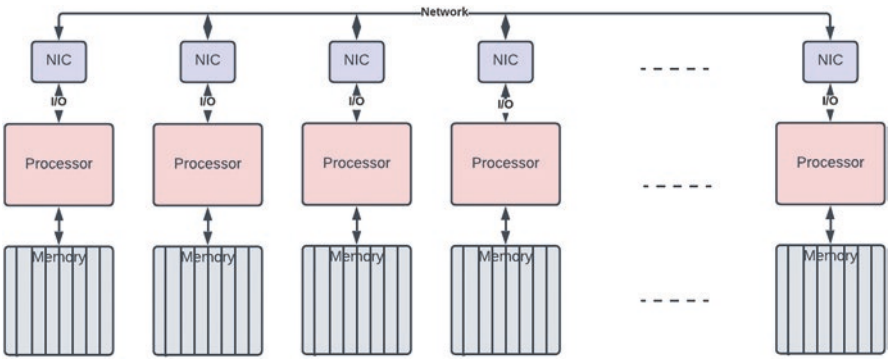


Fig. 2.1 Instruction-driven architecture

2.2.2 *Data-driven Architecture*

The performance of the SOC can be increased phenomenally by architecting systems by minimizing unnecessary data movement and processing data close to its storage. This has resulted in **data-driven SOC architectures**. This avoids data movement from the storage to processor thus increasing the speed and decreasing the power consumption. Reduction in data movement reduces power hungry input-output data fetch operations by the processor from the data stored in memories. Data is processed in near to its storage. Hence in SOC architectures, one can decide to use most appropriate processors for the kind of data to be processed. An application processor core, interface processor core, and multimedia processor core in the mobile SOC example seen in the previous chapter illustrate the spatially distributed processors. This speeds up data processing.

An explosion in data is forcing chipmakers to rethink where to process data, which are the best types of processors and memories for different types of data, and how to structure, partition, and prioritize the movement of raw and processed data. Figure 2.2 shows the data-driven SOC architecture using processors.

2.2.3 *Platform-driven Architecture*

To sustain the high peak performance in real time applications, common system architecture called **platform driven architecture**, targeted to number of applications is defined. These SOCs are targeted to a number of applications. The

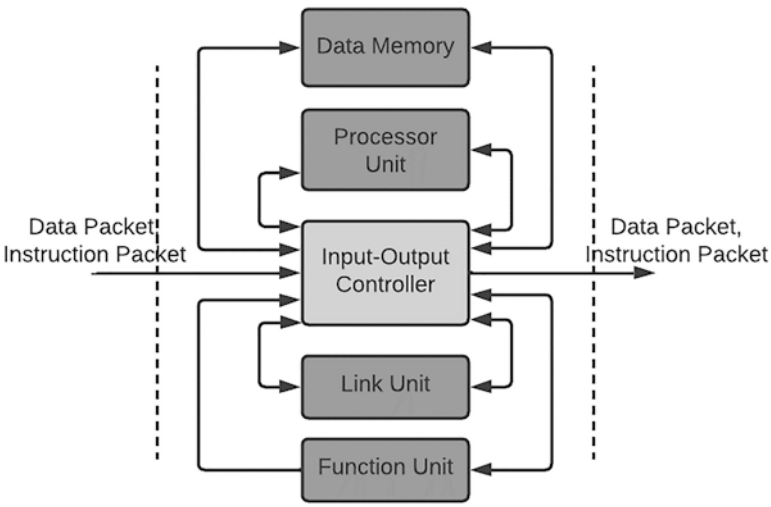


Fig. 2.2 Data-driven architecture

standard common platform SOC architecture avoids typical problems of processor subsystems, such as cache misses, memory stalls, and interrupt issues. The SOC platform chip is a product that results from using a platform-based methodology. It provides numerous configuration options to support a wide variety of SOC application requirements. The concept of platform-based design for SOC is rapidly gaining traction among IOT applications. Some of the advantages of platform-based system architectures are:

- They provide the means to package or bundle previously used discrete components and maximize the value of those offerings. This advantage comes from using a pre-defined, pre-verified set of components to define a variety of systems.
- A particular platform may allow a solution provider to compete in a chosen market segment that otherwise could not compete in.

In video platforms, beyond the processor core, additional functionality is required for the application that is common industry-standard I/O and other peripherals and application-specific IP. In addition, the platform architectures provide system developers an option to offer many software cores and development tools to use with the platform. This helps them develop many generations of multimedia applications using the same SOC. Normally, the developers also provide a reference design based on the SOC. Figure 2.3 shows a general digital video platform architecture.

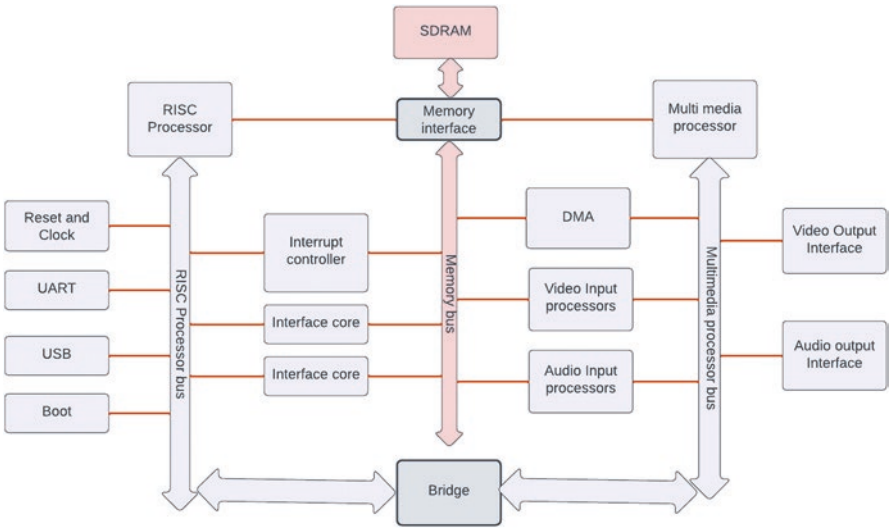


Fig. 2.3 Example of platform architecture

2.2.4 IP Core-driven SOC Processors

There is a provision for providing a large set of virtual intellectual property (IP) cores for SOC development in contrast to the SOC platforms. IP platforms are usually market-specific as opposed to application-specific. Converting IP platforms to SOC platforms is relatively easy. An example is the ARM PrimeXsys platform, originally developed as a wireless IP platform with the ARM processor core. There are a number of software platforms available for the design of embedded systems using these SOCs. This platform consists of an Real time operating System (RTOS) or Operating System (OS), middleware, application-specific software, and the development tools, for use with the SOC IP platform. Examples are the wind river software platform, recent RISC V platforms etc. The major requirements of the IP cores are that they must be available in forms that allow their integration into SOC designs. They must support standard EDA-based design flows and design modeling at different levels of abstraction. They must be available as synthesizable RTL as a soft core that enables SOC designer to add, modify and delete part of the functions to suit integration. The soft IP core must also contain the necessary support and scripts to be synthesizable using synthesis tool and a configurable test bench that allows the user to run verification tests as a part of the SOC integration platform. This level of abstraction will be licensed from an IP vendor to the SOC developer. There are platform-building tools for this type of platform available from EDA tools vendors. IP offerings for SOC development must comply with the format and necessary development support tools as described by the SPIRIT consortium. Advanced SOC design methodologies permit integration of cores in many formats like hard, soft and semi soft cores. Hard macros can not be modified or customised in any way during the design cycle. They can just be integrated into the SOC.

2.3 System-Level/Virtual Modeling Platforms

A system-level model is a functionally accurate and/or timing-accurate component that models the behavior, timing, and interfaces of some or all of the components of the platform and provides control and visibility of the internal state of the hardware. These models provide cycle and timing accuracy or timing-close accuracy and are written in high level programming languages such as System C, C/C++ or a derivation thereof, and these will then run much faster than the source RTL or gate-level simulation. The modeling of the system may use transaction-level modeling (TLM) to describe the interaction of components with a staged progression to more and more accuracy. The Open SystemC Initiative (OSCI) has defined transaction-level modeling specifications for SystemC models. At a high level of abstraction, IP modeling involves virtual platforms. They are configurable or fixed simulation platforms that can run with real operating system images and real

application software. Each virtual platform is an executable representation of the components of an SOC platform. It runs on a host workstation under the control of an application on the host machine. The models focus on the functionality or behavior of the system and sacrifice timing accuracy for performance. This approach allows operating systems to boot and run with reasonable response times. It allows applications, device drivers, and sometimes OS ports to be developed. The models are normally written in C/C++, System C, or other modeling languages. In addition to emulating the "internal" components of the system, a virtual platform also emulates the external devices connected to the peripheral interfaces. This can include memory, hard disks, terminals, switches, LEDs, keyboards, mouse, and displays. In some cases, the Virtual Platform may connect virtual components to real components on the host system. For example, the keyboard input comes from the host's real keyboard. Tools for system-level/virtual platform modeling are available from the major EDA and ESL vendors.

2.4 Physical Platforms

Physical platforms normally take the form of a development board containing the components in a chip or an FPGA. Real devices such as keyboards, displays, hard disks, etc., connect to provide a single board computer running an operating system or monitor. A prototyping area, normally utilizing FPGAs, is available for the expansion of the platform. In addition, external I/O buses, such as USB or PCI-Express, make the platform expandable for vertical applications or debugging. This can be extended to emulation systems.

Usually, the SOC platform developer collaborates with a set of soft IP core providers, and software development partners to bundle the platform offerings. All the components from all collaborators are validated on the SOC platform, and each of them has an abstraction layer due to which individual components can be modified to suit the integration and retain interoperability requirements. A typical example of such platforms are:

- Function critical hardware components such as configurable processors with configurations to maximize the system performance, including the memory subsystem, interrupts, and on-chip interconnects.
- IP platform cores for common peripherals required in most embedded systems such as Real Time Clock (RTC), Serial Port (UART), and General Purpose I/O (GPIO). Customers may choose the source of the common peripheral IP, but still, retain software compatibility by use of the hardware abstraction layer (HAL).
- The key component of the hardware platform is the system controller. It uses a crossbar-switch bus structure for the interconnect fabric, making it suitable for low latency and for high bandwidth applications. System controllers have optimal interfaces to SDR and DDR/DDR2 system memory with a memory controller connected to the interconnect fabric. Additional components may include an

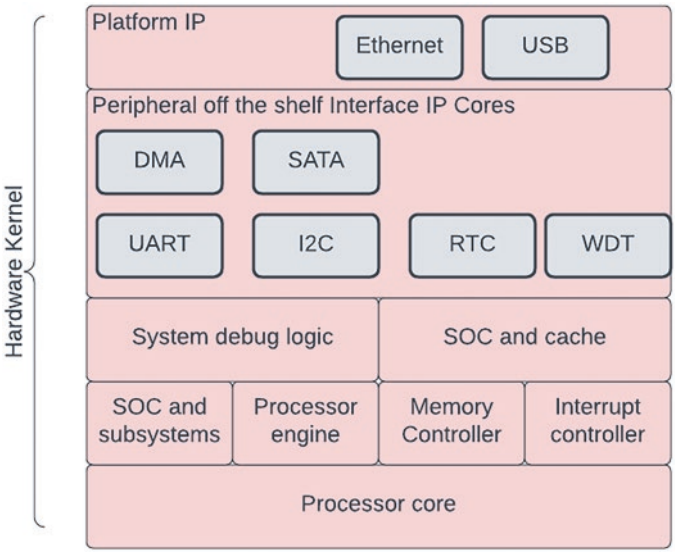


Fig. 2.4 SOC and IP platforms

interrupt controller and bus controller for off-chip devices such as ROM/RAM memories, and L1/L2 cache memories to form processor subsystem. The entire platform will be softcore which can be easily adapted to any standard EDA design flow with required customization. There will be regular support from software developers on the software platform components.

Figure 2.4 shows the full range of SOC and IP platform components.

It is a framework for a hardware processor-based IP platform that comprises reusable, software and hardware-supported subsystems that solution developers can integrate into their SOC designs.

2.5 System Fabrication

Electronic systems are developed by integrating many circuits using different fabrication technologies such as PCBs, monolithic, thick film, and thin film technologies. The circuits are fabricated on different base materials such as alumina or ceramic substrates and silicon wafers over the last couple of decades apart from printed circuits. In PCBs, the circuits are printed on an FR4-glass epoxy-insulating substrate. FR4 is a standard glass-reinforced laminated epoxy substrate material. With phenomenal advancement in integrated circuit (IC) processing technologies, semiconductor process technology has become the dominant technology over the past seven decades for circuit realization. CMOS (complementary metal oxide semiconductor) is the most used VLSI technology for IC fabrications for electronic

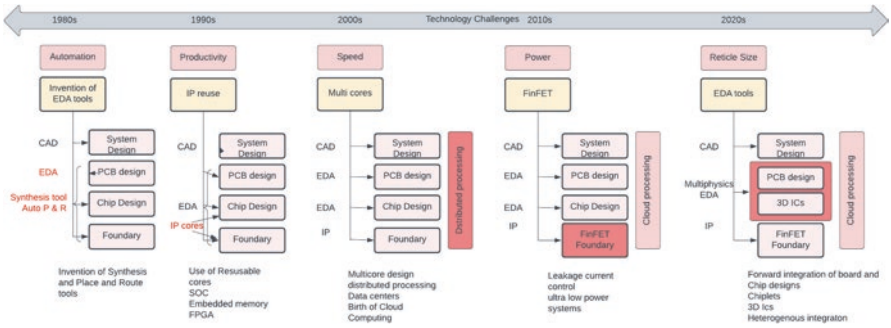


Fig. 2.5 Circuit fabrication technologies evolution over decades

circuits and systems. Figure 2.5 shows the history of technology challenges and how they have been overcome in circuit fabrication technologies over time.

As it is seen, due to the manual process of circuit design during the eighties, small circuits were designed to be on hardware for time-critical functionalities. With the invention of EDA tools such as synthesis, the design complexity of circuits grew, and automation in design technologies boosted this development in designing larger circuits. On the other hand, the scaling of CMOS transistor structure led to the phenomenal growth of processing technologies in integrating high density of transistors to the extent that despite design automation, there was a design-productivity gap in the decade of 1990 to convert the excess transistors to functional logic. This was due to the limitation of skilled designers and design methods. This was overcome by IP reuse and integration strategies in the development of systems. It is the growth in design automation tools, fabrication processes, and everlasting trust in easy-to-use system applications that led to the development of the complex System on Chip (SOC) designs and development. With design strategies such as the reuse of intelligent property (IP) cores, it was possible to develop systems by integrating many types of cores, such as processors, memory, and analog cores, thus reducing the design productivity gap.

The continued scaling resulted in power challenges in system design in the 2000s. Methods to limit power consumption have led to the invention of FinFET technology in the decade of 2010. This technology continued to flourish and went on to become the dominant technology. Also, as process technology, fueled by the phenomenon of scaling of transistors, SOC design methodologies with more sophisticated EDA tools were developed. This enabled the design of complex systems on the chip comprising hundreds of processors, protocol blocks, many interface cores, on-chip sensors, analog cores, and RF modules. Integrating many cores required well-defined architectures to operate reliably with optimum performance. This also demanded distributed development with high computing resources and sophisticated EDA tools using distributed processes. This led to the genesis of cloud development solutions in the previous decade. Unlimited access to computing power helped us to visualize systems in a large perspective. Continued advancement in design tools, backward and forward integration possibilities are

paving ways for development of heterogenous systems with 3D ICs using chip-lets, and larger system-level solutions. The current decade will see development approaches with integrated frameworks where the challenges of board, chip, package, and process technologies are addressed cohesively to develop optimum application-specific larger and complex system solutions.

2.6 Domain-specific SOC Architectures

In this section, some domain-specific SOC architectures are discussed.

2.6.1 IOT Architecture

Due to the wide diversity of sensors, there is no one-size-fits-all architecture for IoT SOCs. However, some of the components can be reused from one IOT SOC to the other IOT SOC. Some of the architectural considerations are

- Scalability: Scalability in terms of the amount of data collected from the use environment over time decides the definition of the architecture in the long run.
- Availability: IOT SOC architecture has to ensure that it is highly available at any given point of time. These devices cannot afford to fail as they usually work with real-time data. Failure of an IOT device means loss of data which could have fatal consequences.
- IOT device architectures have to be flexible enough to accommodate quick and frequent changes. IOT architectures should support add-on features without breaking existing architecture.

The best part of IOT architecture is that it can be defined as consistent layered functions across different IOT applications. Three or five-layer IOT models are quite popular. The three-layer IOT architecture is a great way to define the IOT device functionality. The three layers as shown in Fig. 2.6 are the following:

2.6.1.1 Device Layer

Device layer is also called Thing or Perception layer. It is a complete physical hardware component. This consists of the actual device with multiple sensors and actuators. This layer is responsible for interacting with the environment for sensing, capturing, and controlling the parameters/other things. This can be a physical device like cameras, simple sensors, electrodes, etc. This is often referred to as the thing in the Internet of Things. The thing can sometimes be smart enough to capture, process and communicate onto the internet directly through wired or RF links. This typically happens through gateways. Such devices or things are referred to as edge devices.

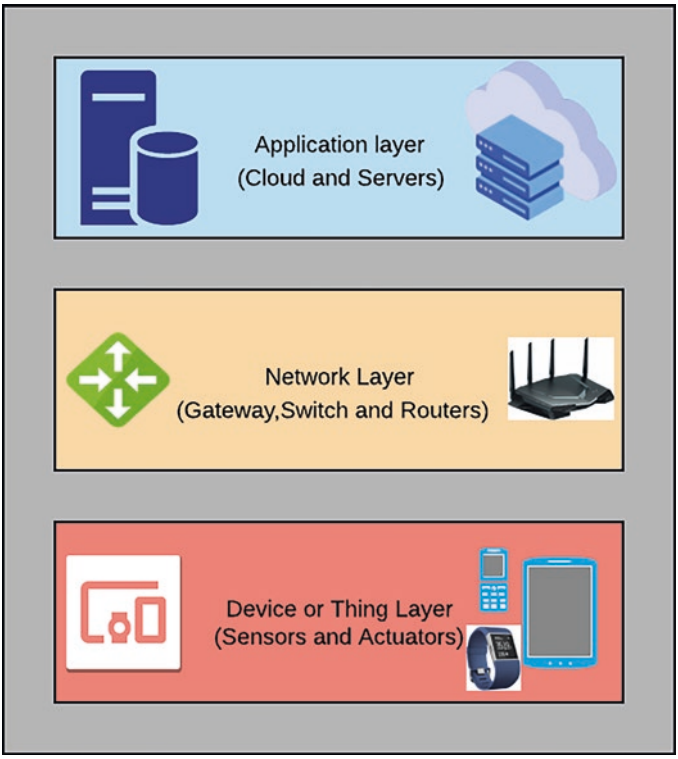


Fig. 2.6 Three-layer IoT architecture

When a thing has the capability to communicate on the internet, it is called an edge device. So, while taking up the IoT design, it is essential to understand the requirements of functionalities to be implemented in the device layer. Most devices of today are IoT-ready with M2M communications through Bluetooth, Wi-Fi, or ethernet functionalities. But the device features with sensors and actuators have to be customized for target applications. The differentiation among things comes from the type of sensors/actuators the device supports.

2.6.1.2 Network Layer

In the IoT architecture, this layer connects various devices on the internet to collect the data for backend processes or services. It describes the amount of data that gets processed in the application. The device access to the internet is through gateways or switches, or routers. This layer is a standard network layer but optimized to collect and distribute data sets generated by the thing and device layer. Network layer elements serve as messengers between devices and the cloud layer on the internet where actual applications can be developed. There are physical devices or

software programs that typically run from the field in close proximity to the edge sensors and other devices. Large IoT systems might use a multitude of gateways to serve high volumes of edge nodes. They can provide a range of functionality, but most importantly, they normalize, connect, and transfer data between the physical device layer and the cloud. In fact, all data moving between the cloud and the physical device layer goes through a gateway. IoT gateways are sometimes called "intelligent gateways" or "control tiers." Today, gateways also support additional computing and peripheral functionality such as telemetry, multiple protocol translation, artificial intelligence, pre-processing and filtering massive raw sensor data sets, provisioning, and device management. It is becoming common practice to implement data encryption and security monitoring on the intelligent gateway so as to prevent malicious attacks by the middle man against otherwise vulnerable IoT systems. Certain gateways offer a real-time operating system that is specialized for use in embedded and IoT systems along with optimized low-level support for different hardware interfaces with supporting interface libraries. Library functions for managing memory, I/O, timing, and interface are supported as functional tasks in the library. Libraries are typically available based on standard protocols. They are not targeted for a particular sensor or actuator which are required for IoT.

2.6.1.3 Application Layer

The cloud or data server is the application layer that communicates with the gateway, typically over wired or cellular ethernet. It is a powerful server system with databases that enable robust IoT applications and integrate services such as data storage, big data processing, filtering, analytics, 3rd party APIs, business logic, alerts, monitoring, and user interfaces. In a three-layer IoT architecture, the "Cloud" is also used to control, configure, and trigger events at the gateway and, ultimately, the edge devices. This can also be a human interface or an application to control a device such as a smart home ecosystem or a dashboard showing the status of the devices in the system.

Though the three-layer architecture is a great way to describe an IoT project, it is somewhat limited in scope. For this reason, new proposed architectures are updated to have different or additional layers. A popular one is called the five-layer architecture, which includes Transport (replacing the Network), Processing, and Business layers, in addition to the Perception and Application layers from the three-layer architecture model.

Figure 2.7 shows the functional layers of an IOT architecture.

In addition to the three-layer architecture discussed you will usually see the following three layers:

- Transport: This layer describes the transfer of data between the sensors in the Perception layer and the Processing layer through various networks.
- Processing: Sometimes referred to as the Middleware layer, this one stores, analyses, and pre-processes the data coming from the Transport layer. In modern

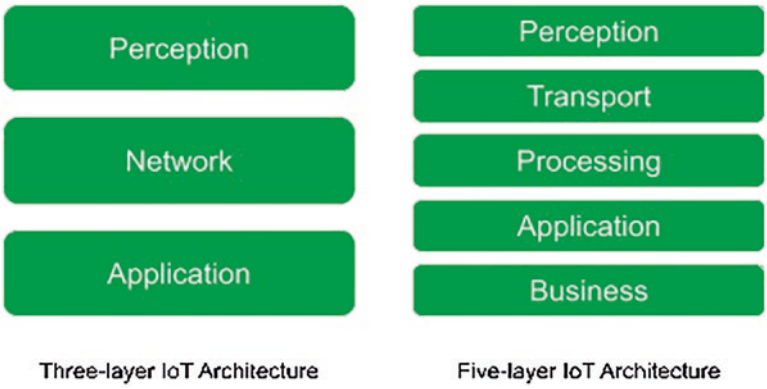


Fig. 2.7 Functional layers of IOT architecture

software applications, this is often located on the edge of the cloud for low-latency communications.

- Business: This layer is often referred to as the Business Intelligence layer. Located at a higher level than the Application layer, the Business layer describes everything that has to do with the application users. Data-based decision-making is done here based on the data found and consumed at the Application layer.

2.6.2 Different Stages of IoT Architecture

Another way to describe an IoT solution architecture is by using a four-stage approach as shown in Fig. 2.8. This architecture describes the various building blocks that constitute the IoT solution. In this scenario, more emphasis is put on edge computing than the other proposed designs.

- Devices: This stage is about the actual devices in the IoT solutions. These devices could be sensors or actuators in the Perception layer. Those devices will generate data (in the case of sensors) or act on their environment (in the case of actuators). The data produced is converted into a digital form and transmitted to the internet gateway stage. Unless a critical decision must be made, the data is typically sent in a raw state to the next stage due to the limited resources of the devices themselves.
- Internet gateways: The internet gateway stage will receive the raw data from the devices and pre-process it before sending it to the cloud. This internet gateway could be physically attached to the device or a stand-alone device that could communicate with sensors over low-power networks and relay the data to the internet.

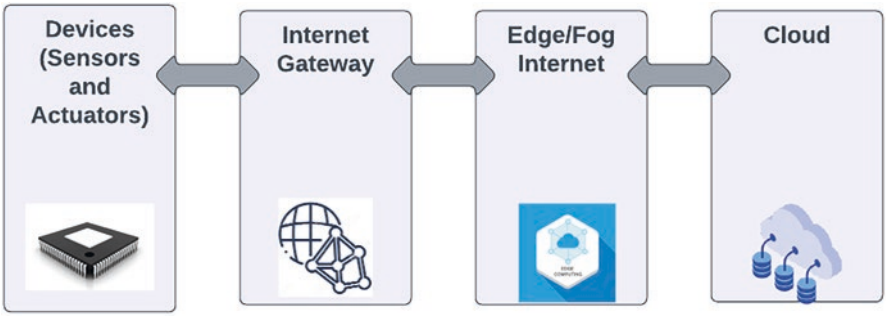


Fig. 2.8 Four-stage approach of defining IOT systems

- Edge or fog computing: In order to process data as quickly as possible, you might want to send your data to the edge of the cloud. This will let you analyze the data quickly and identify if something requires immediate attention. This layer typically would only be concerned with recent data that is required for time-critical operations.
- Cloud or data center: In this final stage, the data is stored for later processing. The application and business layers live in this stage, where dashboards or management software can be fed through the data stored in the cloud. Deep analysis or resource-intensive operations such as machine learning training will happen at this stage.

  Figure 2.9 shows the functional components of IOT SOC architecture

2.6.3 Digital Signal Processors

Digital signal processors are one of the most used SOC subsystems in applications where multimedia processing or physical signal processing is carried out. It consists of extracting the useful properties of signals and applying appropriate mathematical operations.

2.6.3.1 DSP Architectures

System on Chips require signal processing functionalities in many applications. DSP processors are used for these features. The architectures of the digital signal processors are:

- Von Neumann Architecture.
- Harvard Architecture.
- Super Harvard Architecture.

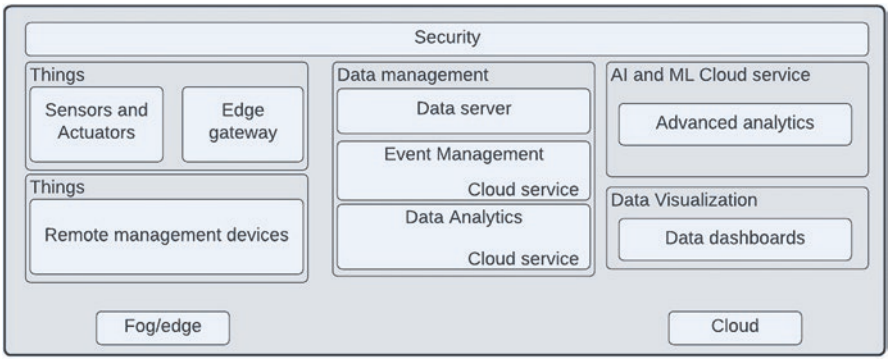


Fig. 2.9 Functional components of IOT architecture

Von Neumann Architecture

Von Neumann architecture of a digital signal processor mainly includes a single memory and a single bus which are used for data transferring into and out of the CPU (central processing unit). Multiplying any two numbers needs at least 3 CLK cycles, where one CLK cycle is used to transmit each of the 3 numbers from the memory to the CPU with the help of the bus.

We don't calculate the time taken to transmit the output back to memory, as we assume that it will stay within the central processing unit for extra manipulation. This type of architecture is quite suitable when you are satisfied to perform all of the necessary tasks in serial. At present, most computers use Von Neumann architecture but other architectures simply need where very fast processing is necessary.

Figure 2.10 shows the generic structure of Von Neumann architecture.

Harvard Architecture

The name Harvard Architecture is taken from the work finished at Harvard University in the 1940s under the Howard Aiken leadership. As shown in this design, it includes two separate memories for both the data and program instructions including separate buses for each. When the buses work independently, then data and program instructions can be fetched together to improve the speed over the single bus. At present, this dual bus architecture is used by DSPs.

Figure 2.11 shows the generic structure of Harvard architecture.

Super Harvard Architecture

The super Harvard architecture of a Sharc DSP is shown in Fig. 2.12.

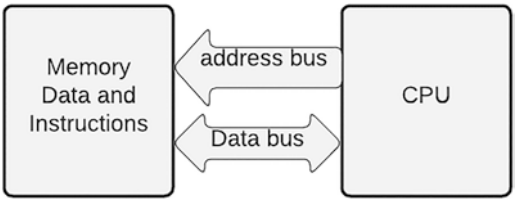


Fig. 2.10 Von Neumann architecture

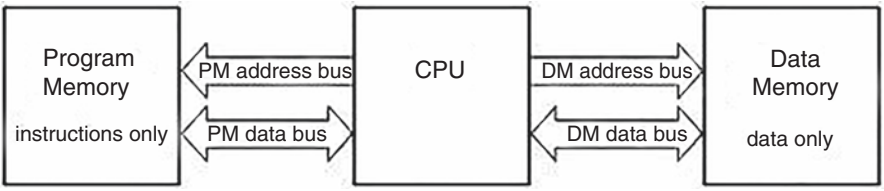


Fig. 2.11 Harvard architecture

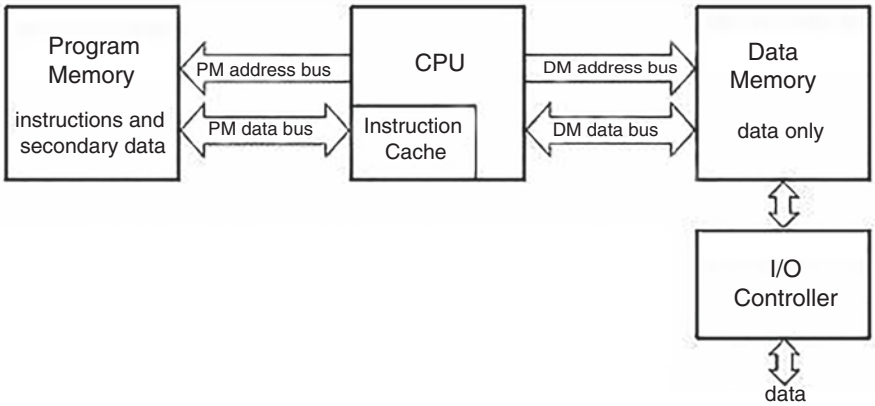


Fig. 2.12 Super Harvard architecture

This architecture was implemented by including some features to increase the performance such as throughput or instruction and an IO controller.

2.6.3.2 Types of Digital Signal Processors

Digital signal processors are available in two types: fixed-point processors and floating-point processors.

Fixed-Point Digital Signal Processor

In a fixed-point digital signal processor, every number can be specified through a minimum of 16 bits, even though a different length can be utilized. The number can be represented with different patterns. The fixed-point means that the fractional point position can be assumed to be fixed and to be identical for the operands as well as the operation result.

Floating-Point Digital Signal Processor

Floating-point digital signal processors mainly use a minimum of 32 bits to store every value. The distinct feature of floating-point DSP is that the signified numbers are not spaced uniformly. Floating-point digital signal processors can simply process the fixed-point numbers, a requirement to implement counters and signals which are received from the analog-to-digital converter and transmitted to the digital-to-analog converter. The programs of floating-point DSPs are simple, however, they are normally very expensive and have high power consumption.

2.6.3.3 DSP Memory Architecture

For memory management, conventional DSPs use Von Neumann architecture where equal memory is utilized to store both the data and the program. Even though this is simple, the architecture uses a number of processor cycles to execute a single instruction because the same bus is utilized for both program and data (Fig. 2.13).

In order to enhance the operation speed, separate memories were utilized to store both the program and data. A separate set of address and data buses are provided to both memories, so this architecture is known as Harvard Architecture (Fig. 2.14).

Harvard Architecture

Even though the separate memories utilization for data and instruction will increase the processing, it will not solve the problem totally. Because many DSP instructions need above one operand, utilization of single data memory can lead to getting the operands continuously, so the processing delay will be increased. This issue can be

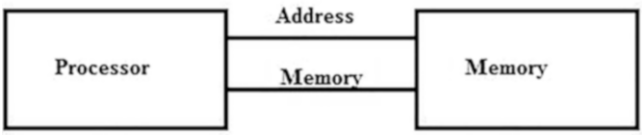


Fig. 2.13 Von Neumann architecture

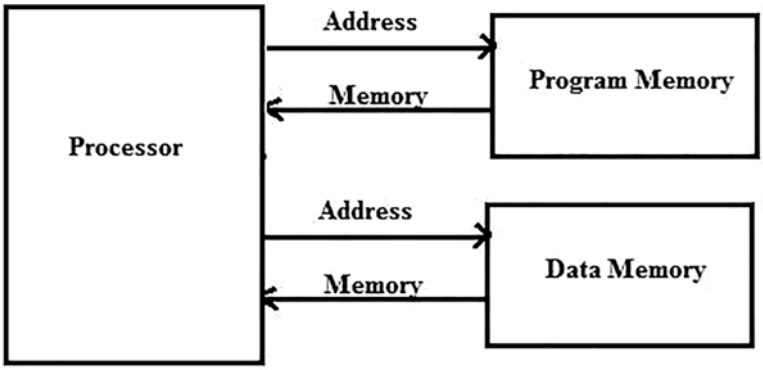


Fig. 2.14 Dual memory Harvard architecture

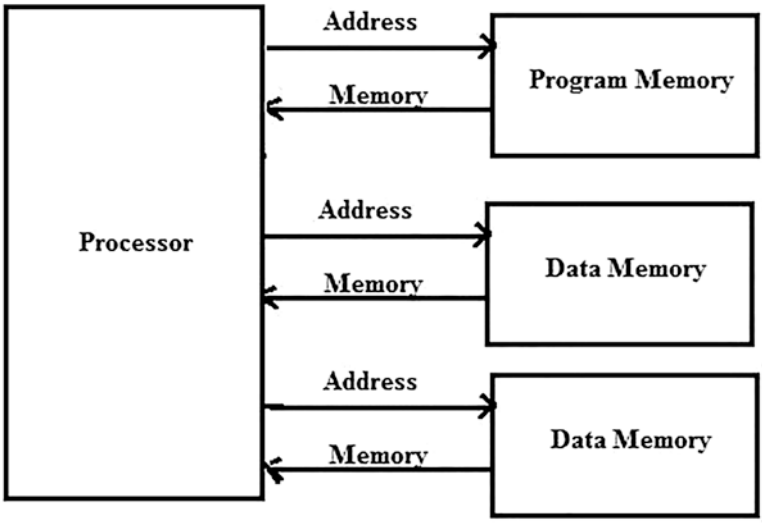


Fig. 2.15 Harvard architecture with dual data memory

solved with two separate data memories to store operands individually, therefore in a single CLK cycle, both the operands can be simply fetched mutually (Fig. 2.15).

Even though the above architecture enhances the operation speed, it needs more hardware and interconnections, and the complexity and cost of the system can be increased. As a result, there should be an exchange between the speed and cost when choosing the memory architecture of a digital signal processor.

2.6.3.4 Difference Between Digital Signal Processor and Microprocessor

The difference between digital signal processors and microprocessors includes the following:

Digital Signal Processor	Microprocessor
It is a specialized microprocessor chip	It is a computer processor
DSPs are extensively used in telecommunications, audio signal processing, digital image processing, etc.	Microprocessors are used in PCs for text editing, computation, multimedia display and communication over the internet.
In DSP, instruction can be simply executed in a single CLK cycle.	The microprocessor uses several clock cycles for one instruction execution.
Parallel execution can be achievable	Sequential execution is possible.
DSP is suitable for the operation of array processing.	It is suitable for general-purpose processing.
Addressing modes used in this processor are direct and indirect.	Addressing modes used in microprocessors are direct, immediate, register indirect, indirect register, etc.
Address generation can be possible by combining program sequencers and DAGs.	The program counter or PC can be incremented to produce an address sequentially.
It includes three separate computational units: MAC, ALU, and Sifter.	It includes simply the main unit like ALU.
The program flow can be controlled by an instruction register and program sequencer.	Program counter can control the execution flow.
It includes separate data and program memories.	It doesn't have separate memories.
In DSP, several operands are fetched at once.	In a microprocessor, the operand can be fetched serially.
In DSP, address and data bus are multiplexed	In a microprocessor, address and data bus are not multiplexed.

2.6.3.5 DSP Advantages and Disadvantages

The **advantages of a digital signal processor** include the following:

- Less overall noise.
- Support for error detection and correction.
- Simple data storage.
- Digital signals are simple to encrypt.
- Transmission of data is possible.
- In digital processing systems, modifying program code is easy.
- DSP systems work through a broader range of frequencies.
- In a digital system, the DSP can be cascaded without any loading issues.
- The operations of DSP can be simply changed by altering the program within a digital programmable system.
- By using the DSP method, a complicated signal processing algorithm can be simply implemented.

- DSPs are lightweight and more compact.
- DSP systems are upgradeable because they are controlled by software.

The **disadvantages of a digital signal processor** include the following:

- Digital communications need high bandwidth to transmit the data as compared to analog.
- Most of the digital signal processors are expensive.
- The DSP system complexity will be increased due to the usage of additional components.
- Digital signal processor uses several transistors which consume more power as compared to analog signal processors.
- The hardware architecture and software instructions of each DSP are different so required highly skilled engineers to program the device.

2.6.3.6 DSP Applications

The **applications of a digital signal processor** include the following:

DSP applications mainly include processing of audio and speech, radar, sonar and other sensor array processing, statistical signal processing, spectral density estimation, data compression, digital image processing, audio coding, video coding, image compression, signal processing for control systems, telecommunications, seismology, biomedical engineering, etc.

Thus, a digital signal processor is a specialized microprocessor. The main function of this processor is to measure, and compressor filter analog signals. Usually, digital signal processors have better power efficiency, so they are mostly used in portable devices like mobile phones due to power utilization constraints. These processors frequently use special memory architectures to fetch multiple data instructions simultaneously.

Chapter 3
System on Chip (SOC) Architecture



3.1 System on Chip (SOC) Architecture

A SOC architecture is an abstract structure of system elements implemented through technologies meeting a set of traceable requirements and supported by life cycle concepts (operational and support). The system elements can be hardware or software. A SOC architecture is defined based on principles, concepts, and logically related properties that are consistent with each other. It is generic and may be applied to more than one SOC, which forms a common pattern, structure, and class, enabling reuse. The intangible notion of system elements primarily defines the functional and physical structure of the system elements. Recent practices extend the architecture to define the behavioral, temporal, and other dimensions. ISO/IEC/IEEE 42010:2022 addresses the creation, analysis, and sustainance of architectures of systems using architecture descriptions. There are attempts to develop standards and apply a systematic approach to characterizing architecture belief systems in systems engineering. These standards provide a framework that contributes to the consistent approach to correct implementation of the architecture with proper governance and responsibility benefiting developers for proper compliance to the requirements.

3.2 Architecture Processes and Their Dependencies

The core architecture processes and their key interactions are shown in Fig. 3.1. There are other life cycle processes that are affected by these architecture processes. For example, system requirements may be derived from the architecture and the architecture may be driven by requirements. The specific nature of how architecture and requirements are related to each other is organization dependent. If the solution

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

V. S. Chakravarthi, S. R. Koteshwar, *System on Chip (SOC) Architecture*,
https://doi.org/10.1007/978-3-031-36242-2_3

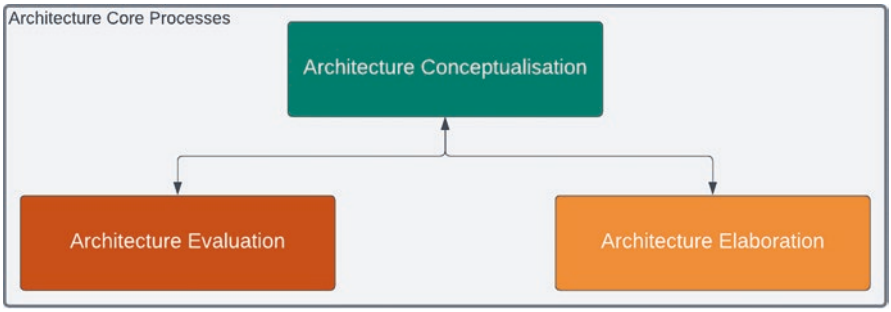


Fig. 3.1 Architecture processes and their interactions

is for a new problem, then the architecture is requirement dependent. If the solution is an alternative solution or an updated solution, then the requirement is derived from the architecture. Architecture processes are also dependent on the risks, supply chain logistics, verification, etc., which are more logistical managerial in nature. These processes can also be triggered by other processes external to these three processes. Conceptualization defines the objectives of the architecture and the quality measures that can be used in evaluating the architecture. Architecture value is defined in terms of the extent to which stakeholder concerns are addressed. These architecture objectives are based on the problem/opportunity identification and definition that occurs in this process. Architecture concepts are generated with value in mind and are then assessed using these quality measures.

Conceptualization helps system architects in characterizing the problem space, identifying potential solutions, mapping to potential system architectures, and expressing these architectures in a form that is suitable for the intended uses. The name of the process being "conceptualization" does not mean that the results are necessarily at the "conceptual" level or consist of a set of conceptual models. The results could include a "logical" architecture or a "physical" architecture, depending on the type of the solution. During the early stages, it can be important to be agile and quick in conceptualizing many alternative architectures. Some of these early architectural descriptions are little more than drawings or block diagrams. After several quick rounds of feasibility evaluation and conceptualization, there can then be a smaller number of viable architectures that are worth capturing in a more complete form and saving them in the work repository for later use.

A more complete form of architecture description would be developed during architecture elaboration. Conceptualization can use the results of architecture elaboration, when appropriate. Architecture conceptualization only needs to describe the architecture to the level of specificity and granularity that is suitable for its intended users, which in many cases does not require significant elaboration. The elaboration of architecture views, models, and descriptions can often occur later in the life cycle of the architecture, after the architecture has become more mature and the extra effort of elaboration becomes necessary to confirm the assumed hypothesis used during conceptualization. Elaboration can often be deferred until after several

architecture alternatives have been examined for their feasibility, both developmental and solution. Elaboration is also used to examine the engineering resource capability and effort.

During evaluation, alternative architectures are checked for both implementation and solution feasibility to narrow down to one or two most appropriate architectures. However, in some cases there can be only a single feasible architecture. Value assessments can be based on analysis of relevant architecture attributes and properties of the situation or on an assessment of the extent of the problem solved. The assessment and analysis results, along with estimates of assessment uncertainty, are returned along with key findings and recommendations to determine if the proposed architecture is sufficiently suitable. If not, then additional iterations of conceptualization and evaluation are carried out. During evaluation, there can sometimes be a need for more complete models and views. In these cases, elaboration could be requested to generate additional models and views. During evaluation, these models and views are annotated with the results of the evaluation and with comments on strengths and weaknesses of the architecture or its description. Any of the architecture processes can generate or contribute to an architecture description of varying scope, granularity, and formality. However, the elaboration process captures the architecture description in a sufficiently complete and correct manner for the system designers, implementors, and intended users of the architecture downstream.

3.3 SOC Architecture and SOC Design

Architecture processes provide one or more architecture alternatives that frame the concerns of stakeholders and addresses their key requirements. Architecture can be widely used across the life cycle processes of the architecture entity. Architecture processes can be applied at many levels of abstraction, highlighting the relevant features that are necessary for the decisions at that level. As defined, architectures provide the fundamental concepts or properties of the architecture entity and associated governing principles. Architectures should be described using a set of views and models that are complete, consistent, and correct. The completeness of an architecture model is determined relative to its intended use. Design, as defined in ISO/IEC/IEEE 15288, is a technical process providing sufficient details about the architecture entity and its elements to enable an implementation consistent with architecture. An effective architecture guides the design activities in such a way that it allows for maximum flexibility in the design. During the initial design stages, insights are gained into the relation between the requirements specified for the architecture entity and the emergent properties and behaviors of the architecture entity that arise from the interactions and relations between the elements and between the properties of those elements. Sometimes the elements of the architecture entity are initially notional until the actual design(s) stage. Sometimes a “reference architecture” is created using these notional elements to convey architectural intent

and to check for design feasibility. Interfaces and interactions between elements are defined at a level of detail necessary to convey the architectural intent and could be further refined in the design. The design definition process considers any applicable technologies and their contribution to the system solution. Design provides the level of the definition necessary for realization, such as block diagrams, detailed design descriptions, software code, task descriptions, etc. The design process provides feedback to the architecture processes to consolidate or confirm the concepts and properties of the architecture entity, along with the allocation, partitioning, and alignment of architectural entities to elements that compose the architecture entity.

3.3.1 Requirement Capture

In most cases, the requirement arrives from market research or technology trends. Market research leads to market requirement documents with identified problems and probable solutions. An example is the smartphone system development. As digitalization and internet technology proliferated the lives of people, the need for powerful easy-to-use cellular phones became a requirement, and system developers started upgrading the systems targeting smartphones. Another enabler for the need to find solutions is the technology trend such as natural progression. One example is generations of cellular technologies 2G, 3G, 4G, 5G, etc. Once the commercialization of one technology is in progress, technology development is taken up by the developers for new generations. This is the first step in identifying marketing requirements as a market requirement document (MRD). MRD also contains business cases, which are essentially needed for commercialization of the solution, risks involved, form factor, and time to market figures. It even includes the commercials such as the cost of development and the potential profit margin and business feasibility. This becomes the starting trigger for the feasibility study of the design and development plan of the system. The engineering team uses this to detail the architecture of the system with more specifics of the target technologies, size, packages, power budget, and resource plan.

3.3.2 Types of SOC Architectures

3.3.2.1 Computing SOC Architectures

System architectures depend on the potential solutions for the identified problem definition. For example, the solution architecture of the system for computing problems is much different from an automotive system which is much different from an IOT system. These systems differ for sure in functionality and even in setting the design goals which determine the system architecture. Different system architectures address a range of applications, performance requirements, power

levels, and price points. Reuse is the major factor considered in defining architectures across generations of systems. Each new generation of system architecture is a superset of its predecessors, providing backward compatibility with older systems and older software while also adding new or enhanced features. This compatibility allows engineers, programmers, and development teams to reuse the software and software development tools from earlier projects, protecting their investment in time and talent. It also makes developing new systems easier by leveraging developers' experience. Software is reused across generations of products, and product teams can protect their investment (in both hardware and software) in a cost-efficient manner. Although it is required to develop part of new software to take advantage of the latest features, the old software will still work as-is. For example, Intel architecture SOCs have undergone many changes over the past five decades. If you look at their solutions, it is very easy to identify the commonness in architecture, which is even referred to as Intel® architecture. Although every branch of the broad Intel architecture (or x86) family tree retains the same basic features and functionality as the earlier chips, and retains backward compatibility with them, each new generation also adds its own unique features to the system. In their chips, you can notice how the multimedia extensions (called MMX™ technology) that accelerated audio and video processing added in Intel Pentium processor, extensions of MMX technology with more streaming-media capabilities (Streaming SIMD Extensions (Intel® SSE) and Intel® Streaming SIMD Extensions 2 (Intel® SSE2), the addition of Floating-point units (FPUs), and today encryption/decryption extensions, power-management features, and multilevel caches on most Intel architecture processors. Data paths have widened from 8 bits to 32 bits, 64 bits, and even 128 bits and more. Operating frequencies have increased from a few megahertz to 2 GHz and more. Typical system architecture based on Intel's Pentium is shown in Fig. 3.2.

You can also observe the same trend on Intel's atom processor systems. Since these system architectures are defined using the same base architecture and extensions, they are easy to scale up for functionality and performance. Scaling up the chip performance is easy and is done simply by adding one or more cores or multiple threads. Multiple cores integrate many instances of the processor core structures in the same chip to boost performance and multi-threading is a technique that allows a single processor architecture core to perform multiple tasks. In both cases, power consumption and heat dissipation are kept under control by advanced power management blocks on the chips, and is managed by software and manufacturing technology.

All high-performance systems are developed using two chip methodology which has one core processor chip and a second support chip which is called peripheral control hub (PCH). The first one offers performance and second one the flexibility to expand the interfacing capability. In addition to these two system components (i.e., processor and PCH) will be other third-party components, including DRAM, a boot ROM, a power supply, and the peripheral interfaces appropriate for the complete system, that can be a network or sensor connection. Most systems will also include some non-volatile memory (e.g., flash or E2PROM), and perhaps some "glue logic" that is specific to the application.

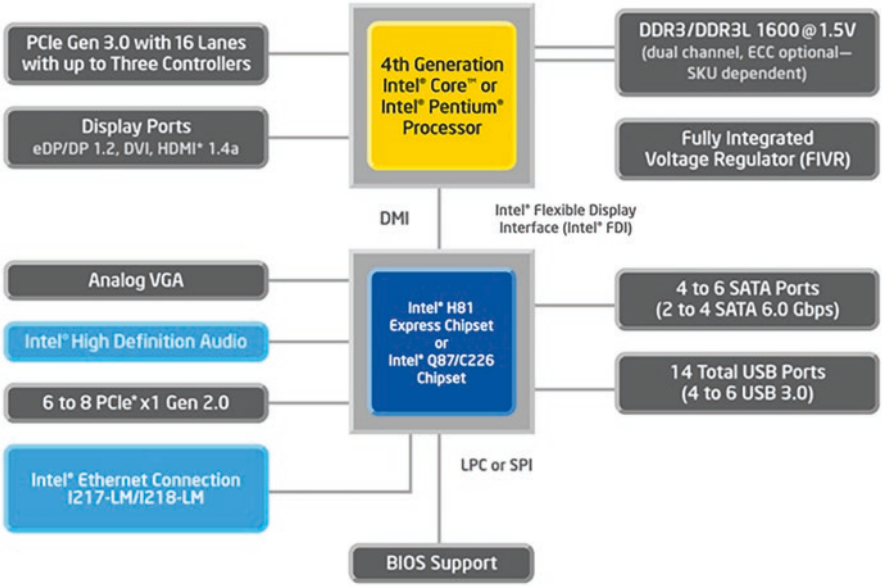


Fig. 3.2 System architecture based on Intel's Pentium processor. (Source: Intel® Core™ i7 processor)

3.3.2.2 IOT SOC Architectures

Internet of Things (IOT) systems have the ability to sense physical parameters around them, process and communicate them to the cloud server for further analysis or control other parameters around them. The systems targeted for IOT application must be energy efficient, interoperable, secure vulnerable information, and have low response times. These systems find a variety of applications ranging from smart home control, automotive, smart appliances, energy meters, and many more. The ***"Things"*** must be interoperable with other devices compliant with the same standards which makes it a high-value-added device. ***Security*** is another important requirement of the IOT SOCs. There are three common IoT system architectures, including application processors (high-end IoT), microcontrollers (low-end IoT – Micro Controller Units: MCU) and smart analog, each with their own set of IP requirements and functional advantages. These architectures are typically designed on matured standard process technologies to save costs and leverage integration of analog, wireless power management, and non-volatile embedded memory. High-end applications include feature-rich wearable devices, where more advanced process technologies of 16nm or 28nmLP are considered to achieve low power consumption, high signal processing, and low cost. IOT system architectures also include smart analog solutions with on-chip power management and sensors. The availability of software IPs is another consideration for IOT SOC architecture.

Figure 3.3 shows a typical base architecture of IOT systems.

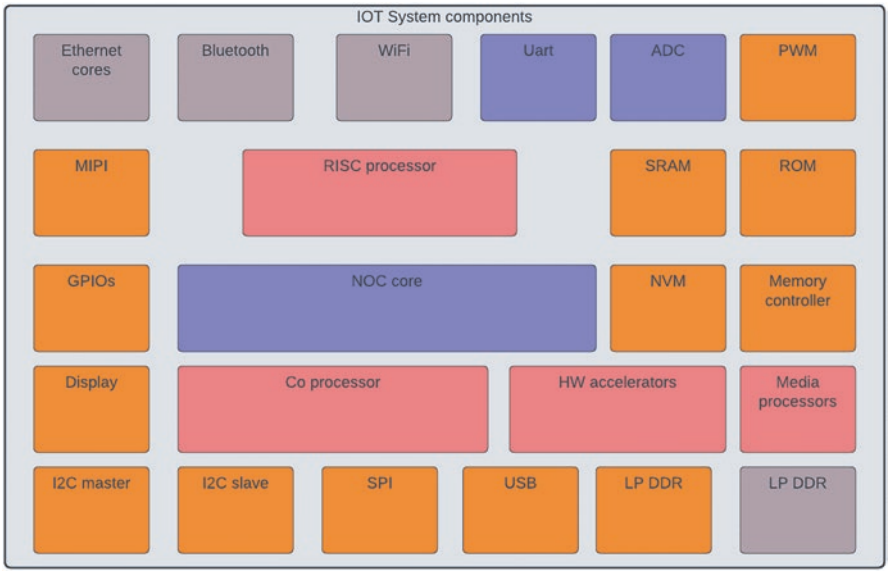


Fig. 3.3 IOT SOC architectures{TBD: Make it generic}

3.3.2.3 Automotive SOC Architectures

An automotive SoC is required to be reliable and safe throughout its lifetime in a dynamic environment of the use of automobiles. This requirement poses many architectural challenges. The major challenge is to ensure that the designs are validated in such environments and also in extreme operating conditions. For safety and reliability, these designs are expected to work for the lifetime usage of automobiles. Thus, SOC architectures are to be defined such that the designs meet these high automotive reliability requirements. The design architecture must be validation friendly to test the designs and devices at all stages of development. This ensures design defects do not escape to the applications. These designs are to be validated for dynamic electrothermal analysis, which is carried out for automobiles. The aging tests are important for these designs at all stages for ensuring safety and reliability.

Vehicles are often called “drivable computers.” The automotive SOCs are used in a wide range of automotive applications. They provide drivers and passengers safety with advanced danger avoidance systems using video recognition capability based on cutting-edge embedded Artificial Intelligence (AI) technologies and support the different levels of safety as per ISO 26262 up to automotive safety integrity level D (ASIL D). These SOCs use a number of networking MCUs which can acquire large amounts of data from sensors and actuators, process them and assist drivers with advanced navigation guidance, and in-vehicle infotainment with driver-friendly cockpit and dashboards.

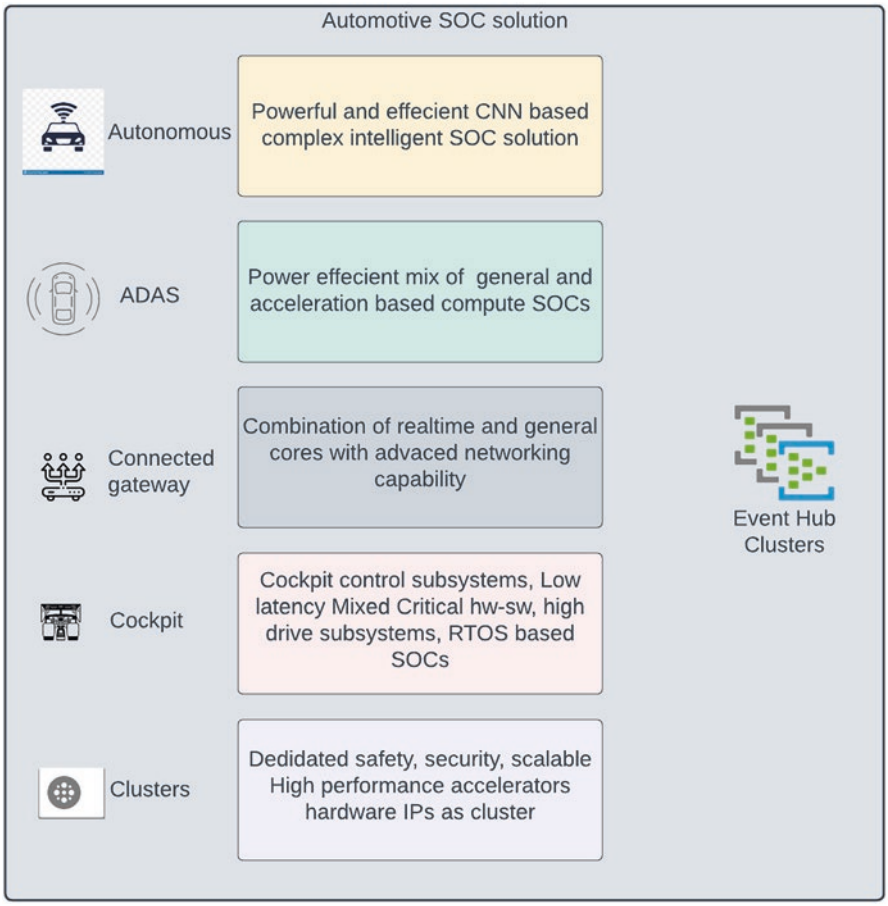


Fig. 3.4 Typical automotive SOC architecture {TBD: make it generic}

The automotive SOC architecture must be defined considering the above requirements and with the capability of testing for defects even when in use, as these are safety-critical applications. The system hardware architecture is to be scalable with flexible software to cover a wide range of applications. Figure 3.4 shows the typical automotive SOC architecture.

3.4 Approach to Defining SOC Architecture

Most of the computing SOCs have ALU as the basic building block. It performs all arithmetic and logic operations on data and manipulates addresses for pointer arithmetic needed for various scientific and engineering applications. ALUs

typically perform arithmetic operations such as addition, subtraction, multiplication and division, and logic operations such as AND, OR, NOT, XOR, and XNOR. The latency in these operations depends on the implementations of these circuits. ALUs are implemented using standard logic gates, such as AND, OR, INVERTER gates and Multiplexers and some Macro cells optimized for performance. These building blocks work with individual bits, but the actual ALU works with multiple bit data inputs stored in registers, for example, 32-bit registers to perform a variety of tasks such as arithmetic, logic, and shift operations. The performance of ALU depends on the degree of parallelism and the operand arrival latencies at the ALU blocks in the implementation. For example, in multiple bit adder circuit, adder of higher bits requires carry generated in the previous stages and hence depends on the availability of carry bit from the completion of previous stage adders. ALU circuits for arithmetic and logic operations are available as resources for the supported instructions. The final multiplexer decides the result from the collection of functional units operating in parallel depending on the chosen instruction. Important things to remember about ALUs are:

- All of the gates are working in parallel.
- The speed of a gate is affected by the number of inputs called the degree of fan-in.
- The speed of a circuit depends on the number of gates in the longest computational path through the circuit (this can vary per operation).

Finally, architectural organization affects performance, just like better algorithms improve efficiency in software.

In a computing SOC, ALU is the main block which is seen as a brain of the processor. ALU is responsible for performing fetch (opcode fetch), Decode (decoding the opcode to enable relevant Arithmetic and logic engine), and execute (perform operations on the operands) operations. A set of opcodes make an instruction set. The type of instructions forms class of instructions referred as instruction class. The architecture is referred as instruction set architecture (ISA). In these SOCs, we typically have the following components:

- *Processor (CPU)* is the active part of the computer, which does all the work of data manipulation and decision making.
- *Datapath* is the hardware that performs all the required operations, for example, ALU, registers, and internal buses. The datapath design involves

  1. Determining the instruction classes and formats in the ISA,
  2. Design datapath components and interconnections for each instruction class or format.
  3. Compose the datapath segments designed in Step 2 to arrive at the composite datapath. Simple datapath components include memory that stores the current instruction, Program Counter (PC) that stores the address of current instruction, and ALU that executes the current instruction. The interconnection of these simple components to form a basic datapath is illustrated in Fig. 3.5.

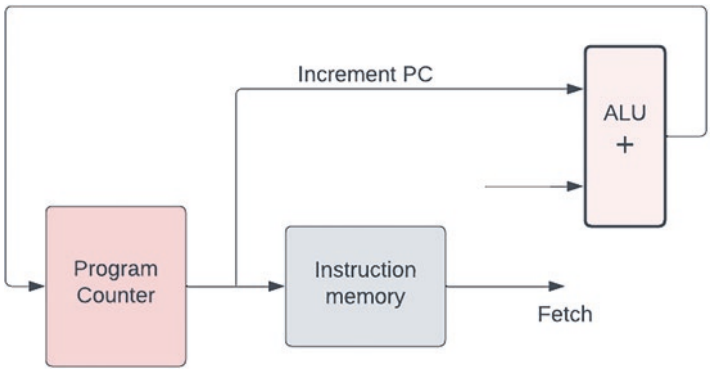


Fig. 3.5 Simple datapath

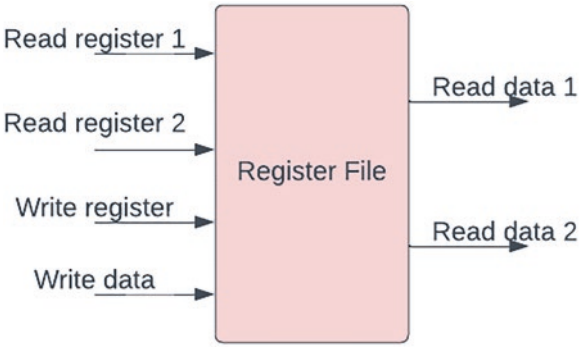


Fig. 3.6 Register file which is embedded storage

The output or result from an ALU is written into the register file as shown in Fig. 3.6 The register file (RF) is the on chip storage that has two read ports and one write port corresponding to the two operands and one output (result) of the ALU.

- *Control* is the hardware that controls the flow and operations on the data by sequencing them and enabling the right processes by switching, operation selection, data movement between ALU components.

Register file-based datapath has only two components. ALU reads the operands from the read ports and writes the result into the register file. The control signals are generated for these operations. As can be seen, it starts by defining the methods to interact between the two components Register file and ALU. Another type of processor architecture typically uses the *load/store datapath*. The load/store datapath uses instructions such as load/store instructions which reads instruction from and writes the result to the memory with the absolute or referenced address using offset to predefined base address.

The load/store datapath is illustrated in Fig. 3.7, and performs the following actions in the order given:

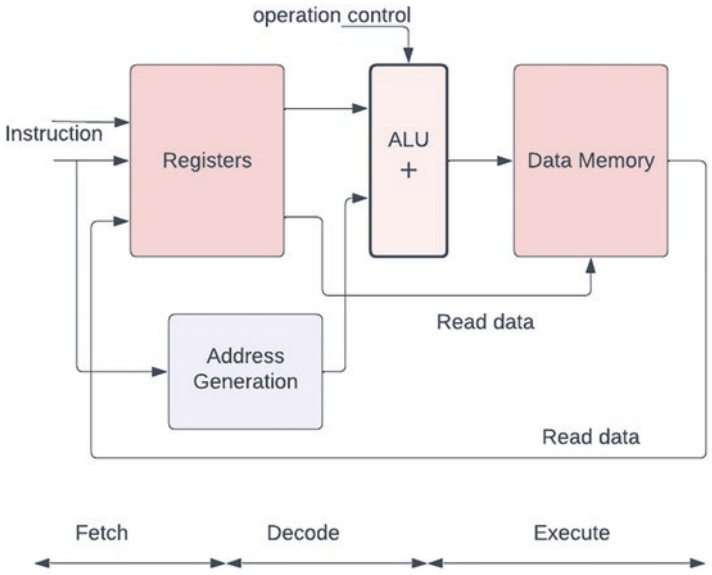


Fig. 3.7 Load/store datapath.

1. Register access takes input from the register file, to implement the instruction, data, or address fetch step of the fetch-decode-execute cycle.
2. Memory address generation block decodes the base address and offset, combining them to produce the actual memory address. This forms the decode step of the fetch-decode-execute cycle.
3. Read/Write from Memory takes data or instructions from the data memory, and executes the operation. This forms the initial phase of execute cycle.
4. Results are then written into Register File forming the second part of the execute step of the fetch/decode/execute cycle.

Datapath shown above is the single cycle datapath. If each of the opcode fetch, decode, and execute cycles are run in individual clock cycles, each of the functional units such as register file, ALU, and memory components can be used multiple times in one instruction cycle. This has lots of advantages over single cycle datapaths like reducing the hardware, using them for execution of multiple instructions by pipelining, etc. The execution time of different instructions is different in Multicycle datapath and hence the datapath is faster as against single cycle datapath where all instructions have same cycle. This makes datapath to run at slowest instruction cycle. Multiple cycle datapaths require additional memory and buffer components to separately store the data to be used in the next instruction and those used in the next cycle of current instruction. This requires additional buffer memories in the datapath and the corresponding Multiplexers to path right operands to ALU for operation. Additional multiplexer control signals are generated to control the dataflow in the datapath.

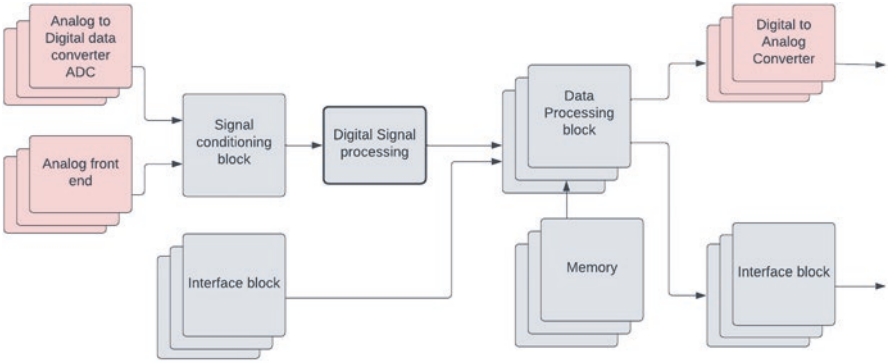


Fig. 3.8 Generic SOC datapath

The same can be extended to define complex datapaths to interact with the external world with input-output data set for an application, the data flow paths, and different processes and computational requirements on them to generate the desired outputs. Extending it further, the SOC input-output data is used to define the interface cores with the chosen protocol required for the system. The type of input-output data decides the data sensing and conversion functions required. A generic SOC datapath is shown in Fig. 3.8.

Chapter 4
Application-specific SOCs



4.1 Application-specific SOCs

Advancement to Internet Technologies (IT) and Semiconductor process technology has led to incredible innovations leading to the great revolution in computing applications. Since the invention of the first computer in the 1940s, we have seen a gamut of computers, each exceedingly more powerful than the predecessor and this trend is still ongoing. This will revolutionize the way we live by proliferating every domain. Computing has become an integral part of every system solution. Computing uses processors of different types, which can compute variety of data, information and signals in systems targeted for an application. To name a few, the following applications using compute resources are most evident:

- Automotive systems: Computing systems in automotive applications play a key role in automobiles in assisting drivers, in-vehicle infotainment, increasing the safety of occupants, and improving fuel efficiency via engine controls
- Smartphones: High-performance computing in today's smartphones enable networking across people anywhere and anytime in the world along with many other personal and time critical applications for financial and shopping experiences in almost real-time.
- Computing on the internet: Word Wide Web has become an integral part of our lives today, and this is enabled by high-performance computing in real-time.This offers theortically unlimited compute resources on demand for any application.
- Human genome applications: Human DNA sequencing, which was unheard of is becoming a reality, thanks to high-performance computing systems.

The above applications and many more one can think of are possible due to the software systems which are developed and run on the most powerful hardware of the most reliable architectures. This has made computing systems omnipresent in many forms such as personal computers, server systems, supercomputers, and

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

V. S. Chakravarthi, S. R. Koteshwar, *System on Chip (SOC) Architecture*,
https://doi.org/10.1007/978-3-031-36242-2_4

embedded computers, each of which uses systems of chips of various capabilities and complexities. Application requirements of each of them vary from each other drastically, giving rise to different architectures. For example, embedded computers have unique application requirements of lower performance with stringent limitations on cost or power. In consumer-oriented embedded applications, such as home appliances, simplicity is the primary requirement—the emphasis is on doing one function as perfectly as possible. In large embedded systems, techniques of redundancy from the server world are often employed. Embedded computers use systems using embedded processors or processor cores. The processor cores allow integration of many other Intelligent Property (IP) cores to realize different functionalities in a system. Embedded processors also enabled the development of many ***personal mobile devices*** (PMDs) using System on Chips developed using embedded processor cores. PMDs are battery-operated with wireless connectivity capable of accessing the Internet, and, like Personal Computer (PC)s, users can download software applications (apps) to execute on them. Unlike PCs, some of the PMDs have no keyboard and mouse, and are more likely to rely on a touch-sensitive screen or even speech input. PMDs in future, may include capabilities to be operated by Human gestures or electronic glasses, wearable devices or patches or invasive, miniaturized devices. Although this book deals with system architectures using **embedded computers**, the concepts are equally applicable to large computing systems targeted to any applications.

4.2 Embedded Computers

Embedded computers are a large class of computing engines used with a wide range of applications and performances. They have microprocessors, and related subsystems such as embedded memories and peripherals to support computing. They are designed to execute one application, or a set of instructions as programs related to one application that is integrated with hardware blocks to make a complete system. Embedded applications often have unique application requirements that combine a minimum performance with stringent limitations on cost or power. For example, in an audio music player, the processor needs only to be as fast as necessary to play music in real-time, and it must be cost-effective and reliable. In consumer-oriented embedded applications, it must be reliable and simple. Hence, embedded computers have a dedicated function, which is to be more reliable.

4.2.1 Embedded Processor Subsystems

Processor is the heart of embedded computers. The quality of the processor is determined by its processing power as its performance. The performance of the processor depends on the defined ***number of instructions*** and number of clock cycles

elapsed per instruction ***clock cycles per instruction (CPI)*** and the ***clock cycle time***. Processor architecture and implementation determines the clock cycle time and clock cycles per instruction. Concepts of Pipelining and Parallelism in processor architecture are used to get the best performance.

4.2.1.1 Pipelining

Pipelining is an implementation technique in which multiple instructions are overlapped during execution. The concept of pipelining is used in all data processing system architectures. If the execution of instruction is split into many stages like instruction fetch, register write/read, and execute, the concept of pipelining ensures all stages run concurrently but on different sets of instructions. This speed up the throughput. In pipelining, the execution time of single instruction does not get reduced but when there are many instructions, it improves the number of instructions executed per cycle, in contrast to the average time between instructions of a single-cycle implementation, in which all instructions take one clock cycle. Assume that the execution times for the major functional operations in an example are 100 ps and 50 ps for memory/register file read or write operations. In the single-cycle model, every instruction takes exactly one clock cycle, so the clock cycle must be stretched to accommodate the slowest instruction.

To understand the concept of pipelining clearer, consider the processor executing instructions shown in Table 4.1.

Table 4.2 shows the time required for each of the stages of execution of some of these instructions.

Observe in Table 4.2 that the load instruction takes the longest time to execute which is 400ps. All other instruction should be given 400ps despite them finishing early. With pipelining, the first state of second load instruction will start after 100ps, when it gets over for first load instruction.This is shown in the Fig. 4.1. Figure 4.1 compares the program execution time of three successive load instructions with and without pipelining.

Thus, the time between the first and fourth instructions in the nonpipelined design is 3×400 ps or 1200 ps. And, with pipelining, the same is reduced to 700 ps. In pipelining, the time between instructions can be computed as

Table 4.1 Instructions to be executed by processor

Instructions to be run	Description
Load	Memory writes
Store	Memory read
Add	Addition
Subtract	Subtraction
AND	Logic operation
OR	Logic operation OR
Branch	Jump to location

Table 4.2 Execution times of different stages of different instructions

Instructions to be run	Instruction fetch	Register/ Memory read	Execute	Data access	Register/ Memory write	Total time taken
Load	100ps	50ps	100ps	100ps	50ps	400ps
Store	100ps	50ps	100ps	100ps		350ps
Add/Subtract/ AND/OR	100ps	50ps	100ps		50ps	300ps
Branch	100ps	50ps	100ps			250ps

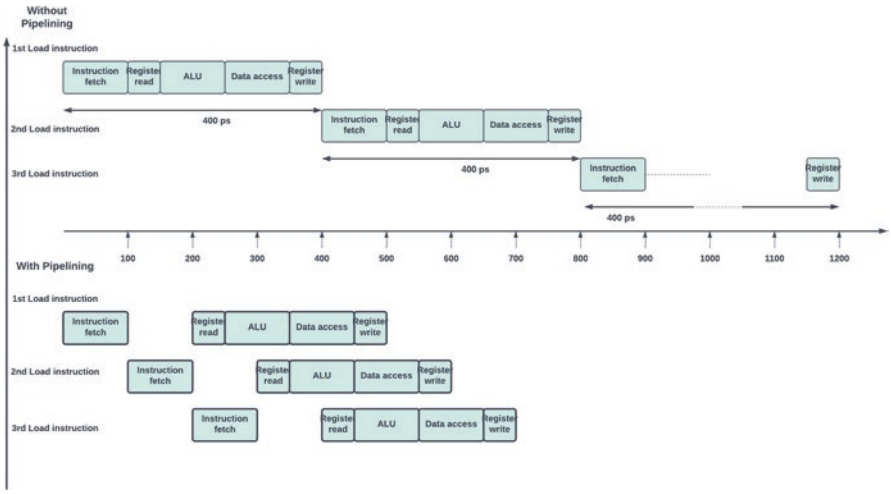


Fig. 4.1 Comparison of the program execution time of three successive load instructions

$$\text{Time between Instructions}_{\text{Pipelined}} = \frac{\text{Time between Instructions}_{\text{without pipeline}}}{\text{Number of stages}}$$

Arriving at the equation it is assumed that the execution times of all stages are equal, which is not practical. However, this gives the overall performance impact of pipelining concept. Pipelining improves performance by increasing ***instruction throughput***, in contrast to decreasing the execution time of individual instruction. Still, instruction ***throughput*** is an important metric because real programs have billions of instructions.

Pipelining is easier when all stages have an equal time of execution, but some of the stages in instructions take longer time to execute which makes pipelining more challenging. Many times, it will not be possible to execute the subsequent stages in the next clock cycle of the completion of the preceding stage. This is because stage to be processed will still be busy processing previous instruction stage. This condition is called ***pipeline hazards***. There are three types of hazards: ***structural hazards***, ***data hazards,*** and ***instruction hazards***. Structural hazards are limitations of hardware architectures. Pipeline data hazards occur when the pipeline is stalled

because the current stage is made to wait for the completion of another stage. This occurs when the next stage requires the result of the previous stage operation. The third type of hazard called a instruction hazard or control hazard or branch hazard, arises from the need for the results of one instruction while others are executing. This occurs when the next instruction is a branch instruction which requires jumping to a new address and then executing the instruction. These hazards can be addressed by additional hardware for high-performance processors using techniques such as **branch prediction**.

4.2.1.2 Parallelism

Another most used concept in processor architectures is ***parallelism***. In fact, pipelining is executing different stages of instruction execution in parallel, but the term parallelism is used to represent the ***instruction level parallelism or ILP.*** It is a scheme whereby multiple instructions are launched in one clock cycle. This may require replicating pipeline stages in hardware to support instruction-level parallelism.

Both pipelining and Parallelism increase peak instruction throughput and attempt to exploit instruction-level parallelism (ILP).

4.2.2 Hardware-Software Interface

Software program interfaces to hardware through instructions or commands, which are generally referred to as an instruction set. Hardware architecture will be the implementation of the classification of the instruction set, parsing them and acting upon the operand provided in the instructions. Such hardware architectures for processors are called ***instruction set architecture (ISA).*** The execution sequence of hardware is easily controlled by a set of instructions. This interface scheme is followed since the invention of processors and is time-tested. Hence there are similarities in the instruction set of different processors.

Every processor has an arithmetic and logic unit (ALU) and hence supports the execution of a set of arithmetic and logic operations. The instruction set consists of other data operation instructions apart from arithmetic and logic instructions. Some of these instructions are copy instructions or move instructions which change the course of execution of the processors. An indicative set of processor instructions is given in Table 4.3.

Programs written using a set of supported instructions for hardware directly are called ***assembly language***. Programmers write code in high-level programming languages such as C, Python, and Java. it is the ***compiler*** which translates the C programs to instructions of the processors. The compiler translates from C to processor-supported assembly language instructions.

Consider an example with the expression

Table 4.3 Indicative set of processor instructions

Instructions	Operation code	Description of operations
Add a, b	Add	Adds operands a, b
Sub a,b	Sub	Subtracts a from b
ADI 32'ha5a5a5a5	ADI	Immediately adds 32-bit data given in the instruction
AND A, B	AND	Logical AND operation of operands A and B
OR A, B	OR	Logical OR operation of operands A and B

$$f = (a + b) - (c + d)$$

The compiler breaks the statement into several assembly instructions. Only one operation is executed by the processing engine per processor instruction. In this example, the processing engine first computes the sum of a and b. The compiler then creates a temporary variable t1 and places the result. It then finds an instruction to add c and d, and creates another temporary variable t2 to place the result. It finally, finds the subtract instruction to subtract the content of t2 from t1, and the result f is stored in the register.

Unlike variables in high-level programs, the operands in assembly language are restricted in usage and can correspond to the available size of registers in hardware. The typical sizes of hardware registers used in processors are word (32-bit) and double word (64-bit). Also, there are a limited number of such registers in the processors available for arithmatic and logic operations. The results stored in one of the preassigned registers are finally moved to the output registers or memory.

In earlier days, hardware architectures supported a single register where the result of arithmetic and logic operations was saved. This register was called an ***accumulator*** since all the results of operations would accumulate in one register. Today technological advancement enabled many on-chip registers in the processors that are used for arithmetic and logic operations. Such processor architectures where all operations are supported on any register of a set of registers is called ***general purpose register architecture***.

This also allows, through some instructions, one or both operands to be stored in memory. Such architectures are called ***load-store or register-to-register architectures***. In this architecture, the instruction set uses registers to hold a few of the operands in registers and the rest of the operands to be accessed in memory through load/store instructions. Programming languages not only have simple variables that contain single data elements, as in the examples discussed previously, but also have more complex data structures—arrays and structures. These composite data structures can contain many more data elements than there are registers in a computer. These use memories in the processors. The instructions used to move the data from registers to memory are called ***data transfer instructions.*** These

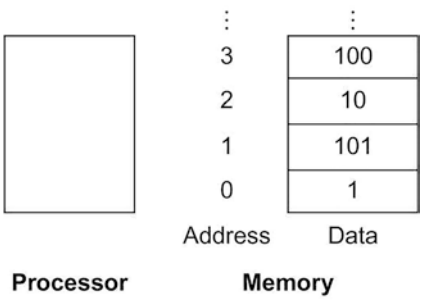


Fig. 4.2 Data movement from memory to register

instructions move data from memory to registers or vice versa depending on the address provided in the instructions, as shown in Fig. 4.2. The data transfer instruction that copies data from memory to a register is traditionally called ***load***. The format of the load instruction is the name of the operation (load) followed by the register to be loaded, then the register and a constant used to access memory in some processors. It involved computing the address of the memory containing data to be moved to the register. It could be a direct memory address or a derived indirect address of the memory, depending on the instruction. Writing data from registers to memory is done through ***store*** instructions. The most common examples of processors with load-store architecture are MIPS, ARMv7, ARMv8, and RISC V processors.

Embedded processors are required to have optimal program code as they support limited instruction memory. This resulted in processor architectures with variable instruction lengths to accommodate varying operand widths. Intel's x86 processors, ARM thumb 1, thumb 2, MIPS 16, and RISC V compressed are some examples of variable-length instructions architectures. Processor architectures further aim to support optimized code size by their ***stack-based architectures*** where no registers are used to store data, but the data is held in a memory array called stack from which the data is pulled or pushed as per the instructions. Code size and eventually memory size is an important parameter for choosing the right processors in embedded applications. Processor architectures supporting small code sizes and hence small memory sizes have the advantage of transferring codes of small footprint efficiently on the internet remotely.

4.2.2.1 Stored Program Concept

Storing the instructions and different types of data in memory is another useful method in processors so that they can be changed easily is the stored-program concept.

4.2.3 *Exceptions and Interrupts*

Exceptions are unplanned or unscheduled events or interrupts occurred due to data overflow or underflow while executing programs. These are indications to the user about the unscheduled occurrence of data underflow or overflows in registers. When such an event occurs, by design, control of execution transfers to a known set of instructions which does some data cleanup and logs the status before it restarts or resets to a known state. This is be as per the implementation of the processor control state machine. **Interrupts** are external events which occur anytime asynchronously and require immediate processor response within critical timing to avoid any errors during its expected functions. So, interrupts and exceptions are asynchronous to the processor execution flow. Interrupts are expected and execution control is transferred by design to serve them. Exceptions are unplanned but handled like interrupts as damage control procedures. Detecting exceptional conditions are classified and grouped to be detected for taking the appropriate action within the critical timing of a processor clock cycle time. The implementation of this affects the speed performance. It is important to understand, identify, and handle the critical exceptions while defining the architecture of the control unit. If not, attempts to add exceptions to an intricate implementation can significantly reduce performance, as well as complicate the task of getting the processor design correct. The basic action when interrupts or exceptions occur is to save the address of the instruction of execution code when such an event has occurred and transfer the control to the operating system for the appropriate set of actions. This also requires knowing the cause of the critical event, which is generally collected as extra information stored in a register. The operating system can then take the appropriate action, which may involve providing some service to the user program, taking some predefined action in response to a malfunction, or stopping the execution of the program and reporting an error. The execution control will return to the saved address to proceed after the critical events are served. The interrupts and exceptions are graded in terms of criticality. Exceptions are of the highest order of criticality and sometimes require a restart of the processor. The transfer of execution control to the address depends on the cause of critical events, whether it is an interrupt or an exception. This is called ***vectored interrupts.*** In a vectored interrupt, the address to which control is transferred is determined by the cause of the exception, possibly by considering it as offset and adding it to a base register that points to the memory range for vectored interrupts. Examples of some of the exceptions are hardware malfunction or unsupported instructions in the processor. To handle these exceptions, the typical vector addresses defined are shown in Table 4.4.

Table 4.4 Vector addresses used for handling exceptions

Exception	Base offset	Reason
Unsupported instruction	0x00ff	Occurrence of unknown instruction.
Hardware failure	0x0fff	Expected hardware response not received. Such as less than required power supply or shutdown

The causes of the exceptions and interrupts are encoded or vectored as per the control architecture in the system, and the operating system will accordingly decode or serve them. These events can be pipelined such that an interrupt or exception is executed after the completion of current instruction in progress. This requires good coordination of hardware and operating system (OS) software. When an interrupt or exception occurs, hardware typically completes executing all the previous instructions and flush the future instructions, captures the event, saves the code identity of the cause of occurrence of the exception/interrupt in the register, saves the address of the offending instruction, and then jump to a predefined address. The operating system (OS) will respond depending on the cause of the interrupt or exception event. For the undefined instruction or hardware failure, the software typically suspends and kills the program and indicates the cause of the event. For the request of Input-Output (I/O) device or an OS service request, it saves the state of the program, performs the predetermined task, and, after the completion, restores the program to continue execution from the suspended instruction.

4.3 System Modelling

The performance of the embedded processor depends on the number of implemented instructions, parallel and pipeline logic and hardware-software interaction through interrupt mechanism and handling of exceptions. To implement all of these, the processor requires many peripherals such as on-chip memories, interrupt controllers, and Direct Memory Access (DMA) controller blocks. On chip memory stores the data and program. All of these together forms the **processor subsystem**. This determines the subsystem performance in terms of speed, and power. To determine the overall performance at the system level, the systems are modeled on a high-performance system with the help of programming platforms. The system modeling platforms are designed specifically to analyze and design systems. These are also called **system simulators**, and some of the well-known platforms are Matlab, Scilab, Systemvue, and Sagemath. System models are sometimes written in system modeling languages like SystemC. This step-in system design is referred to as ***architectural exploration***. Architecture exploration selects a set of processing elements and maps the computation behavior of the specification onto the ***processing elements (PEs).*** Architecture exploration refines the specification model into the intermediate architecture model. The architecture model describes the PE structure of the system architecture and the mapping of computation behaviors onto the PEs, including estimated execution times for the behavior of each PE.

Architecture exploration is followed by **communication synthesis** to complete the system synthesis process. Communication synthesis selects a system bus, protocols and maps the communication functionality onto the system bus. Communication synthesis creates the communication model which reflects the bus architecture of the system and the maps the communication function onto the bus. The communication model is the result of the **system synthesis** process. It describes

the structure of the system architecture consisting of PEs and busses, and the implementation of the system functionality on this architecture. It is timed in computation and communication, i.e., simulation detail is increased by events for estimated execution and communication delays. The communication model is a structural view at the system level. Evaluating the computational performance of the system determines the following decisions in architecture:

- Need for single or multiple cores
- Hardware-software partitioning
- On-chip organization of the peripherals in sub-system
- Workload balancing in multicore systems
- Embedded memory organization
- Accelerator-based designs

4.4 Capturing System Requirements

System is modeled on virtual platforms and the functional features are analyzed for its timing and power performance. A virtual platform is used to determine the feasibility of designs for achieving the required timing performance with a choice of hierarchical or flat models of different organizations to determine the right architecture. Timing models can be stated as machine based which are developed in SystemC/C++ description to define functional and timed virtual platform (black box) components.

The modeling framework supports the integration of a broad range of system-level timing and power models into virtual platforms. If the models are unavailable, they are coded using high-level languages for their behaviors. These enable simulating system timing and power behavior of hardware and software for a given target architecture by annotating low-level timing and power numbers in the functional behavior models. This allows a dynamic estimation of timing and power properties without interpreting the target code on the simulation host. This specific model is written by the user and forms the input to the design process.

To consider custom hardware, the resulting controller and data path timing and power properties are modeled and synthesized using high-level synthesis (HLS) methods and are simulated in addition to the functionality. High-level synthesis is a process of converting functionalities modeled using high-level programming languages into hardware description language (HDL) models.

At the same time, the specification of the functionality of each PE of the system in the form of a behavioral view at the register-transfer level forms the input to the RTL synthesis of those components in the backend. In a hierarchical fashion, each PE is synthesized separately in the backend, and the behavioral view of the PE is replaced with a structural view of its RTL or instruction-set (IS) microarchitecture. The result of this backend process is the implementation model. The implementation model is a cycle-accurate, structural description of the RTL/IS architecture of the

whole system. In a hierarchical fashion, the implementation model describes the system structure and the RTL structure of each PE in the system. Simulation detail is defined using the system clock, i.e., the timing resolution is in terms of clock events for each local PE clock.

By combining these approaches with a common timing and power model, the interaction between software, custom hardware, and third-party black-box IP components are analyzed for processor-based complete system architectures. The same method is used for complex multi-processor System on Chip (MPSOC) architectures running real application code using host-based simulations. The system modeling flow is shown in Fig. 4.3.

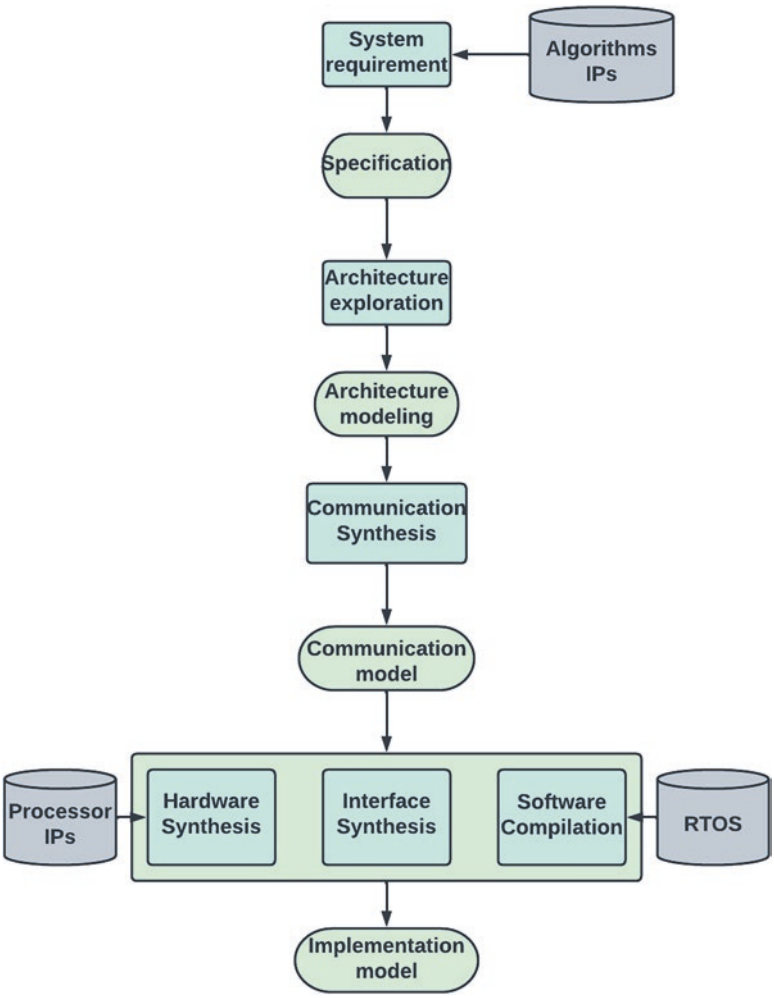


Fig. 4.3 System modeling flow

4.4.1 Explicit and Implicit Requirements

If computer-based system models are specified using an executable modeling language, it is possible to gradually evolve them starting with very abstract description application models, into prescriptive ones that are precise and detailed enough to be implementable on chip. The system requirements are classified as **explicit** and **implicit** requirements. The functional requirements are explicit requirements that state compliance with applicable standards, define the interfaces to its use case environment and other user-friendly features. In addition to these explicit requirements, there are many implicit requirements, such as chip must consume the lowest power, lower latency, faster response times to external critical events or reconfigurability, etc. The implicit requirements are additional factors that provide an advantage to the user in some way or the other in terms of performance.

4.5 Deriving SOC Specifications

The requirements are converted into system specifications. Typical specifications include functional requirements, interface definitions, and performance requirements in terms of speed, power, and area dictated by the package choices. System specifications are derived by simulating the system model. System modeling with a verification model is an iterative process. The system parameters are validated by estimation and compared with the verification models to derive the specifications. The integrated system modeling and verification flow are shown in Fig. 4.4. Following architectural decisions are taken by this process to arrive at the feasibility of the system implementation model:

4.5.1 Clock Frequency

Depending on the overall bandwidth requirement and defined bus widths in the bus-based architecture, the clock frequency for the system design is determined. For example, for the system which processes one Gigabit per second of data from four 8-bit interface ports simultaneously, the clock frequency is calculated as follows:

Data rate to be processed simultaneously: 1Gbps
Number of interface ports: 4
Number of data flow directions: upstream and downstream= 2
Total data processing needed per direction: 4 * 1 * 2 = 8 Gbps

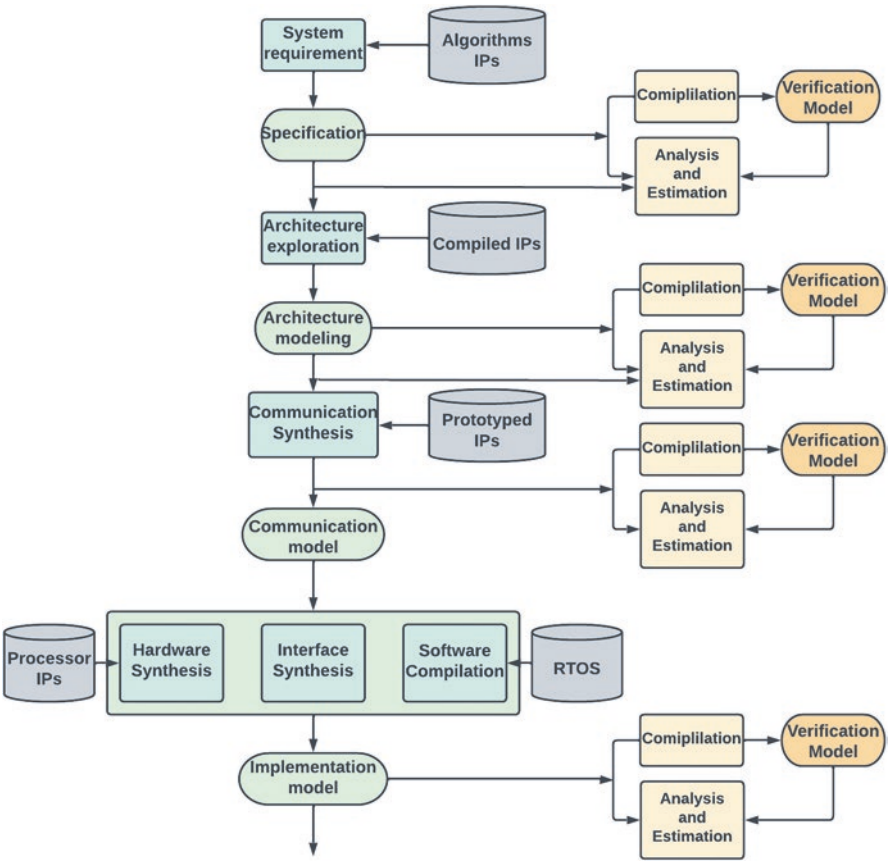


Fig. 4.4 Integrated system modeling flow with verification

Assuming internal system bus width of 32 bits, the clock frequency of the-system $f = \frac{8 * 10^9}{32} = 250\,\text{MHz}$.

If the internal bus width is assumed to be 64-bit wide, then the system clockfrequency will be $\frac{250}{2} = 125\,\text{MHz}$.

Lower the clock frequency lesser the timining issues faced to make the design work at required speed, power. However, it is challenging to fix issues of congestion during Physical synthesis which are to be considered. Also, another issue to be considered is the availability of clock circuit sources like on-chip Phase Locked Loop (PLL) and external high-quality crystals for reference clocks.

4.5.2 Choice of Processor Cores

Selection of processor cores is the techno-managerial decision depending on the processing power required to process the upstream and downstream data on the chip. Some of the important design decisions in this stage are determined by thinking through following factors:

1. Do you want to use a centralized processor to process all the data on chip, chip configurations, and Interface protocol management or have independent processors for interface cores and centralized processors for data processing?
2. Use of accelerators for time-critical processing and leave part of the data processing to the software for flexibility. This decision is usually taken if there are specification changes expected in the protocols or user requirements.
3. HW-SW interface: Do you use a standard bus interface for the software interface, such as high-performance standard interface cores such as AXI, AHB/APB, PCI express, etc. or a proprietary interface bus? Standard interfaces provide lots of freedom to reuse third-party readily available cores to connect to processors and future expansions. High-performance bus architectures even use the standard network on chips to connect many interface cores, processor cores, expandable memories, and low data rate interfaces such as APB, I2C, I3C, etc. to connect to low data rate interface cores on the system.
4. DSP core: Need for digital signal processing arrives in many high-performance low latency IOT and multimedia applications where many standard signal conditioning and processing functionality are part of the functionality.
5. On-chip and off-chip memory: Decision on embedded memory and off-chip memory is a function of acceptable data processing latencies specified. However, for a commonly shared processor used for data processing, the data from other interfaces are to be stored till the completion of the data processing from the currently active interface. On-chip or off-chip memory decision depends on the access times and the internal bus width decided. It is to be noted that embedded memories are preferred for low latency, and low power, as input-output transactions have larger access times and are power-hungry.
6. Technology decisions: Finally, process technology node decision is a techno-commercial call taken considering the following:

   (a) Estimated die size
   (b) Preferred packages
   (c) Cost per square silicon
   (d) Need for additional heat sinks which will add to development cost

4.5.3 System Software

The choice of processor core decides the software architecture for the System on Chip. It is appropriate to define the following functional procedures at this stage:

- Power on sequence: It is necessary to decide on the power on sequence for the system as this is crucial to bring up the system by power on. The initialization procedures, running built-in self-tests (BIST) on critical memories, core interface configurations, and exception handling procedures are critical to the success of the system functioning.
- Soft and hard reset: Reset mechanisms for processor-based system architectures are equally important for the proper functioning of the systems.
- Booting: Power on sequence leads to system booting, which will do initial housekeeping functionalities of the systems for the operating system to take on. The entire software stack functions in the coordination of the operating system, which interacts with the hardware beneath.
- Compiler and interpreter: The availability of a stable compiler, and interpreter are essential which will ease the development of the software stack for any application. This also ensures the reuse of the software across hardware generations, making it independent of the hardware architectures.
- Debugger: A good debugger helps in hardware-software development, ensuring faster time to market. Debugger also helps in identifying the system issues by isolating them to be in hardware or software. This enables exposing all the contents of the internal memories and registers during software development and debugging.

4.6 Processor Subsystem IP selection

In conclusion, making or buying decision of the processor subsystem for a system architecture is predominantly a commercial decision. It depends on project timelines, development capabilities, the resources available in house to develop and the cost. However, the selection considerations for acquiring a processor core as intelligent property (IP) core for System on Chip integration and development are the following:

- Power, speed performance, area, target process technology and the cost
- Flexibility of the core in terms of internal bus width, clock frequency, register set, supported set of instructions and high-performance and low-performance busses and adding peripheral IP cores
- Availability as soft or hard core
- Availability of system software
- Software Development environment
- Supported model for simulations
- Support libraries for system development
- Support real time operating system (RTOS)
- Interoperability with other IP core blocks

Chapter 5
Storage in SOCs



5.1 Storage in SOCs

Storage embedded on chip has become an integral part of any SOC today. There are systems that contain memories to the extent of 60–70% of the SoC silicon area. The multi-processor subsystems invariably use **chip multi-processor (CMP)** architectures with defined memory organization for optimized SOC performance. CMP architectures have become the de-facto architecture in multi-processor SOCs in which there are more than one processor core. The number of processor cores on chip is increasing day by day. In such architectures, it becomes equally necessary to organize the on-chip memories at multiple levels of hierarchy. Level of memory in SOC architectures refers to proximity of memories to processor and the speed of accessing the data stored in them. The size, type of memories, and level of a particular memory in the architecture determine the performance of the SOC. Because of the large difference in the speed of data access times between ***on-chip*** and ***off-chip*** memories, modern CMPs employ large and multilevel memory hierarchy on chip. The number of processor cores in a typical SOC will be tens to hundreds of cores depending on the application. Each of the processor cores will typically have its own memory clusters arranged in memory hierarchy. Depending on the function a processor subsystem performs in a SOC, it forms different subystems. The simplest memory hierarchies in SOC processor subsystem are the ***buffer memory level***, ***cache memory level,*** and ***register memory levels***. This arrangement is aimed to achieve minimized data access and data movement. However, the organization of embedded memories in SOC often depends on the data flow and requirement of data for processing in the systems.

Data is stored in SOCs to ensure its easy access by the processor when needed. Different types of embedded memories used as storage in the SOC are listed in Table 5.1.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

V. S. Chakravarthi, S. R. Koteshwar, *System on Chip (SOC) Architecture*,
https://doi.org/10.1007/978-3-031-36242-2_5

Table 5.1 Different storages in system

Storage	Description
Register set	Individual registers in the register set are used to store the operands. ALU performs arithmetic and logic operations on these operands. The width of the registers is chosen dependent on the operations supported in ALU. For example, if the processor supports a 64-bit operation, the registers will be 64-bit wide. Register set is also used as scratchpad memory to store partially processed data. It is also used to store the configuration and status of the system blocks.
Cache memory	Embedded memories are organized in different levels of hierarchy in accessing the processor. Cache memories are of different sizes and performances organized at different levels in terms of proximity to the processor. Level one Cache memory is at the first level of hierarchy and is interfaced directly to the Processor. The processor looks for instruction and data, first in the Cache memory.
Buffer memory	Memory organized to hold data of predefined chunks of data. They are organized as buffers with the ability to operate in batches of buffer sizes defined during the system architecture. They can be arranged as first in first out, linked lists or queues for data processing in the systems
External memory	Large memory to hold data and programs

5.1.1 On-chip Cache Memories

In processor architectures, memories are used as cache memory. Cache memory is the temporary storage which stores data and programs which are frequently accessed by the processor elements. It is high-speed memory that is used to reduce the access time of most frequently used data for processing, thus enhancing the data processing latency. Cache memory is an extremely fast memory type that acts as a buffer between RAM and the processor unit. Therefore, it stores data or programs in duplicate to make them available to the processor when needed. There can be many independent cache memories in a processor system. Typical cache memory in a processor system is shown in Fig. 5.1.

5.1.1.1 Cache Memory Organization

Cache memory performances are important for many applications. Or in other words, most applications are limited by the memory bandwidth. Better memory performance and cache usage by applications results in successful SOC solutions.

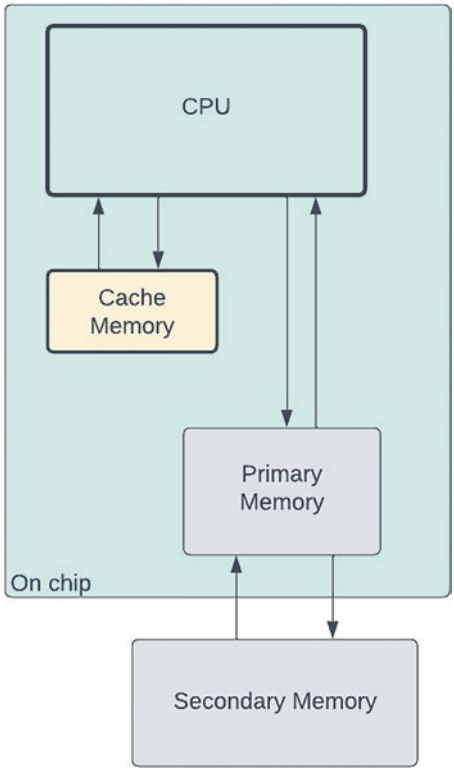


Fig. 5.1 Cache memory hierarchy in CMP system architectures

5.1.1.2 Cache Hierarchy

Cache hierarchy in processor subsystems depends on the data, which is reused quite frequently. When the processor uses data repeatedly, it can effectively use cache memories. It is seen that data that has been processed recently has a greater likelihood of being used again by the processor for further processing. This property is called ***temporal locality*** of data which is used for prefetching the data in the cache memory. Data is fetched into the cache memory from the main memory in blocks called ***cache lines,*** as there is a large likely hood of fetching the data from nearby addresses in the main memory. This access mechanism is based on the data property called ***spatial locality***. Usually, algorithms will use data from nearby locations as it is believed that related data is stored in nearby locations. Cache memories in systems are organized on many levels exploiting these data properties depending on the applications.

5.1.1.3 Levels of Cache Memory in Subsystems of SOC

Processors use a set of registers for processing data in the CPU. These are typically a set of 32/64 registers which are used directly by the CPU. Graphics processor (GPU) architectures use many sets of a large number of registers compared to CPUs. The first level of cache memory referred to as ***L1 cache*** is small, high-speed high bandwidth, low latency memories which are accessed first by the processor for data. L1 cache data can typically be accessed in two or three clock cycles. If the required data is not found in the ***L1 cache*** which is termed as a ***cache miss,*** *the* processor accesses second-level cache memory which is referred to as ***L2 cache***. L2 cache also has good bandwidth and good latency. If the required data is not found here, data is fetched from third-level cache memory called ***L3 cache,*** which is large memories with high latency and low bandwidth. Many processor cores often share this memory on chip in multi-processor system architectures. Cache memory structure often used in typical processor sub-systems in multi-processor CMP architectures is shown in Fig. 5.2. As discussed, you will often find processor

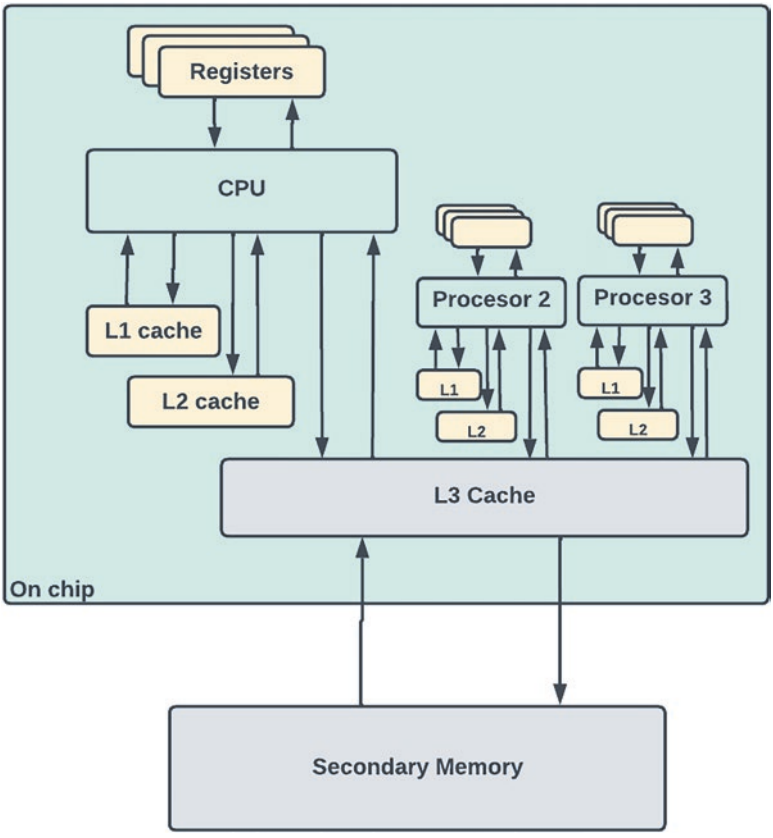


Fig. 5.2 Cache memory levels in CMP architecture

architectures with three levels of cache memories. Depending on the requirement of applications, more levels are added as specific customization. Accessing data from the high-level memories takes many hundreds of clock cycles which may adversely affect the system performance. A good cache organization in systems is to get 100% ***cache hit***s which is an ideal situation. However, any designs with above 95% cache performance are termed good designs.

5.1.1.4 Cache Line and Cache Bandwidth

Cache memory is often high-speed memory of small size and is directly interfaced to the processor. When the processor accesses data from memory first, a chunk of data around the data accessed is read and stored in cache memory with the hope that this data will be accessed by the processor soon. This chunk of data copied to cache is called ***cache line.*** Therefore, the cache line directly maps to a part of main memory. Typical sizes of cache lines are 32 bytes, 64 bytes, and 128 bytes. ***Cache size*** is a number of cache lines the cache memory can hold. A 128 Kbyte cache with 64-byte cache line can have only 2048 cache lines. In multi-processor systems, the cache size assigned to different processors is different depending on the application.

5.1.1.5 Cache Properties

When all the cache lines are being used by the processor and when the new data is accessed by the processor, it accesses from main memory, and to fill the cache data, the cache memory must be freed. The scheme used to determine the cache lines to be replaced is called ***replacement policy***. The replacement policy of cache memory can be least recently used or random depending on the architecture of the system. The scheme of replacing the latest used cache line by the processor is called the least recently used (LRU) policy. Managing the policy to ensure the cache coherence is called Cache Management.

When a processor accesses data that is not in the cache, it is termed as ***cache miss***. When a cache miss occurs, the processor will access it from subsequent levels of cache and main memory until it finds it. Hence a large number of cache misses directly affect the processor performance. A good system architecture aims to have a smaller miss rate for cache designs. The miss rate is the ratio of the number of misses to the cache size. Larger the cache size, the smaller the miss rate but higher the cost. It is a tradeoff between system performance and cost. There are techniques like ***prefetch*** that are used to have a smaller cache miss ratio. The target applications mainly drive required cache performance.

Multiple threads running in the processor or multiple processor cores may have different copies of the data in their respective local cache memories and when they write back, the data can be wrongly fetched by the other processors. This can cause unintended behavior. This is avoided by maintaining the synchronization among different cache memories by a protocol called ***cache coherence***. There are many variants of cache coherence protocols. Most processors use the MESI protocol or

some variation such as the MOESI protocol. The MESI protocol is named after the ***four states*** of a cache line: modified, exclusive, shared, and invalid.

The states of cache line in the MESI protocol are as follows:

- M – Modified

The data in the cache line is modified and is guaranteed to only reside in this cache. The copy in main memory is not up to date, so when the cache line leaves the modified state the data must be written back to main memory.

- E – Exclusive

The data in the cache line is unmodified but is guaranteed to only reside in this cache.

- S – Shared

The data in the cache line is unmodified, and there may also be copies of it in other caches.

- I – Invalid

The cache line does not contain valid data.

Implementation of the cache coherence protocol is to be supported by the system architecture. In addition to the protocol, architecture must support collecting some of the performance statistics of the cache. Important in them are ***upgrade count, coherence misses, and coherence write-backs,*** which help the programmers to arrive at the efficient coding guidelines for the architecture. The ***upgrade count*** is the number of memory accesses that cause a cache line to be upgraded from a shared state to either an exclusive or modified state. The ***coherence miss count*** is the number of memory accesses that miss because a cache line that would otherwise be present in the thread's cache has been invalidated by a write from another thread. The ***coherence write-back count*** is the number of memory access that forces a cache line that is in the modified state in another thread's cache to be written back.

Many times, it is difficult to predict the required cache performance at the architectural level of the system definition. Incorporating support such as prefetch to improve cache performance may sometimes backfire and increase cache misses in a multi-processor environment. This would limit memory bandwidth for applications resulting in reduced operation speed at the application level resulting in a bad user experience. It is always better to add capabilities to enable/disable switches to the part of the techniques adopted in hardware and controlled by firmware which will offer enough flexibility to the architecture to use the hardware resources as required by the applications.

5.1.2 Translation Lookaside Buffer (TLB)

Translation lookaside buffer (TLB) is another major support feature in chip multiprocessor (CMP) architectures. TLBs hold logical or virtual addresses mapped to actual addresses of the physical memory. The virtual address is the address seen by

the software. Memory access for data starts from the process of physical address map by search. This search process takes multiple clocks and is called a page walk. The mapping helps in managing the system memory efficiently. The TLB organization, with its size, is a major factor determining the overall system performance. Cache memories in such architectures hold TLBs instead of actual cache lines. Such a cache is called address translation cache. This is used in memory management units (MMU) s of CMP architectures. TLBs are sometimes implemented as content addressable memory (CAM); when a virtual address is searched, it returns the physical address of the data to be accessed. If a processor finds an address in TLB memory, it is called a TLB hit but sometimes, there is a possibility of TLB miss. Whenever the processors access data from memory, TLB lookup is carried out to determine the physical address and then the cache or primary memory is accessed. Therefore, the TLB resides between the processor and the cache memory or primary memory. Major parameters for TLB-based systems are: TLB size, Hit time, Miss penalty, and miss rate.

$$\text{The average effective memory cycle rate} = n + (1 - m)h + mn \text{ cycles},$$

where n is the number of cycles required for a memory read,

m is the miss rate, and
h is the hit time in cycles.

For example, if a TLB hit takes 1 clock cycle,

a TLB miss takes 30 clock cycles,
a memory read access takes 30 clock cycles,
and the miss rate is 1%,

the average effective rate of memory cycle = 30+(1−0.01) * 1+0.01* 30

= 31.29 clock cycles per access

5.1.3 On-chip Data or Buffers

In non-computing systems, the data however small or big is stored in memories either for rate adjustment or as temporary storage for partially processed data. Depending on the storage needs of the data and control paths in system architecture, different types of memories are chosen. On-chip memories are often chosen considering the data rate, memory size, data retention, power efficiency, and read/write characteristics.

5.2 Types of Memories

There are four different types of memories available for System on Chip integration. They are listed below:

- Dynamic random-access memory (DRAM)
- Static random-access memory (SRAM)
- Electrically erasable programmable read-only memory (EEPROM)
- Flash Memory

DRAM and SRAM memories are used as on-chip primary memory or RAM. EEPROM and Flash memories are used as secondary memory or ROM. Often in systems, used for many data and signal processing applications such as smartphones, and communication processors, both primary and secondary memories are required. The comparison of primary and secondary memories is given in Table 5.2

As shown in the table, the primary memories can be both types, SRAM and DRAM. DRAM cells have simple structures with a single transistor and capacitor compared to SRAM which uses multiple transistors (six transistors). DRAMs offer high packing density but need periodic refreshing. For on-chip system requirements, Table 5.3 shows the selection criteria of memories depending on the size of the memory needed in the system.

Table 5.2 Comparison of primary and secondary memories

	Primary		Secondary	
Memory type	SRAM	DRAM	EEPROM	Flash
Size	Small	Small	Large	Large
Interface with CPU	Direct	Direct	Indirect	Indirect
Speed	Fast Few cycles	Fast Few cycles	Slow hundreds of cycles	Slow hundreds of cycles
Data retention	No	No	Retains	Retains
Refresh cycles	No	Needed	No	No

Table 5.3 Selection criteria for on-chip memories

System requirement in bytes	Type of memory chosen	Remarks
< 1 K	Register array	Memories come with overheads such as BIST. It is necessary to consider them while choosing smaller memories. If they are intolerable, simple register arrays designed using flip-flops are right choice.
< 8 K	SRAM	SRAMs are faster memory without the overhead of refresh cycles. They are chosen for faster memory
> 8 K	DRAM	For larger on-chip memory, DRAM is the preferred choice. However, they need periodic refresh cycles. They are dense memories. DRAMs of 1 T cells are the best choice for large on-chip data storage in systems such as in multimedia applications. These on-chip memories can be of sizes as large as terra bytes.

5.2.1 *Redundancy in Memory*

Sometimes in systems, memories are used for storing critical data such as boot loader, watchdog routines, and for storing encryption keys in security systems. In such safety-critical applications, it is essential that the memory faults are intolerable. In such cases technique redundancy is used wherein memory of a larger size than required is used on chip and whenever the memory failure is detected, then alternatively extra redundant memory locations are used for compensating memory errors. Such a memory is called redundant memory. Memory faults are detected by running CRC or checksum algorithms on the read data.

5.3 Memories in Advanced System Architectures

Techniques such as near-memory processing is an advanced concept used in high-performance systems. The data from the memory is read from the memory, processed in the logic around memory, and used in the data path of the system architecture without they are read into the CPU. This technique is called near-memory processing. It is a known fact that accessing the memory has a higher cost in terms of performance and power directly affecting the system performance. In such applications, where the performance is of high priority, near-memory processing is used in the system architecture. An example of system architecture with processor in memory core is shown in Fig. 5.3.

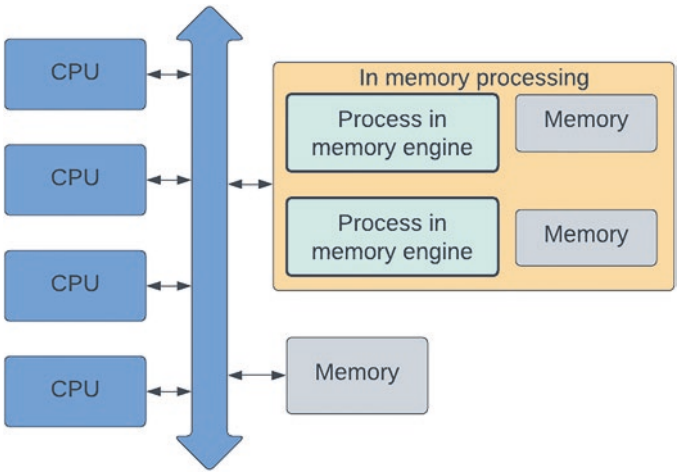


Fig. 5.3 System architecture within memory processing core

Chapter 6
SOC Architecture: A Case Study



6.1 Introduction to SOC Requirements (Environment Parameters)

Monitoring the environment is necessary for several reasons, the main reason being the sustainable growth of humanity. In general, environmental parameters give us insight into the potentially harmful pollutants and their levels. It is necessary to find the effect of the pollutants that exist in the environment on human health and on the climate. Monitoring the environmental parameters means determining the constituents of the environment and their levels to provide necessary information that the state administration can use to define legislative controls on emissions/discharge of pollutants and to ensure compliance with standards.

Recent advancements in Internet of Things (IoTs), Wireless Sensor Network (WSN) technologies coupled with Machine learning (ML) and artificial intelligence (AI) techniques play a major role in making environment monitoring a smart monitoring solution. Wireless sensor network nodes use IoT devices connected using wireless technologies like Bluetooth and WLAN to monitor the environment parameters such as temperature, humidity, and pollution level and control them. Many types of solutions are possible for monitoring parameters of our environment using multi-model spectral data captured by remote sensing satellite imaging, and IoT-based environment monitoring systems. Figure 6.1 shows a conceptual model of a cloud-based smart environment monitoring solution using these technologies.

Periodic monitoring of the temperature and humidity of the environment is one of the important functions of a smart environment monitoring solution. The captured data must be processed and stored enough to derive useful information which helps to make an informed decision regarding controlling them or alerting them about the impending danger which would be caused if the parameters went out of bounds. As a reference solution, the architecture of IoT-based smart environment monitoring system is discussed with step-by-step design methodology. The proposed solution is

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

V. S. Chakravarthi, S. R. Koteshwar, *System on Chip (SOC) Architecture*,
https://doi.org/10.1007/978-3-031-36242-2_6

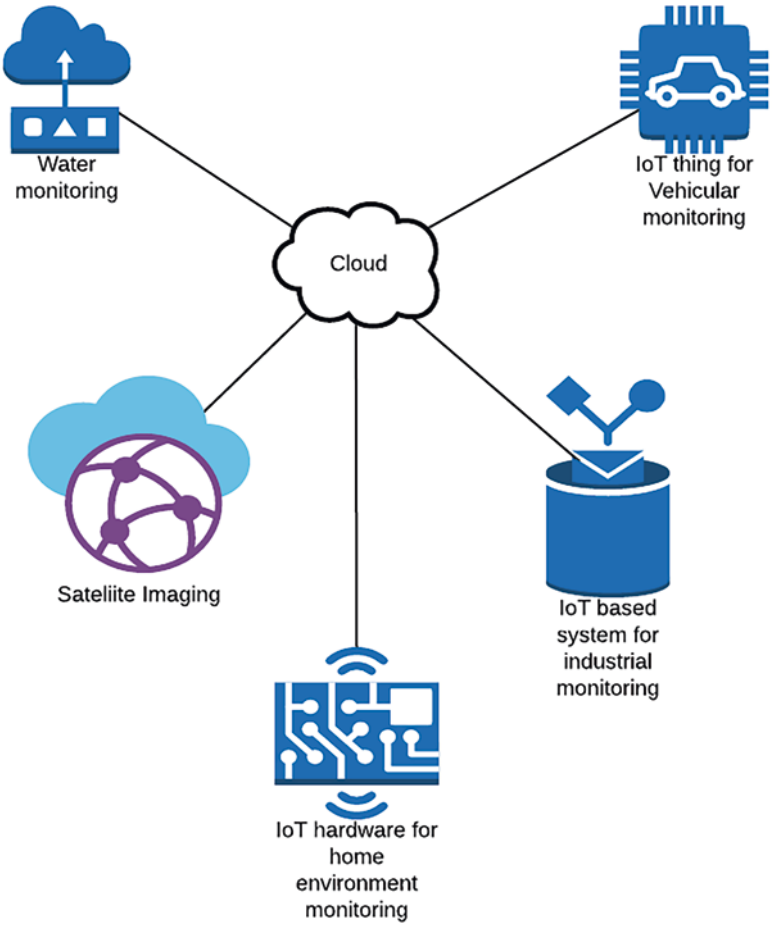


Fig. 6.1 Conceptual model of smart environment monitoring solution

IoT-based system that captures and updates the database on a cloud server with the measured parameters on a need basis and periodically in real time so that users can access the data from anywhere, anytime.

6.2 Smart IoT SOC for Environment Monitoring—An Architectural Case Study

Solution architecture provides the base for the development of solution to a problem by tailoring systems based on different technologies defining their functional requirements and stages of implementation. To better understand the role of solution

architecture in the context of technology development, you first need to know the proposed solution. Even though this might seem quite basic, it is necessary to arrive at a unique and optimal solution for the identified problem. It involves evaluating the problems and addressing them with systems that replace or improve the existing system. In solution architecture, the needs are detailed and mapped to business needs in one way or another. The needs usually crystallize through re-assessing existing systems and finding out how they benefit or harm the organization in the long run. Many times, these evaluations are reviewed with business analysts who also provide a definition of the problem.

In the case of environmental monitoring, it involves alternate technology solution analysis, SOC-based solutions, defining the places where the systems to monitor environmental parameters are to be located, the number of nodes required, the parameter variations observed in the environment and identifying the requirements of the SOC in terms of important parameters to be monitored, their frequency of capturing, accuracy needed and identifying suitable analysis procedures.

In the next step, solution architects take this problem and start crafting a description of solutions that appropriately address this need.

Thus, solution architecture translates technical needs into practical solutions while establishing rules and instructions for proper implementation and delivery. It considers all factors, including external factors that could impact the design and development process. This could be technical feasibility such as processing power, scalability of architecture, availability of additional off-the-chip components that are required in this SOC-based solution. These could be components such as crystals for clock generation, off-chip sensors, and other peripheral components. Availability of them immediately and in the long run, cloud storage requirements to save data when it grows big, and the analysis techniques, the possibility of self-learning and self-correcting machine learning and intelligent algorithms, etc., are studied. This ensures long-term solution to a problem.

In addition to the technical requirements, the general SOC requirements such as expandability, feasibility to use SOC for other similar applications, flexibility for updating analysis methods and algorithms, software upgradability, maintainability, and reliability are few factors which are considered while arriving at SOC specifications.

Solution to monitor environment parameters involves the following steps:

6.2.1 Identifying SOC Requirements

The solution consists of one or many sensor-based processor nodes that can be implemented as an IOT SOCs. The smart IoT SOC solution for environment monitoring must have the following functionalities:

1. Capturing environmental parameters such as temperature and humidity of the surrounding area on demand or at regular intervals. Note that though many parameters can be captured, only two parameters are chosen for the case study. The procedure can be extended to any number of parameters to be captured.
2. The captured parametric data is used to trigger short-term actions such as remotely controlling heating or cooling devices in a closed environment or to get trends and statistics.
3. The sensed and captured data is uploaded to the storage on the cloud server for further processing.
4. Users should be able to access the raw or processed data from the cloud through Android [3]/iOS-based standard devices such as smartphone application.

6.2.2 *Proof of Concept System*

The proof-of-concept of the proposed IoT SOC architecture consists of

- Sensors for measuring temperature and humidity
- Processor to process the captured data
- Communication interface to transfer the processed data onto the cloud server on the internet
- Data visualization setup for the data stored in the cloud
- Mobile/system application to connect to the cloud server and retrieve the data from the cloud server, anytime

The architecture of the POC IoT-based smart environment monitoring system is obviously a processor-based SOC solution with the interface to sensors and communication modules. One such system architecture is shown in Fig. 6.2. It consists of an IoT sensor node whose internal block diagram is shown in light blue. It requires a microcontroller (uC) subsystem with 32-bit processor with

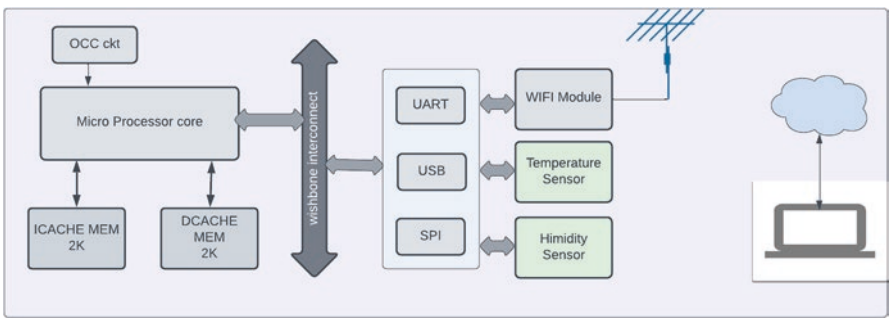


Fig. 6.2 IoT-based smart environment monitoring system

the cache interface to manage the firmware and has interconnection bus on which different peripherals subsystems are connected. The choice of 32 bit processor core is simply to make the IOT light weight as it is used to capture the environment parameters and in this case, temperature and humidity. If more number of parameters to be captured, then 64 bit processor must be explored. The proposed System uses Wi-Fi, Sensors external to the system. Only the signal conditioning circuits are integrated on chip. The Wi-Fi module, humidity, and temperature sensor circuitry can be integrated on-chip. The system is interfaced with a Wi-Fi module which enables two-way wireless data communication to write the data into any cloud storage, in this reference case, MathWorks's thingspeak cloud [2]. The chip processing technologies even permits sensors to be integrated on chip.

Study of external components considering the ease of availability are sensors to monitor temperature and humidity, radio module, and power supply components. The proposed SOC is planned to use 32 bit RISC V processor [1] with the following additional interfaces shown in Fig. 6.3.

- Interface to DHT11 temperature and humidity sensor
- Interface to ESP8266 Wi-Fi module
- 32-bit processor-based SOC replaceable with multiple cores for expansion as data to be processed increases
- Power supply module (which is on the board)

There are several ways the systems are architected for the solution. Some of the important criteria are:

- The processor configuration based on single or multiple processor cores.
- Width of the processor bus on which the processor interfaces the peripherals.

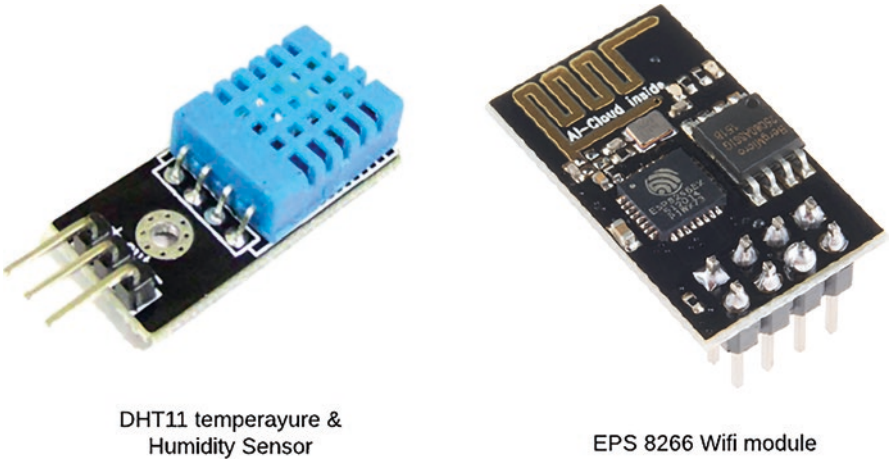


Fig. 6.3 The discrete components used in the IoT device for smart environment monitoring

- If there are many processors on the chip, do we share the on-chip cache memories with them or do we have independent memories associated with the processors?
- The methods of interaction between SOC hardware and software.
- Support for SOC software development platforms. This includes compiler, operating system, and driver development? Can this integrated development environments (IDEs) be generic which can be used for other hardware platforms available in the market?
- Do we provide cloud storage for expandability to store big data to future proof it?

Listed above are some of the considerations in planning the SOC architecture for the solution. These can be techno-commercial decisions. Answering them provide clarity in defining hardware and software system modules of the SOC solution. Let us now answer the identified questions to arrive at the architectural plan for our case study:

1. Processor subsystem for the SOC is a single core, that can be expanded with one or more identical processor subsystems. This covers the present need and the requirements of increased processing of the future. The SOC will be defined as single processor subsystem core which can be expanded as dual core or quad core in future.
2. The processor must support the 32-bit operation and hence has a data width of 32-bit. This decision is a technical feasibility arrived at by determining the internal processing power or bandwidth needed to process the data. The target SOC is a node SOC that collects the sensor data from the environment. The incoming data is sensor data of environment parameters which does not change very rapidly, and periodicity of the data collection will also be twice or thrice a day. The SOC operating at 125Mhz with 32-bit internal bus will provide a data rate of 4Gbps and processing power determined by the processor ISA discussed in earlier chapters.
3. Processor-based SOCs require a software development platform that can be made standard Arduino compatible. This will help adopt this SOC-based solution to any similar application.
4. The WiFi module for the proposed system is external to the SOC and hence the suitable interface is to be developed on chip. WiFi communication module has its own processor used for configuration and communication protocol implementation. Please note that in SOC solution, this can also be integrated as a subsystem if the software core of this module is available.
5. Sensor modules are external to the SOC and are interfaced using standard general-purpose interfaces such as UART, and SPI interfaces. To do this, it requires the interface cores in our IOT SOC. The chosen internal bus is a wishbone interconnect, which is arbitrary for the case study. You can choose a suitable interconnect standard bus which meets the performance requirements. For larger SOCs, it can even be a network on chip (NOC) of high performance onto which many processor subsystem cores are connected.

6. The IoT SOC has its instruction and data stored in the iCache and dCache memories. SOC has an external flash memory that is connected to the wishbone interconnect bus.
7. Wi-Fi module has its own processor, which is accessed through the UART module on the wishbone interface. Wi-Fi module should have enough resources to house the internet software stack, which is standard and compatible with third-party internet software on cloud server storage and analytics applications

The stored data on cloud must be accessed by any device on the internet like a laptop, computer or a smartphone or an iPad. Study on the requirement of such device suggests that to be able to access data on cloud through internet require devices to support Message queuing telemetry transport (MQTT) protocol API or REpresentational State Transfer (REST) APIs protocols as shown in Figure 6.2. MQTT and REST protocols are M2M communication protocols on TCP/IP layer of Internet access. Cloud application is required to perform process the data and support user-friendly way of presenting the processed information on the internet.

6.2.3 IoT Device

The conceptual solution to prove the concept of IoT-based environment monitoring system is shown in Fig. 6.4. As shown in Fig. 6.4, the system with an integrated development environment (IDE), where firmware development is carried out, is interfaced with the IOT_SOC using a serial interface through UART signals.

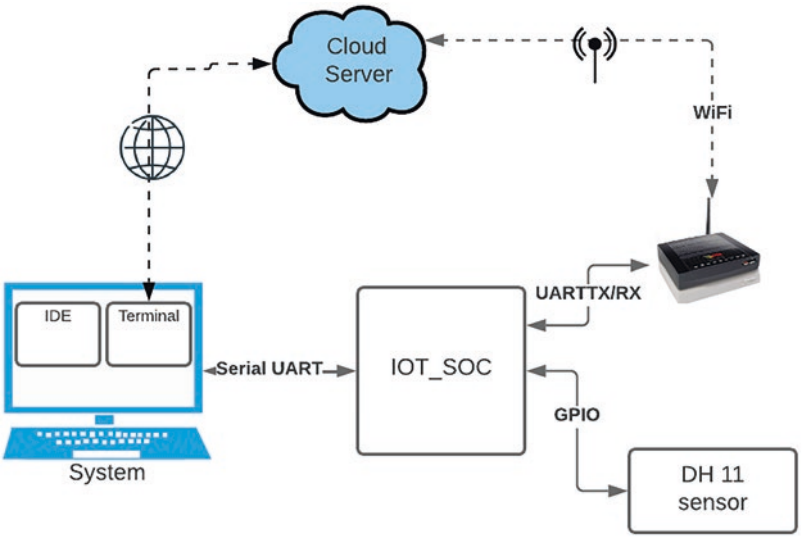


Fig. 6.4 Environment monitoring system setup

Table 6.1 Interface signal description

Sl. No.	Signal name	Signal description	Remarks
DH11 sensor ADC interface			
1	GPIO 0	Sensed temperature or humidity data digital	Sensor data input
2	GPIO 1		
3	GPIO 2		
4	GPIO 3		
5	GPIO 4		
6	GPIO 5		
7	GPIO 6		
8	GPIO 7		
UART for WI-FI module			
1	Txd	UART TX	Data transmitted to Wi-Fi module
2	Rxd	UART RX	Data received from Wi-Fi module
3	GPIO 8	NC	Not used in this design and hence not connected
4	GPIO_9	For controlling chip enable	Chip enable High: normal Low: low power mode
5	GPIO 9	NC	High: normal functional mode Low: sets Wi-Fi module to bootloader mode
System generic signals			
1	Ext_Rstb		Active low reset
2	Xtal in	Reference Xtal input	48Mhz Xtal input for clock generation
3	Xtal in		
4	Vcc		
5	Gnd		
IOT Soft UART			
1	UART_ Rx	UART_RX	From software Terminal application to get data. Serial in signal from UART
2	UART_ Tx	UART_TX	To software terminal application to display the sensed output. Serial out signal through UART
IOT SPI S			
1	SCL	Serial clock	SPI clock
2	SDA	Serial data	Serial data input-output
IOT SPI M			
1	SCK	Serial clock	SPI clock
2	MOSI	Serial data in	
3	MISO	Serial data out	

Temperature-Humidity sensor DH11 and the Wi-Fi module EPS8622 are interfaced to IOT_SOC through GPIO signal as shown in the interface signal description Table 6.1. The firmware program when executed will function as an application program which will enable temperature-humidity data from the DH 11 module to be captured every minute and transmit to the cloud Thing speak platform through the EPS 8622 WiFi module connected to the Arduino Uno board. The Things speak cloud service is used to visualize the updated data transmitted from anywhere, anytime, on smart devices or systems through the internet by the registered user.

6.2.3.1 IOT SOC

The IOT SOC used in the Environment monitoring system must therefore contain the means to power on, clock, and interfaces to sensor modules and communication modules. The other features to be supported are system booting, storage for programs and data to be processed. The processor core can be any RISC processor such as RISC V and interface and communication cores are to be protocol cores such as UART, USB, SPI, I2C master and slave configurations. The internal block diagram of IOT SOC is shown in Fig. 6.5.

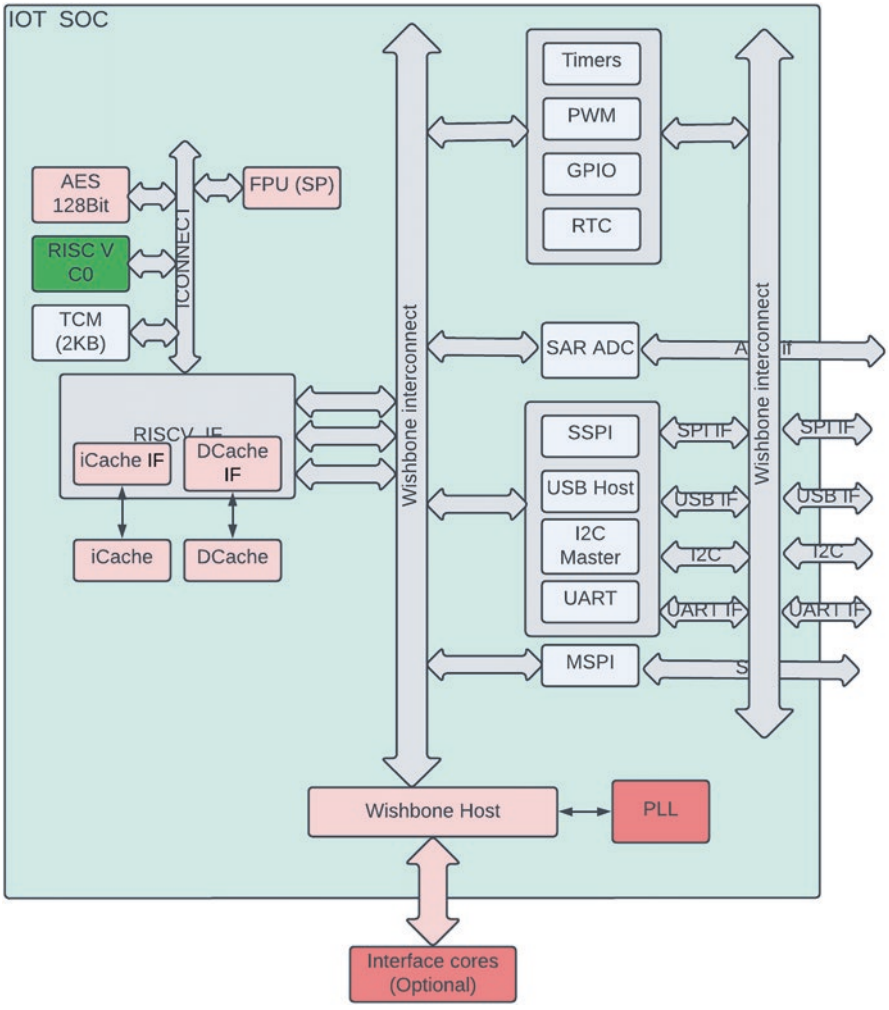


Fig. 6.5 Internal block diagram of IOT SOC.

Note that there are extra interfaces with interface cores defined in the IOT SOC considering the expandability and much-needed flexibility. This is to make it as generic as possible so that it can be used as a general IOT SOC not only for environment monitoring.

The RISC core with RISC interface for cache memory design is shown in Fig. 6.6.

The important input-output interface signals from different modules and systems are shown in Table 6.1. The table does not cover the RISC V interface signals for Processor memory interface.

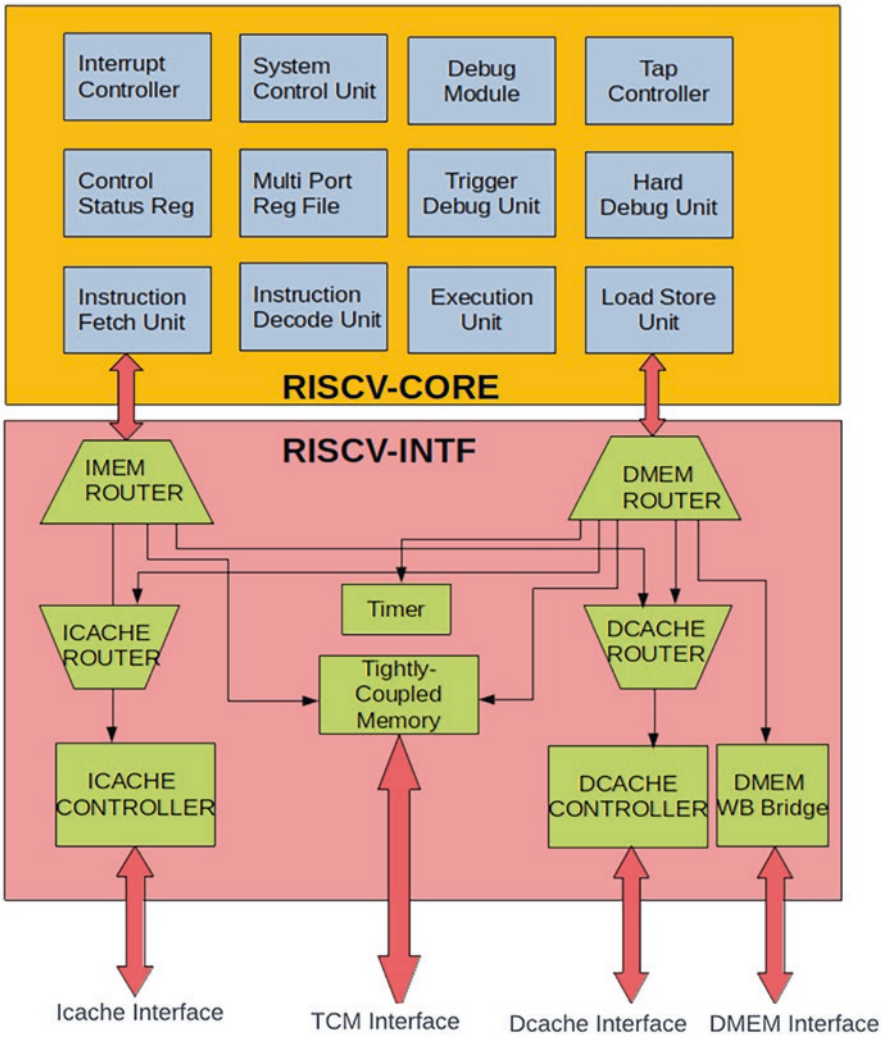


Fig. 6.6 RISC V Cache memory design

6.2.3.2 IoT Device Firmware

It is easy to develop software using standard software development environment (SDE) if the IOT SOC is made pin compatible with the existing processors available. Though this is not necessary, it reduces development time and makes the solution more suitable for similar generic applications. This consideration is used to make IOT-SOC, compatible to integrated development environment (IDE) of Arduino processor. For example, if the SOC pin set is made as drop-in compatible with existing standard processors such as Arduino, and its processor engine based on Arduino instruction set, standard Arduino IDE can be used for the development of software. This enables easy development of software and interface drivers for IOT SOC interface cores such as SPI, UART, and USB which are most likely to be available in the Arduino library. Making IOT SOC design compatible with standard IDE has many advantages apart from faster development of software for the application. It enables the use of already available software functional blocks from the library, which are developed by a third party making it interoperable with other similar hardware systems. The use of library module is as simple as including them as the header files. These are called ***shields*** in Arduino terminology. This IOT SOC needs Wi-Fi shield, DH 11 sensor shields to be included in the firmware program with the ***#include*** commands and application-specific code for connecting to a Wi-Fi modem and accessing cloud services.

6.2.3.3 Scalability

SOC architecture must consider the increase in processing requirements of the future in terms of adding many IOT devices for environmental monitoring. This requires consideration regarding the aggregation needs of data which translates to an increase in the processing power of the system. This may require increased on-chip storage and processing power. On-chip storage of the system is addressed by deciding well thought of memory interface for expandable external memory which requires a wider or multiplexed bus on the system bus in the memory controller on the system. An increase in processing power is easy to be done by making the architecture scalable by adding one or more processor subsystems. The provision of making the SOC dual core, and quad-core are considered while defining base architecture. Figs. 6.7 and 6.8 show the dual core and quad core configurations of the expansion of processing capabilities of IOT SOC.

6.2.3.4 Hardware Software Partition

Hardware Software Partition is the major activity of SOC architecture definition.

It is the process of dividing the probable computations of a system into a part that executes as sequential instructions on a processor (software) and a part that runs as

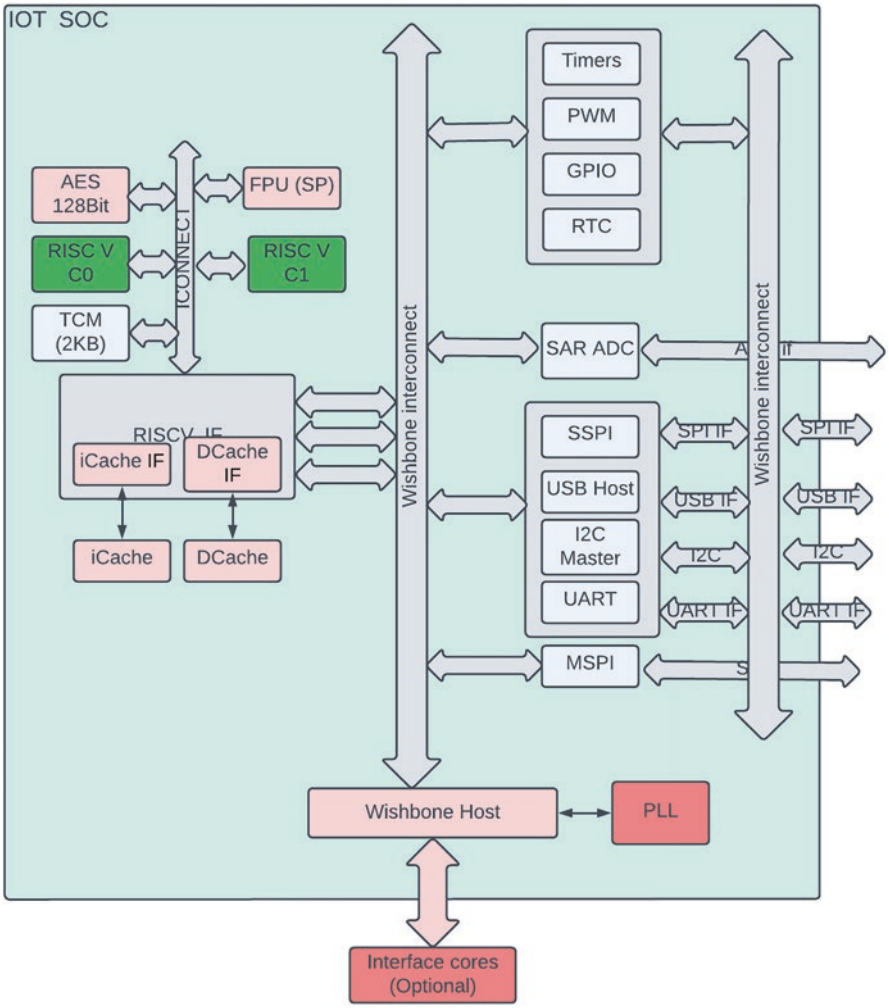


Fig. 6.7 Dual core configuration of IOT SOC system

concurrent logic on a SOC (hardware), achieving system metrics like performance, power, size, and cost.

HW/SW partitioning enables HW/SW co-design to ensure system performance as embedded system-on-chip (SoC) development. The partitioning is done during architecture definition at the stage where there is a possibility of exploring many alternatives. The hardware-software partitioning tries to exploit the synergy of hardware and software.

The classic HW/SW partitioning process includes the system functions that interact closely as defined into hardware and software components. The hardware and software are developed separately. The "Hardware First" approach is often adopted. Some of the implications of these features are that the HW/SW trade-offs

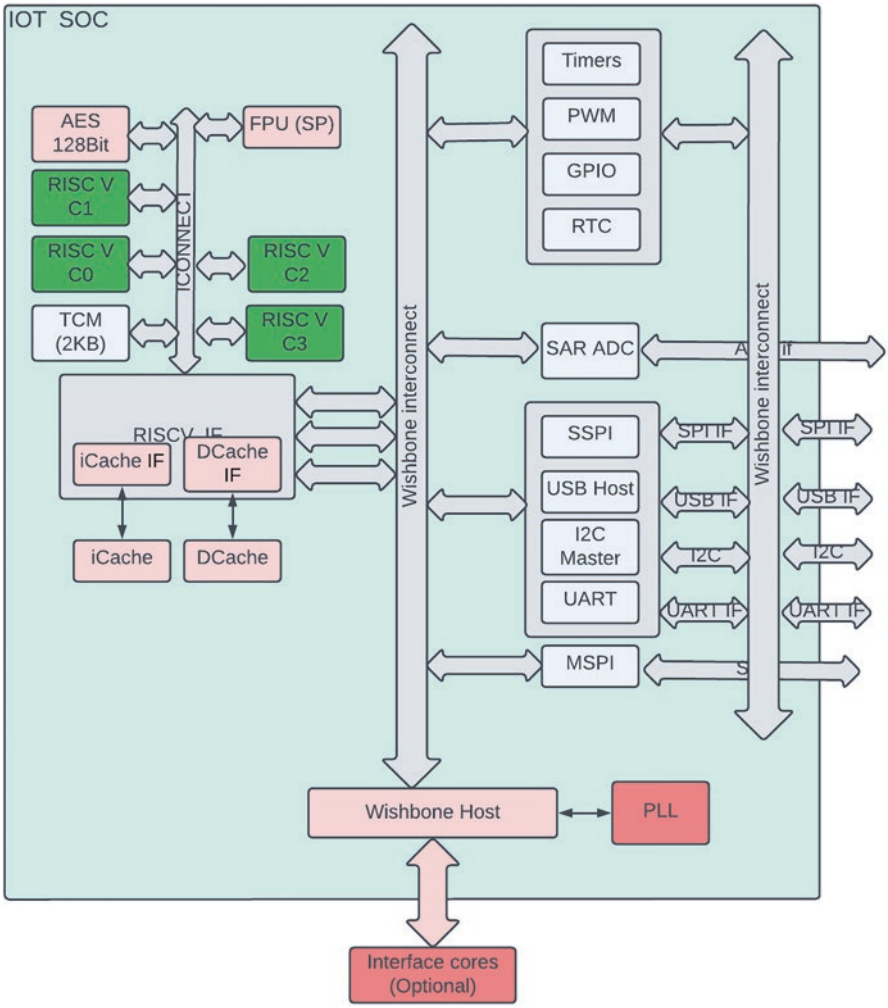


Fig. 6.8 Quad core configuration of IOT SOC system

are restricted. The impact of HW and SW on each other cannot be assessed easily and is mostly expensive. Consequences of incorrect partition are poor quality designs, frequent modifications during development involving cost, and increased time to market. Partitioning has a dramatic impact on the cost and performance of the system.

The goal of HW/SW partitioning in SOC architecture is to improve overall system performance, reliability, and cost-effectiveness because defects found in hardware can be corrected before it is taped out. Partitioning benefits the design of embedded systems and SoCs, which need the HW/SW tailored for a particular application.

Some of the key benefits of HW/SW partitioning are:

- Clarity of functional requirements
- Flexibility of the system
- Faster and better integration
  - Reduced design time and cost
  - Better performance
  - Correct by design

HW/SW partitioning considers architectural details such as:

- Type of processors
- Type of interactions
  - Interrupts or polling
- Memory organization
- Access to other hardware resources like
  - Configuration and status registers
  - Scratchpad memory

The main objectives for partitioning are increased frequency of operation, latency requirement, silicon size, and cost.

HW/SW partitioning allows alternate designs (HW/SW partitions) to be explored and evaluated to determine the best implementation for a system including partitioning applied to system modules to best meet design criteria (functionality and performance goals).

Key issues faced during the HW/SW partitioning are the partition algorithm and the hardware-software estimation. Multi-core processing is recognized as a key component for continued performance improvements. Single-core SOCs are showing a diminishing ability to increase product performance at pace with the complex system requirements. Therefore, the use of multi-core systems becomes essential. Multi-core processing involves complex changes to systems and software to obtain optimal performance. Partitioning these multi-cores is the central challenge of developing a multi-core system environment.

Implementing a system in which complex computational functions are executed concurrently in a computing environment and maintaining their order or sequence of execution is a complex problem. In such cases, partitioning the functions among the multiple cores provide a path forward for increased performance. This part of the designing requires comprehensive and pervasive system and software changes, as well as new and innovative hardware designs to ensure that the software can take advantage of the increased computational power.

Along with this is the use of multi-core IP to be integrated on a single chip. The buses and point-to-point communication architecture are replaced by network-on-chip (NoC) in system-on-chip (SoC). NoC handles the communication of hundreds

of cores and allows several transactions concurrently. The NoC topology determines performance parameters such as area, power consumption, latency, and throughput. System topology describes the arrangement of the multiple cores and their coordination. NOC-based architecture topology are many types. Major ones are the ***mesh partition topology*** and the ***ring partition topology***. Mesh topology is typically used for reducing silicon area, and ring topology for latency. The ***heterogeneous*** and ***hybrid clustered topology*** is used to reduce the average latency, and response time and keeps the area of the topology constant. There will always be a trade-off between the partition topology used and the desired performance metrics. The choice of topology depends on the system solution in an application.

6.3 IOT SOC Hardware Software Partition

In an integrated SOC architecture, a single-core partitions are defined for many functions. The reduction in the number of nodes and networks because of the integration leads to an increase in hardware reliability and a decrease in hardware cost. An integrated distributed architecture for mixed-criticality applications must be decided on a core design that supports the safety requirements of the highest considered criticality class. In the case of IOT SOC architecture, the SOC uses two processor subsystems. Main processor subsystem is used for processing the environment parameter processing which may also be referred as application processing. This is the main processor responsible for system booting, data processing, and handing over the processed data to the communication processor on WLAN. The other subsystem is the communication processor subsystem which is Wi-Fi subsystem which has its own processor to take care of processing the WLAN protocol and necessary data processing.

Partitioning into hardware software for complex SOCs is done using two approaches, software-oriented partitioning, and hardware-oriented partitioning. The difference between the two is whether the functional specification model is written as a software model or a hardware model. There are tools available for modeling such systems.

The IOT SOC is a simple SOC, the hardware-software partitioning is carried out by criticality of process functions. For example, in communication processor Wi-Fi subsystem, the link establishment, link maintenance, and WLAN packet processing is done in hardware and the managing of the network configuration, enabling communication security features are performed by software. In application processor, the processing real-time environment data captured by the sensor, validating it to be the correct, and recognizing the parameter are carried out in hardware and converting it into data packet to be communicated over to the cloud by time stamping it, adding location and node identities are carried out by the software.

6.3.1 System Design Plan

Once the SOC subsystems are identified for a chosen architecture, the chip design and software design activities are independent but go hand in hand as each of them has its own challenges. The chip and software functionalities are validated in co-verification, validation environments at different stages of development flow.

References

1. https://github.com/dineshannayya/riscduino/
2. www.mathworks.com
3. https://developer.android.com

Chapter 7
IOT SOC Architecture Definition



7.1 Chip Architecture Flow

Proof of concept system is developed to study, analyse and gain enough confidence to ensure that the actual system that will be developed serves as the solution for the identified problem without any limitation. After this stage, the System on Chip (SOC) architecture is defined. Chip architecture relies on the hardware-software partition of the functionalities of the solution. It involves identifying functionalities that can be integrated onto the chip, to become the System on Chip (SOC). Traditionally, most system functions except for batteries, and user interfaces like displays and keypads are integrated onto the chip. This include all the necessary configurations and control functions and the necessary interfaces on SOC to be integrated for connecting power supply, display and keypads, external components and expandable additional memories for the complete solution

The SOC architecture definition flow involves the following steps:

- Drafting SOC specification.
- Defining SOC data flow architecture.
- Target library selection.
    - Selection of technology node.
    - Power and frequency selection.
    - Layout density entitlement.
- IP selection.
    - Hard and soft IPs.
    - Internal or external IP.
    - Embedded RAM/ROM.
    - PLLs and analog cores.
    - Custom cores required.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

V. S. Chakravarthi, S. R. Koteshwar, *System on Chip (SOC) Architecture*,
https://doi.org/10.1007/978-3-031-36242-2_7

- SOC-level power estimation.
- Defining verification environment.
- Selection of configuration management environment.
- IO selection.
- Package options evaluation and selection.
- Tool flow definition.
- Die size estimation and trade-offs.
  - Preliminary floorplan.
  - Analysis of choices of IO limited vs. Core limited vs. Macro limited vs. Package limited SOCs and selection.

7.2 Chip Specification

The process of defining the SOC specification involves converting the functional requirements into clear implementable technical specifications. Technical specifications of all the subsystems, internal functional blocks, intermodule interfaces, on-chip interface signals to interact with external components in the system are defined. Apart from defining functional specifications, this stage involves identifying and defining specifications considering the applicable safety and regulatory standards for the proposed solution. The chip specification also includes defining design performance goals for the on chip part of the solution. This is the main goals to be targeted during the chip design. The performance goals include power the chip can consumption (lower the better), target speed performance and softwares to be supported for interfacing with the user and external components. In addition, the specifications regarding testability and reliability, design for verification features, and interoperability specifications for the chip are defined. A typical canonical SOC architecture is shown in Fig. 7.1.

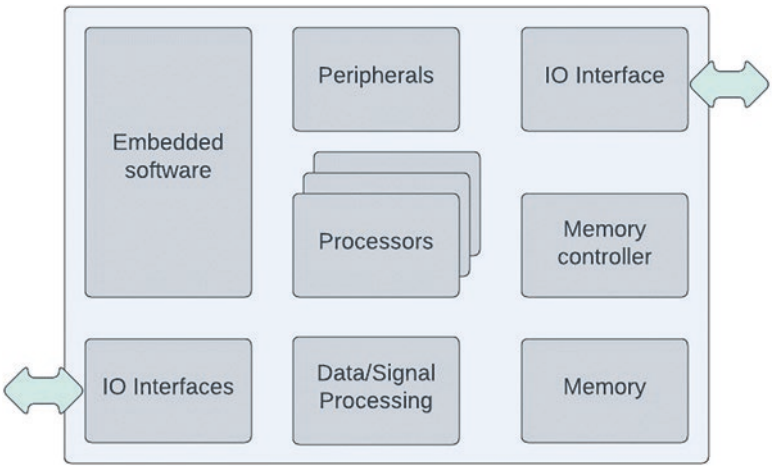


Fig. 7.1 Canonical SOC architecture

The on-chip system components of the architecture can be of many formats. The critical sub blocks are designed completely or acquired as third-party off-the-shelf ready IP core. They are often reusable across SOCs. Some examples of virtual IP cores which are readily available for integration onto the chip are:

- Processor core.
- Signal processing cores for Audio/Video signals.
- Accelerators.
- Analog to Digital Converters/Digital to Analog Converter (ADC/DAC)s.
- Memory Controllers like DDR II controllers.
- Flash Memory Blocks
- Interface cores such as USB, SPI, I2C, Transceivers, Serdes, PLL, DLL, Physical signal processing blocks, and Analog front end (AFE) blocks.

Make or buy decisions of the IP cores are based on the design and development strategies adopted for the SOC. It depends on the project schedule of the design. Accordingly, it translates to whether you want the design to be done fast by integrating proven off-the-shelf cores or develop them in-house from the scratch. Deciding to design complete chip in house gives complete freedom to the designers to achieve the target chip performance applying relevant design strategies within the time to market targeted. On the other hand, if the decision is to buy ready virtual pre-validated IP cores, it is still required to develop wrapper or glue logic for integration to SOC architecture. Most used practice is to buy some of the standard IP cores from the IP companies off the shelf, and develop some of the critical blocks as proprietary cores, as in the example shown in Fig. 7.2. In the Wi-Fi SOC

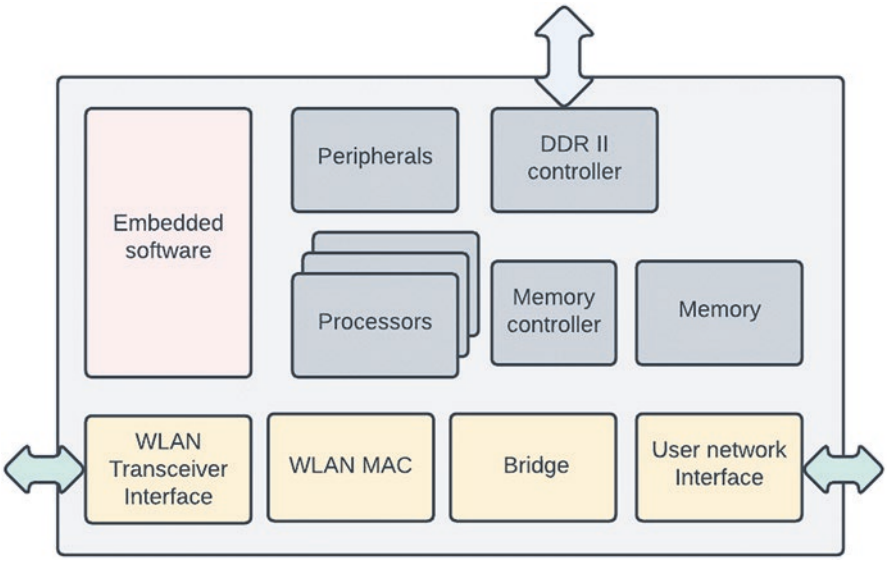


Fig. 7.2 Wi-Fi SOC subsystem for IOT SOC

subsystem of the IOT SOC shown, the IP cores colored in gray are readily available to be acquired, and the ones in peach color are planned to be designed. This includes RISC processor core subsystem.

The pre-validated IP cores interact with the newly-designed cores through the glue logic developed to interface them into the SOC data path architecture.

The virtual IP cores are often referred to with equivalent terms interchangeably as *cores*, *IPs, Macros,* or *virtual components (VCs)*. They are of three types depending on the stage of design. They are as follows:

- Hard IP cores.
- Soft IP cores.
- Firm IP cores.

7.3 Hard IP Cores

Hard IP cores are hard macros which are not customizable during integration into SOC. They are available in physical layout form and are 'ready to fabricate' cores with only input-output signals (IO)s available for interfacing with the rest of the blocks on the system. They are available as IPs in GDS II, which is to be placed and interconnected by the process of routing during the physical design stage of the SOC development. The internal logic of IP cores cannot be accessed by any means. It appears as a black box or is hidden from viewing/ modifying, but functionality is accessible for the integrated verification of the SOC. The performance parameters like the speed of operation, area, orientation and power consumption of the hard IP cores are pre-validated, specified, and cannot be altered. They are targeted to a specific fabrication process and has specific size on the layout. The hard macros are used for integration by complying to the physical and timing constraints specified for them. Figure 7.3 shows a hard IP core. Some examples of hard IP cores are Processor cores, Phase locked loop (PLL), and DLL.

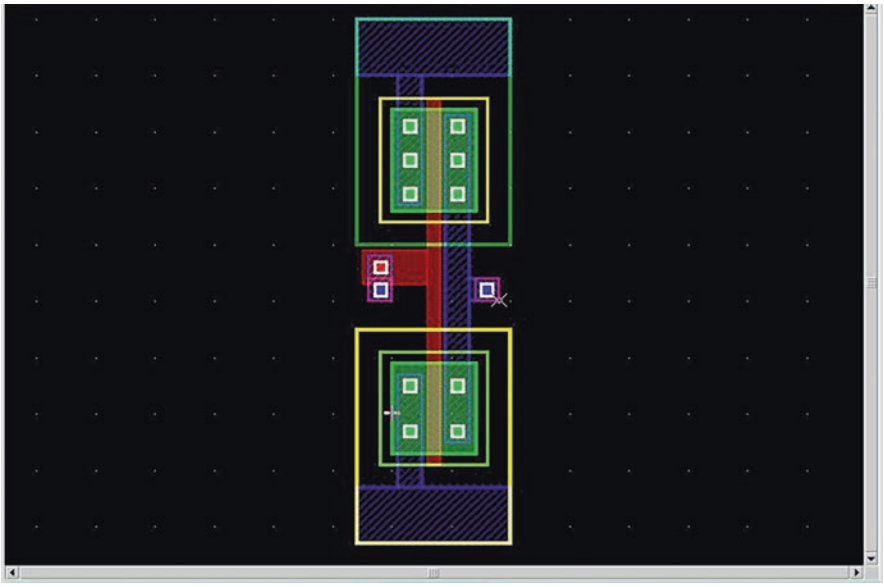


Fig. 7.3 Hard IP Core

7.4 Soft IP Cores

Soft IP core is the synthesizable RTL core that can be integrated onto the SOC architecture. Softcores have the corresponding test bench cores with test cases and an executable script to run the test cases for verification. These IPs can be modified to achieve the overall system performance. The performance of the Soft IP core depends on the design timing, power and physical constraints, synthesis process and the target technology library. RTL model of soft IP can be in any hardware description language such as Verilog/VHDL or SystemVerilog. The Soft IP core comes with a synthesis script, design constraints, DFT insertion scripts (scan insertion and ATPG), and verification environment as deliverables. An example of the Soft IP core with a function of sequence detector of 10,101 in Verilog HDL is shown in Fig. 7.4. and the test bench model of the sequencer in Verilog HDL is shown in Fig. 7.5.

The test bench for the sequence detector Soft IP core most times will not be synthesizable as it is used only for verification.

Some of the examples of Soft IP are multipliers, timers, Interface cores such as UART, USB, and SPI and sometimes processor cores. Small functional modules like multipliers, timers etc are available as technology library components or user defined primitive (UDP)s or complex cells. The performance of the soft IP core is dependent on targeted process technology and the tool flow.

```
/*******************************
Sequence detector of 10101
Inputs: Serial input data
Outputs: sequence_detected
Function: The design works to detect the sequence 10101 for which the output
signal sequence_detected will be high.
Design file: fsm.v
************************************
// Module works only to detect the sequence 10101
// This is sequential block which require clock and reset
// User can refer to any Verilog HDL language book to understand the syntax of
// commands.
************************************/
// Sequence detector of 10101 without overlap
module fsm (
//------------------clock_reset-----------------//
                                        clk          ,
                                        reset_n    ,
//----------------Input---------------------//
                                        input_data ,
//--------------Output----------------------//
           seq_detected
        );
//------------------clock_reset-----------------//
input                          clk , reset_n  ;
//----------------Input---------------------//
input              input_data     ;
//--------------Output----------------------//
output          seq_detected  ;

reg  [2:0]   curr_state       ,
             next_state       ;

parameter    IDLE   =3'd0 ,
              SEQ_A  =3'd1 ,
              SEQ_B  =3'd2 ,
             SEQ_C  =3'd3 ,
                 SEQ_D  =3'd4 ;

//------------------next_state_logic-------------------------------//
always@ ( curr_state   , input_data   )
begin
   case (curr_state)

       IDLE  : if (input_data)
                    next_state= SEQ_A ;
                  else
                    next_state= IDLE;

       SEQ_A : if (!input_data)
                    next_state =SEQ_B ;
                  else
                  next_state =SEQ_A ;
```

Fig. 7.4 Sequence detector (sequence: 10101)

```
        SEQ_B  : if (input_data)
                   next_state = SEQ_C ;
                 else
                   next_state =IDLE ;

        SEQ_C  : if (!input_data)
                    next_state = SEQ_D;
                   else
                   next_state=SEQ_A ;

        SEQ_D : if (input_data )
                 next_state = SEQ_A;
                   else
                 next_state = IDLE ;
    default : next_state = IDLE ;
    endcase
end

//-------------CURRENT_STATE_LOGIC-------------------------//
always@ (posedge clk or negedge reset_n)
begin
  if (!reset_n) begin
     curr_state<=3'd0 ;
 end
else begin

     curr_state<=next_state;
   end
end

//------------output_logic---------------------------//
assign  seq_detected = (curr_state==SEQ_D && input_data);
endmodule
```

Fig. 7.4 (continued)

```
/*******************************
Inputs: Nil
Outputs: Nil
Function: The test bench applies random values and detects the sequence. The
waveform fsm_tb.vcd can be observed using a waveform viewer.
Test bench file: fsm_tb.v
*******************************/
module fsm_tb;
reg Clk;
reg Reset_n;
reg [8:0] sequence;
reg data_in;
wire seq_detected;
//clock generation
  always #5 Clk = ~Clk;

  initial
  begin
   Clk = 0;
   Reset_n = 1;

   $display("--------- Test  Started ---------");
   #10  Reset_n = 0;
   #10  Reset_n = 1;

$display("--------- Sending Data sequence 111010101  ---------");
@ (posedge Clk);
 sequence = 9'b111010101;

 #10 data_in = sequence[8];
#10 data_in = sequence[7];
#10 data_in = sequence[6];
#10 data_in = sequence[5];
#10 data_in = sequence[4];
#10 data_in = sequence[3];
#10 data_in = sequence[2];
#10 data_in = sequence[1];
#10 data_in = sequence[0];

$display("--------- Sending Data sequence 110010101  ---------");
        @ (posedge Clk);
  sequence = 9'b110010101;
      data_in = sequence[8];
#10 data_in = sequence[7];
#10 data_in = sequence[6];
#10 data_in = sequence[5];
#10 data_in = sequence[4];
#10 data_in = sequence[3];
```

Fig. 7.5 Test bench of the sequencer module

```
#10 data_in = sequence[2];
#10 data_in = sequence[1];
#10 data_in = sequence[0];

$display("--------- Sending Data sequence 101010101  ---------");

  sequence = 9'b101010101;
@ (posedge Clk);
#10 data_in = sequence[8];
#10 data_in = sequence[7];
#10 data_in = sequence[6];
#10 data_in = sequence[5];
#10 data_in = sequence[4];
#10 data_in = sequence[3];
#10 data_in = sequence[2];
#10 data_in = sequence[1];
#10 data_in = sequence[0];
$display("--------- Test Ended ---------");
  #1000  $finish;
end

fsm u_fsm(
                .clk(Clk),              // Clock input of the design
                .reset_n(Reset_n),// active low, synchronous Reset input
                .input_data(data_in),// Input data bit.
               .seq_detected(seq_detected)// sequence detected

          );                    // End of port list
 initial
 begin
    $dumpfile ("fsm_tb.vcd");
    $dumpvars (0,fsm_tb);
  end
endmodule
```

Fig. 7.5 (continued)

7.5 Firm IP Core

Firm IP core lies somewhere in between hard and soft IP cores in terms of flexibility. Firm IP cores exist as RTL or design netlist and the integration designer will have the flexibility to alter the design constraint to fine-tune the performance goals. Many times, these cores have limited use only to the specific process technologies.

7.6 Chip Data Flow Architecture

After the system functions are identified, and corresponding functional IP cores are decided, the data path architecture is defined around the chosen IP cores. A data path is a set of functional processing units organized in sequence or in concurrence to process the incoming data to derive the necessary output data. The sequence in which the data processing is performed on incoming data is controlled by the control signals generated across the data path as it traverses from one set of processing to the other till the final output is generated. The data path with the necessary control unit forms a computing system. Larger data paths are created by connecting more than one data path and integrating them together using multiplexers. Typical processor data path contains the following components:

- Processors, which are mainly active processing elements in a system which does all data manipulations and decision making.
- Data path is a logic path which consists of logic elements such as ALU, internal bus, memories, registers.
- Control path is hardware elements that determine the datapath what to do on the data, in terms of switching, selection of operation, data movement between different processing elements such as ALU components, etc.

Figure 7.6 shows the simplified processor-based datapath architecture.
A basic processor system in a data path architecture is shown in Fig. 7.7.
Data path in an application-specific processor SOC contains more than the processor path elements discussed in earlier sections. It will contain functional elements not limited to the following components:

- Analog front end or interface block, which captures the sensor signals and conditions them to make them suitable for further processing.
- Digital signal processing block, which selects desired part of data by filtering and extracting relevant parts of data or signals for processing.

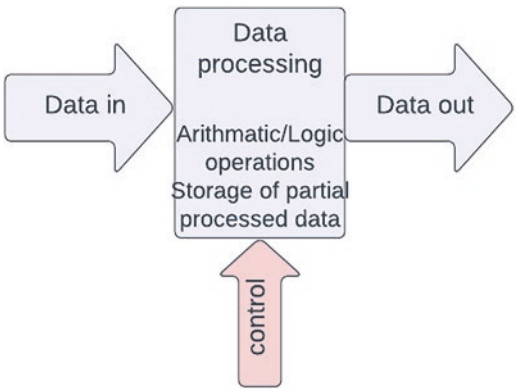


Fig. 7.6 Simplified data path architecture

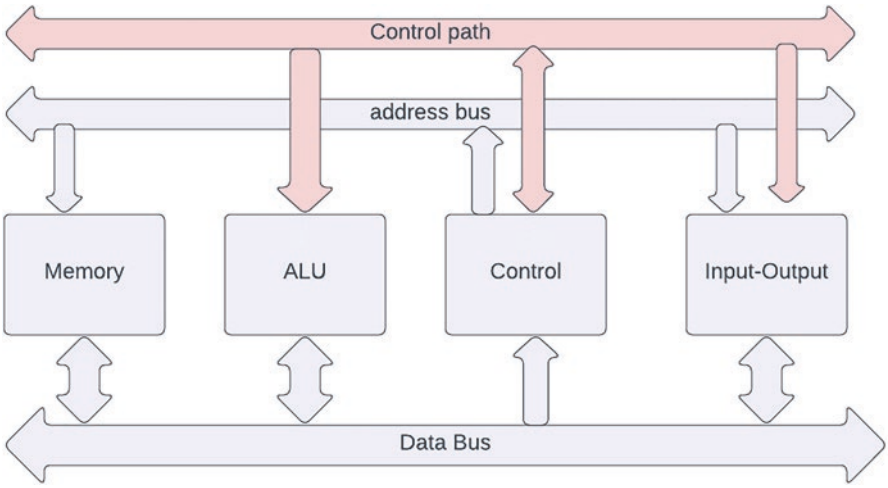


Fig. 7.7 General processor system data path architecture

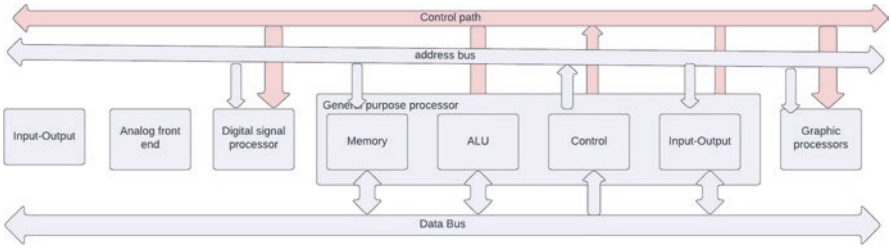


Fig. 7.8 Data path architecture of application-specific system

- General processor containing its own data path which configures the entire system elements and prepares the data path for processing as required for the application. This can contain all the components discussed above.
- Data classifiers which identify the incoming data in terms of packets and classify them to be controlled or prepare the data for further processing or route the data to desired data path for processing.
- Special processors which prepare the data suitable for the chosen user interface, such as display, and audio-video format.

Figure 7.8 shows an example of an application-specific data path System on Chip architecture.

7.7 Case Study of SOC for Environmental Monitoring

Continuing our discussion on the study of environmental monitoring, here we define the SOC architecture of the solution. As discussed in Chap. 6, the hardware partition requires a processor subsystem, sensors to capture the temperature, humidity of the environment, Wi-Fi transceiver. The off-chip components are power supply module and the oscillator circuit to feed the system clock to the system.

The chip consists of a processor core which was dealt in detail in the previous chapter, and the peripheral subsystem. The peripheral subsystem consisting of on-chip memory for storing instruction and captured and processed data. Let us call these as (ICache) and data cache (DCache). They are connected to the processor through the internal bus on to which the other subsystems also get connected. The block diagram of the processor subsystem of IOT SOC under consideration is shown in Fig. 7.9.

The processor core is a single 32-bit RISC V core. The SOC architecture is defined using the RISC V processor core with 2 KB tightly coupled memory (TCM), AES core with a floating-point unit (FPU). The RISC V processor can access 2 K

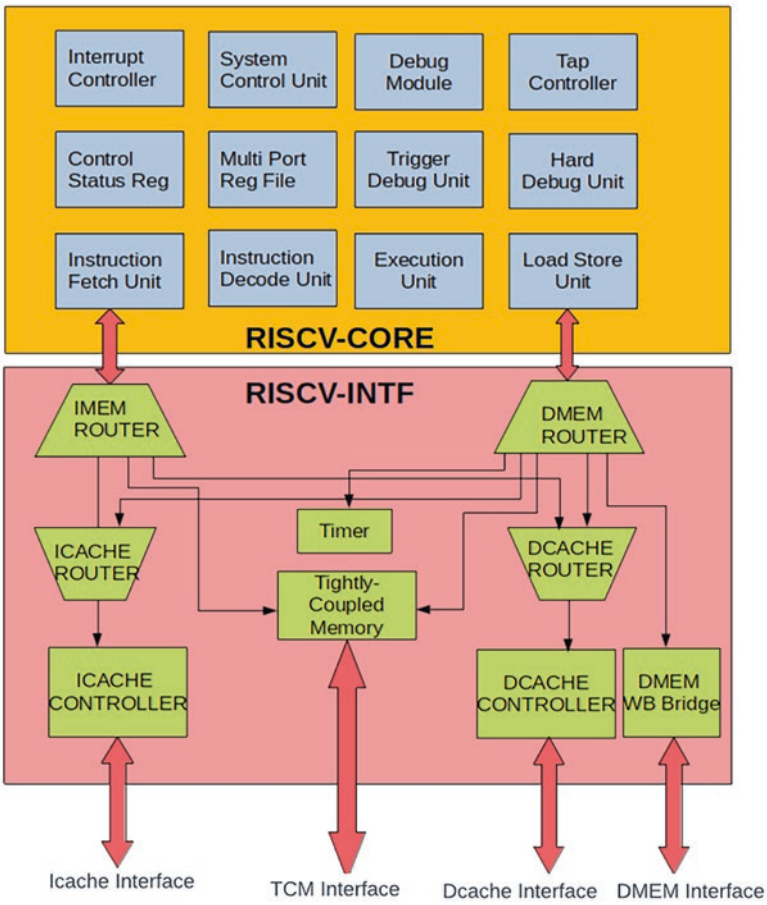


Fig. 7.9 SOC block diagram

bytes of on-chip memories for instruction cache (ICache) and Data cache (DCache) each through Wishbone interconnects. The SOC is made pin compatible with Arduino module so that the standard Arduino integrated development environment (IDE) can be used for software development and debugging. Having a bus-based architecture enables all standard interface cores to be integrated into the system bus as most sensor modules have standard interfaces such as I2C or UART. The example shown uses Wish bone interconnect as a system bus on which the peripherals are connected. The other interface cores integrated onto the Wish bone bus are Wish bone host, USB, SPI-master and SPI-slave, UART-master and UART slave, Six channel ADC core apart from standard cores like real-time clock (RTC), timers, 32-bit general-purpose IOs (GPIOs), PWM. Standard subsystems such as PLL, RTC, timers, and PWM are used by the real-time operating system (RTOS) in the software stack for logic synchronization and housekeeping of the system. The internal block diagram of the IOT SOC with these considerations is shown in Fig. 7.10.

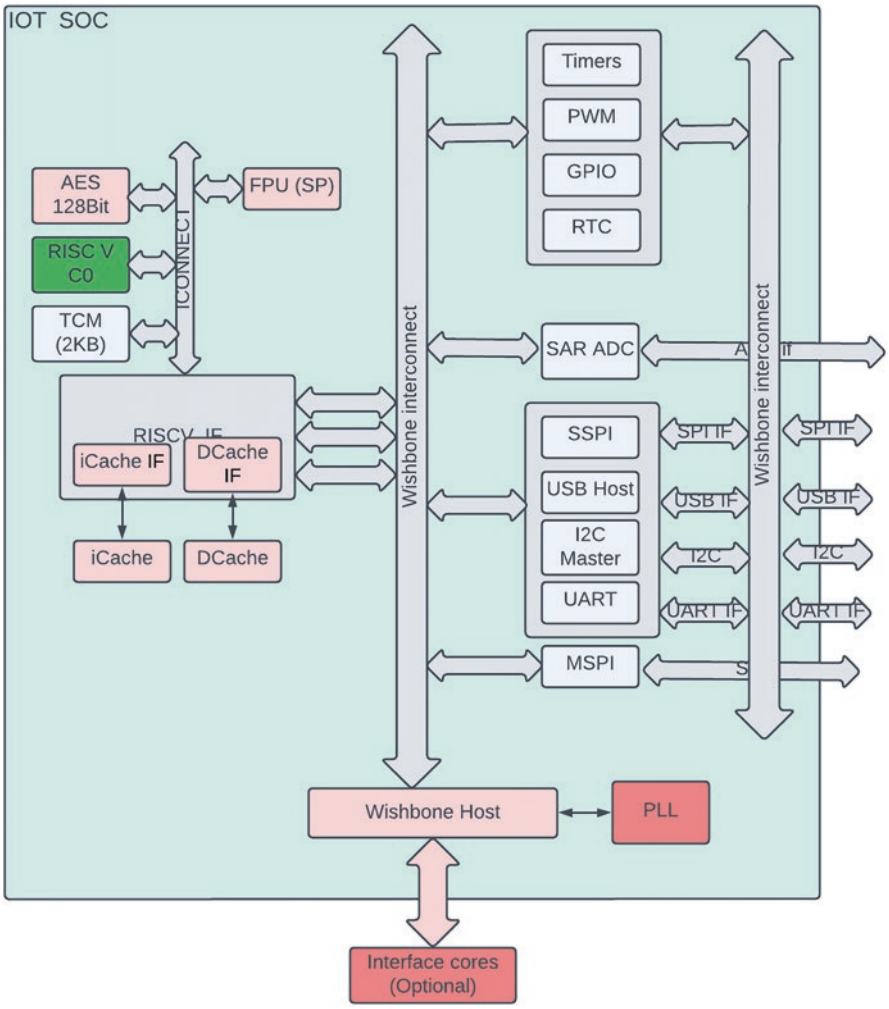


Fig. 7.10 IOT SOC internal block diagram

7.7.1 SOC Architecture as Standard Data Path

Standard practice is to define the system architecture as the data path and control architectures. Many functional blocks of a complex digital system, known as subsystems or processing elements (PEs), are partitioned so that they are designed and verified independently. This is most followed approach for any complex system. They are partitioned into subsystems. These subsystems contain resources such as adders, subtractors, multipliers, dividers, comparators, and shift registers are identified which are required for data processing in an application. This is called top-down approach. The general consideration in such SOC architectures is similar size and high regularity, and simple interconnection to other subsystems. Minimum interactions between submodules are another consideration for partitioning to subsystems. Subsystems working at high frequency are placed in physical proximity to avoid other interconnect-related challenges during design implementation.

Chapter 8
SOC Software



8.1 Software Development Life Cycle (SDLC)

Software development life cycle (SDLC) defines the development cycle of the SOC software in a structured way to ensure high quality, optimum cost, and minimum time. Any software is expected to be developed with high quality in a particular time duration and within an allocated budget. This requires thorough planning of every phase of software development. SDLC consists of planning and explaining each phase of the development process. Every phase of the SDLC life cycle has its own process and deliverables that feed into the next phase. SDLC is also referred to as the *Application Development life cycle*. SDLC provides a framework for a standard set of activities and deliverables.

SDLC phases are shown in Fig. 8.1, and they are as follows:

8.1.1 Phase 1: Requirement Collection and Analysis

The Software requirement collection is the first phase in the SDLC process. Software requirement collection and analysis is typically conducted with inputs from all the stakeholders and domain experts in the industry. This phase involves defining the quality assurance requirements and identifying the risks involved in not meeting the requirements. This helps to plan the development process with the necessary quality checks during the development.

It is to get a clear idea about the scope of the entire software development and it helps to foresee the possible issues, opportunities, and facilitates the development by including the risks and coming up with the mitigation plan. These are captured in Software requirement Specification (SRS) Document.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

V. S. Chakravarthi, S. R. Koteshwar, *System on Chip (SOC) Architecture*,
https://doi.org/10.1007/978-3-031-36242-2_8

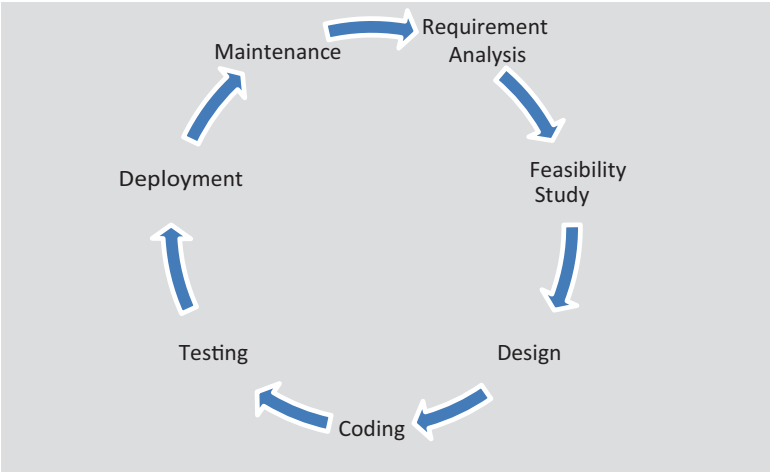


Fig. 8.1 Software development life cycle (SDLC)

Gathering the Software requirements helps development team members to understand the detailed and precise requirements. This helps in project planning especially in determining project schedules and deliverables.

8.1.2 Phase 2: Feasibility Study

This stage involves defining the software needs by documenting the requirements in detail. It uses the "Software Requirement Specification" (SRS) document as input. It includes every function to be designed and developed during the project life cycle.

There are five types of feasibility checks:

- *Economic*: What will be the development cost?
- *Compliance requirements*: What are the applicable technical and regulatory standards to be complied with?
- *Development feasibility*: Can we develop functional blocks as intended?
- *Technical*: Do we have the right resources for the development, including systems, development platforms, and human resources with the right skills?
- *Schedule*: Breaking down each functional blocks and determining its duration for development and thus arriving at the overall project costs.

8.1.3 Phase 3: Software Design

In this phase, the system and software design documents are prepared per the requirement specification document. This helps define overall system architecture.

There are two kinds of design documents developed in this phase:

8.2 High-Level Design (HLD)

High level design document details the design approach to be followed for the functional blocks and how these are integrated to implement the overall system software level functionality.

- Brief description and name of each module.
- An outline of the functionality of every module.
- Interface relationship and dependencies between modules.
- Database tables identified along with their key elements.
- Complete architecture diagrams along with technology details.

8.3 Low-Level Design (LLD)

- Functional logic of the modules with internal state machines if any.
- Database tables, which include type and size.
- Complete detail of the interface.
- Addresses all types of dependency issues.
- Listing of error messages.
- Complete input and outputs for every module.

8.3.1 Phase 4: Coding

Once the system design phase is over, the next phase is coding. In this phase, developers start to build the entire system by writing code using the chosen programming language. In the coding phase, tasks are divided into units or modules and assigned to the various developers. It is the longest phase of the Software Development Life Cycle process.

In this phase, developer needs to follow the coding guidelines, defined for code compilation and future maintanance. Coding a functional block is called programming. The functional code blocks are coded using high level programming language,compiled and interpreted. They use software tools such as compilers, interpreters and debuggers.

8.3.2 Phase 5: Testing

Once the software is developed, it is verified for its intended functionality in a testing environment. The testing team starts testing the functionality of the entire system. This is done to verify that the entire application works according to the customer's requirement. During this phase, QA and testing team may find some bugs/defects, which they communicate to developers. The development team fixes the bug and sends it back to QA for a re-test. This process continues until the software is bug-free, stable, and working according to the business needs of that system.

8.3.3 Phase 6: Installation/Deployment

Once the software testing phase is over and no bugs or errors are found in the system then, the final deployment process begins. Based on the feedback collected by reviews and testing, the final software is released and checked for deployment issues if any.

8.3.4 Phase 7: Maintenance

Once the system is deployed and customers start using the developed system, the following 3 activities occur:

- Bug fixing – bugs are reported because of some scenarios which are not tested at all.
- Upgrade – Upgrading the application to the newer versions of the Software.
- Enhancement – Adding some new features into the existing software.

The main focus of this SDLC phase is to ensure that needs continue to be met and that the system continues to perform as per the specification mentioned in the first phase.

8.4 Software Architecture Styles

Software architecture has become an integral part of software engineering and design. Software architecture is a definition of properties of a set of software component structures needed in a system and their relationship. Software components are the design building blocks. Defining software architecture involves

understanding and organizing all the system components and their interactions, the environment in which they operate along with the core principles used for their design. It is a conceptual model of the system whereas the design is a detailed implementation which makes the concept work. Software design is all about how the software architecture can be implemented into the overall system.

Software design involves the following steps:

- Coding also called programming.
- Using specific design patterns.
- Choosing classes, interfaces, functions, and responsibilities.
- Working with specific components and modules.

Software architecture represents the main purpose and design decisions considered during the implementations of the system. Some key functions of software architecture include the following:

- It helps stakeholders to visualize, understand, and analyze a system's properties.
- It focuses on the structure rather than implementation.
- It can be used to anticipate difficult design decisions (e.g., buying the IP cores) in a system development project before they are implemented.
- It is used as the basis for making core decisions to ensure high-quality system implementation.

Software architecture documents the following sections:

- System functional requirements: This defines the system functions.
- Nonfunctional performance requirements of the system such as efficiency, speed, reliability, usability, maintainability, and scalability.
- Constraints such as standard compliances needed to be followed, cost, Time to market, and skillset required.

Different architecture styles of software are the following:

8.5 Layered Architecture

The database-driven architecture is the most common architectural style in software as it is one of the best ways to store information effectively in tables. In a layered approach, the data begins at the top layer and works its way down until it reaches the bottom, which is typically a database. As it descends, each layer has a different task or function which focuses the data and solves issues such as reformatting the values. This architecture style is adopted in some of the biggest software frameworks like Express, Java EE, and Drupal. Figure 8.2 shows the layered software architecture.

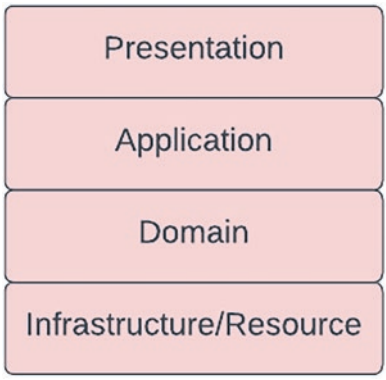


Fig. 8.2 Layered software architecture

8.6 Event-Driven Architecture

This approach provides high system performance and is agile. In these systems, the programs spend most of their time waiting for an event to occur (e.g., a user intervention while using a computer). It is built around the concept that only sometimes data is to be processed, and other times there is no need for data processing. This is managed through a central processing unit that processes all the data accessed before transmitting the data to the relevant modules.

Where a layered architecture will pass data through all levels, event-driven architectures will only send the data to the appropriate design components; hence, they are considered highly efficient. This architecture can adapt to complex environments. It can be scaled up/down easily and extended if new event types are used. However, testing can only be done on individual modules if they are independent; otherwise, they will have to be tested in a fully functional system. Error handling can be challenging to structure if several modules are handling the same events. Figure 8.3 shows the event-driven architecture.

8.7 IOT Software Development for Environment Monitoring

Let us continue the discussion on an architectural case study of IOT for environment parameter monitoring which is made up of SOC with embedded software. The IOT-SOC software development is carried out in the *software development environment (SDE)* of the targeted hardware development board. SDE is also called *Integrated development environment (IDE)*, which is used for embedded system software development. The standard development platforms support multiple programming languages like C, C++ Embedded C, system C, etc. It will have built-in compiler, simulator, Flash programmer, Debugger running on the personal computer which can be connected to the development board through a programming cable dedicated to the

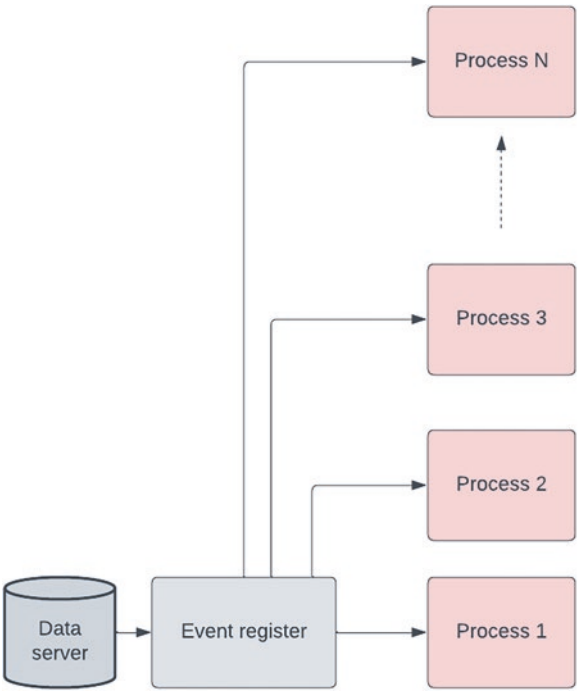


Fig. 8.3 Event-driven software architecture

board. Some software development platforms run on cloud servers which can be accessed on the internet. One can develop the entire IOT-SOC software on the cloud server and generate executable bit code, which can be downloaded and programmed onto the board through the local computer system through the programming cable. Embedded software development needs the following software modules:

- Operating systems (Windows CE, Yocto Linux, ThreadX, Nucleus RTOS).
- Languages (C, C++, Python, JavaScript, etc.)
- Tools (IDE, PDK, SDK, compiler tool chains, hardware and software debuggers (e.g., ST-Link, Segger)).

Figure 8.4 shows the embedded development environment.

8.7.1 Communication Technologies

The communication interface core, which is connected to IOT SOC through either serial interface such as UART or USB cores, usually provides internet access. This is enabled through ethernet cable and supporting functionality or through machine-to-machine (M2M) communication technologies to the nearby smart devices which support internet access. The M2M communication is achieved by either wired cable or wireless technologies like Bluetooth, Wireless LAN or Zigbee or Lora WAN or 6LowPAN.

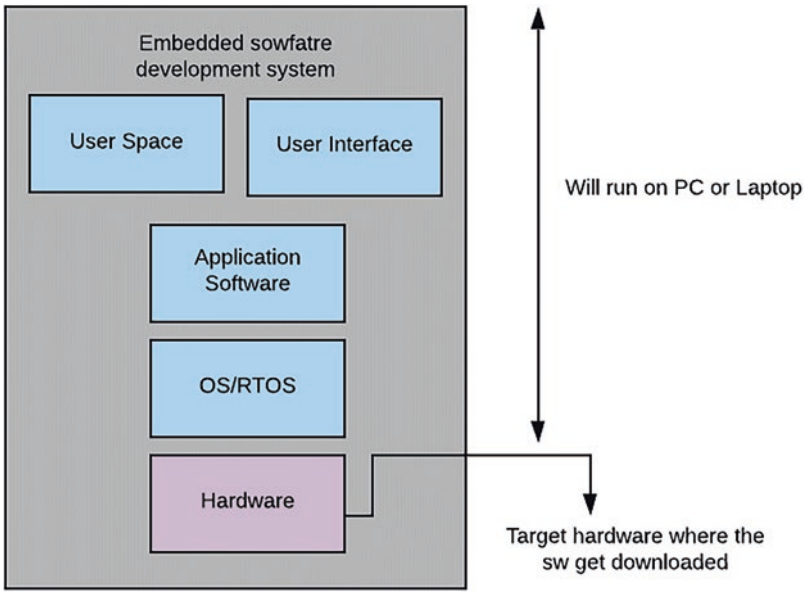


Fig. 8.4 Embedded development environment

8.7.2 OSI Model in Software Architecture

The OSI Model (Open Systems Interconnection Model) is a conceptual framework used to describe the functions of a networking system. The OSI Model characterizes computing functions into a universal set of rules and requirements in order to support interoperability between different products and software. In the OSI reference model, the communications between a computing system are split into seven different abstraction layers: Physical, Data Link, Network, Transport, Session, Presentation, and Application. The functionality of any layer is partitioned between hardware and software as discussed previously depending on the timing critically of the function. In most cases, the physical layer of most of the functions is implemented in hardware but for configurations and initializations of hardware it is implemented in software. In the data link layer apart from initialization, communication link establishment and maintenance and some part of packet processing/data processing functions are implemented in software. Most of the network, transport, session, and presentation layers are implemented in Software. A typical IOT architecture is shown in Fig. 8.5.

8.8 Application Software for IOT SOC

Software layer which closely interacts with hardware SOC is often referred to as *firmware*. Firmware is stored in on-chip embedded flash memory or external memory interfaced to SOC.

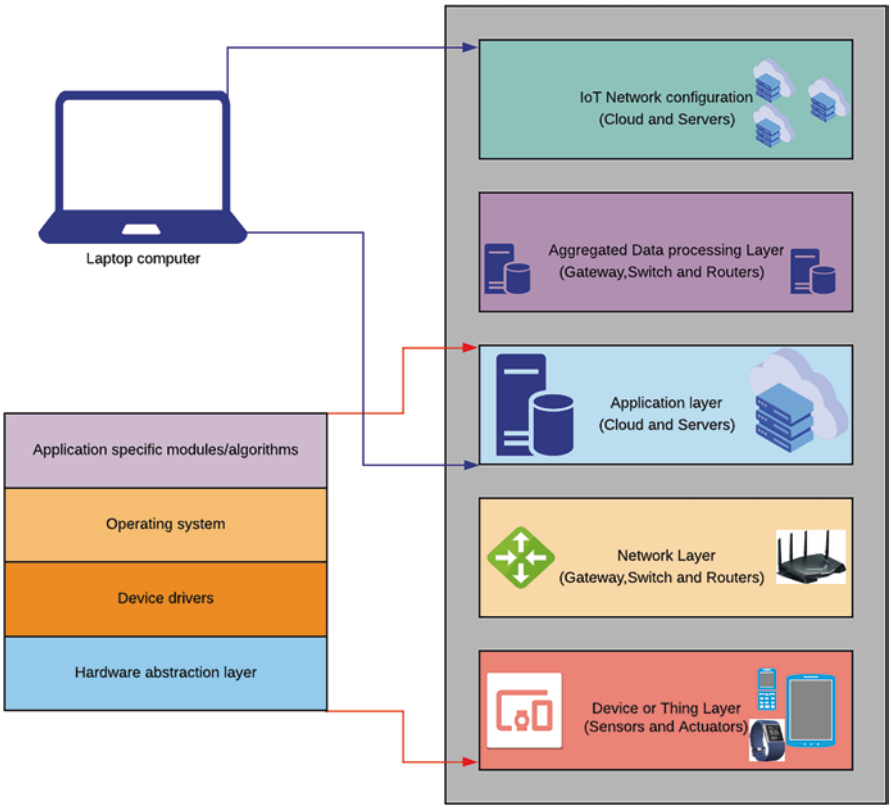


Fig. 8.5 IOT architecture as layered architecture

```
include <SoftwareSerial.h>. //include serial interface shield
#include "DHT.h" //include DHT.h file
#define dht11Pin 2 // Digital pin connected to the DHT sensor
#define DHTTYPE DHT11 // DHT 11
DHT dht (dht11Pin, DHTTYPE) //Create DHT object
SoftwareSerial esp (10, 11); // RX, TX
```

The embedded application-specific firmware for capturing the temperature and humidity in our case study is given in Fig. 8.6:

The firmware program starts with including necessary libraries using *#include* command, followed by connectivity information (which interface pin is connected to which pin of the external module) and then creating the object corresponding to the connected external module as done in the first part of the code shown.

The second part of the program is the *setup* where the *initialization* of the module is done. For example, setting up serial data rate configuration, driving default logic on the outputs, etc., shown in code clipping below:

```
include <SoftwareSerial.h>.                    //include serial interface
   shield
#include "DHT.h"                                //include DHT.h file

#define dht11Pin          2                    // Digital pin connected to the
   DHT sensor
#define DHTTYPE       DHT11                    // DHT 11

DHT dht (dht11Pin, DHTTYPE);              //Create DHT object
SoftwareSerial esp (10, 11);                    // RX, TX

void setup ()
{
 String network =   "wifi_network";          // write the name of our network
 String password = "our_password";                  // write the password of
   our network here.
 dht.begin();
 Serial.begin(115200);                     // We are starting our communication
   with the serial port.
 Serial.println ("Started");
 esp.begin(115200);                                 // We are starting serial
   communication with ESP8266.
 esp.println("AT");                          // We do the module control with
   the AT command.

 while (!esp.find("OK"))
 {
  Serial.println("Resend");
  esp.println("AT");
 }
 Serial.println("OK Command Received");
 esp.println("AT+CWMODE=1");                        // We set the ESP8266
   module as a client.
 while (!esp.find("OK"))                    // wait until we recieve response
   from Wifi module
 {
  Serial.println("Setting is ....");
  esp.println("AT+CWMODE=1");
 }
 Serial.println("Set as client");

 Serial.println("Connecting to the Network ...");
 esp.println("AT+CWJAP=\"" + network + "\",\"" + password + "\""); // We are
   connecting to our network.
```

Fig. 8.6 IOT Firmware program for monitoring environment (temperature-humidity) parameters

```
 while (!esp.find("OK"));                                    // We wait until it
   is connected to the network.
 Serial.println("connected to the network.");
 delay(1000);
}
void loop()
{
 String ip = "184.106.153.149";                               //Thingspeak ip
   adresss
 float temperature, humidity;

 delay(2000);
 esp.println("AT+CIPSTART=\"TCP\",\"" + ip + "\",80");       // We connect to
   Thingspeak.
 if (esp.find("Error"))
 { // We check the connection error.
    Serial.println("AT+CIPSTART Error");
 }

 humidity = (float)dht.readHumidity();
 temperature = (float)dht.readTemperature();

 // Serial.print(F(" Humidity: "));
 // Serial.print(humidity);
 // Serial.print(F("%  Temperature: "));
 //Serial.print(temperature);
 //Serial.print(F("C "));


 String veri = "GET
   https://api.thingspeak.com/update?api_key=CRF9H8AJ0DC5UFL7";
                                          // Thingspeak command. We write
   our API key
                                          // in the keypart.
 veri += "&field1=";
 veri += String(temperature);              // The temperature variable we will
   send
 veri += "&field2=";
 veri += String(humidity);                // The moisture variable we will send
 veri += "\r\n\r\n";

 esp.print("AT+CIPSEND=");                 // We give the length of data that we
   will send to ESP.
 esp.println(veri.length() + 2);
 delay (2000);
```

Fig. 8.6 (continued)

```
 if (esp.find(">"))
 {                                                  // The commands in it are running
   when ESP 8266EX is ready..
    esp.print(veri);                               // We send the data.
    Serial.println("Data sent.");
    Delay (1000);
 }
 Serial.println("Connection Closed.");
 esp.println("AT+CIPCLOSE");                       // We close the link
 delay (1000);                                     // We wait 1 minute for sending new data.
}
```

Fig. 8.6 (continued)

```
  void setup ()
  {
  String network = "smile_to_connect"; // We write the name of
our network here.
  String password = "our_passward"; // We write the password of
our network here.
  Serial.begin (115200); // We are starting our communication
with the serial port.
  }
```

The third part is the "loop" function which is executed forever.

```
  void loop()
  {
  float temperature, humidity.
  humidity = (float) dht.readHumidity();
  temperature = (float) dht.readTemperature();
  }
```

The data transmitted from the processor is displayed on the terminal connected through COM port to the on-chip board UART module. The data display on the COM port terminal is shown in Fig. 8.7.

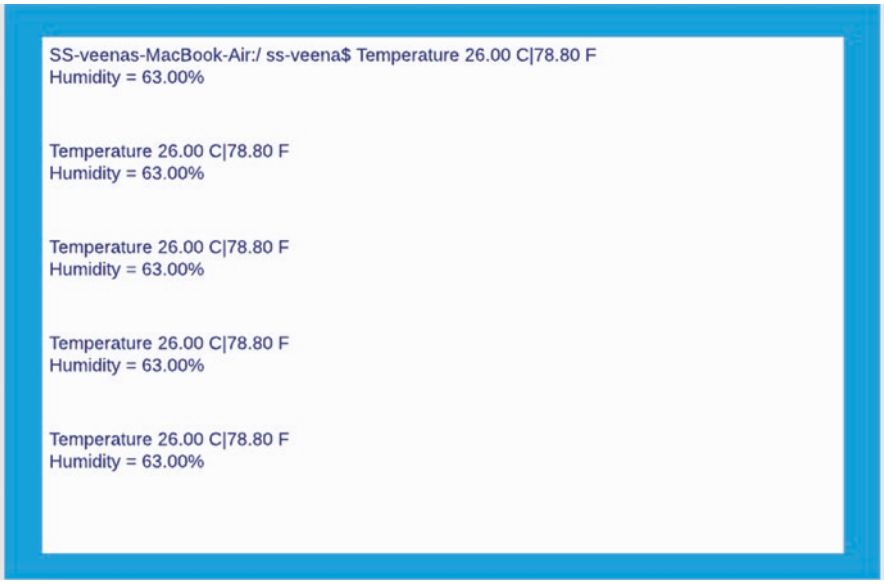


Fig. 8.7 Environmental data captured by the IOT-based system displayed on the COM port terminal

8.9 Prototype Design of IOT SOC-Based Solution

This is an optional step, but however in current aggressive time-to-market situations, prototyping a product is an important step of product design. Product prototype is product design in small numbers. The difference between the prototype and the product is very small only in the ways the molds are made to make a number of products to be made when it comes to mechanical design or product packaging. Therefore, it can be assumed to be almost a product design. Product design is the most important aspect of IOT devices, seen as the physical makeup of the hardware. IOT devices have to be safe, convenient to mount, install and aesthetically appealing in most applications. Key factors considered for the product design are best user experience, ease of use, ease of adoption to everyday life, consistent user experience in traversing digitally from system to cloud applications, and personalized user experience. Product design should use materials whose properties and behavior when exposed to environmental conditions are well known. Features such as waterproofing and biocompatibility depend on the material properties.

Product design and electronic design should go hand in hand. The product design flow is shown in Fig. 8.8.

Product design is carried out using CAD software tools, which have the following features:

- Assess manufacturability.
- Create precise geometry of parts and model.

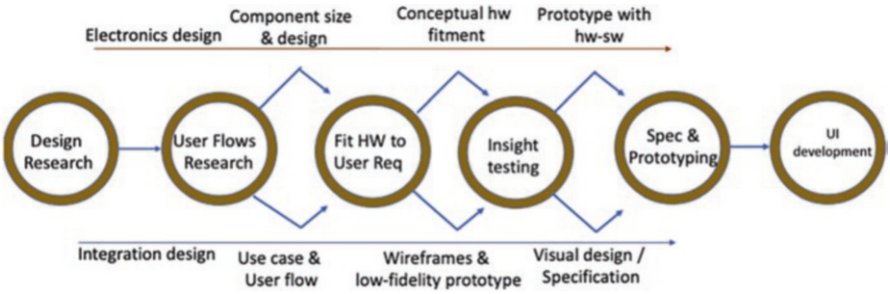


Fig. 8.8 Product design flow

- Support assembly modeling.
- Generate 2D and 3D graphics of parts as per international standards ASME, ISO, and JIS.
- Automatically generate drawings of parts with assembly notes.
- Automatically generate a bill of materials with assembly notes.
- Software tool performance: a fast load of complex parts with lightweight representation.
- Analysis module for thickness.
- Mold filling simulation module.
- Auto creation of holes, springs, and fasteners.
- Auto optimization capability.
- Create parts by reverse engineering of existing product capture.
- Top-down design or tear-down design viewing and drawings.

Many product design tools with most of the features stated above are available. Creo, Inventor, Fusion 360, Illustrator, and Onshape are some of the popular product and machine design tools.

8.9.1 Cloud Storage and Analytics Setup

Most of the cloud platforms provide free access to the cloud storage and analytics with simple registration and enable the solution provider to access different services like accessing other related data hosted by others at a price. For example, if one develops a smart environment monitoring system to monitor parameters, a cloud platform provides storage space, data analytics, visualization services, and access to geographic data like coordinates, and terrains, which uses the same cloud platform at a price. These services are provided in *"Software as a service (SaaS)"* model in a way that makes environmental information more meaningful.

8.10 Registration to Cloud Server

IOT case study uses ThingSpeak cloud platform from Mathworks for remote access and storage. Access to servers such as ThingSpeak cloud services is by registration using user details and with a valid email address. Cloud service provider like ThingSpeak, Amazon, IBM, and CISCO sends the verification link to check the validity of the email, which must be used to login into the cloud for confirmation.

After registration to cloud service, to collect, store, analyze, and display the data, one has to generate a channel. ThingSpeak stores device data in channels as messages. User data from IOT devices are collected, analyzed, and controlled using these channels in ThingSpeak cloud service. Normally, one connected device will require one channel on ThingSpeak. Any other device on the internet which support these protocols can access the stored environment information via the channel.

8.11 Cloud Server Access

Access to cloud-based ThingSpeak channel is through application-specific interfaces (API)s. APIs access web servers through internet and can use HTTP, REST APIs, and MQTT protocols. These protocols are discussed in detail in M2M communication part of the book. To read and write to a ThingSpeak channel, the application sends requests to the ThingSpeak server by issuing HTTP commands, and publishing MQTT messages. Each ThingSpeak channel can have up to eight fields of 255 characters of data in either numeric or alphanumeric format. A channel also has location information and a status update field. Each channel data entry is stored as a message of 8 fields of data, with a date and timestamp. One can retrieve stored data by time or by entry ID.

The IOT device data gets updated to the user channel by the following highlighted code in the device firmware. This is RestAPI for updating the channel with new measured data. The highlighted text in the statement is the API key for writing into the channel.

```
{
    GET https://api.thingspeak.com/update?api_key=CJMNXT0RFLNKH7J1&field1=&fi
eld1&field2=&field2
    }
```

Data retrieval to IOT device is done using code embedded in the firmware below:

```
{
    PUT https://api.thingspeak.com/update?api_key=CJMNXT0RFLNKH7J1&field1=&fi
eld1&field2=&field2
    }
```

Data set can be viewed by any other authorized client anytime and from anywhere by logging into the ThingSpeak cloud with proper credentials. The snapshot of the channel data is shown in Fig. 8.9.

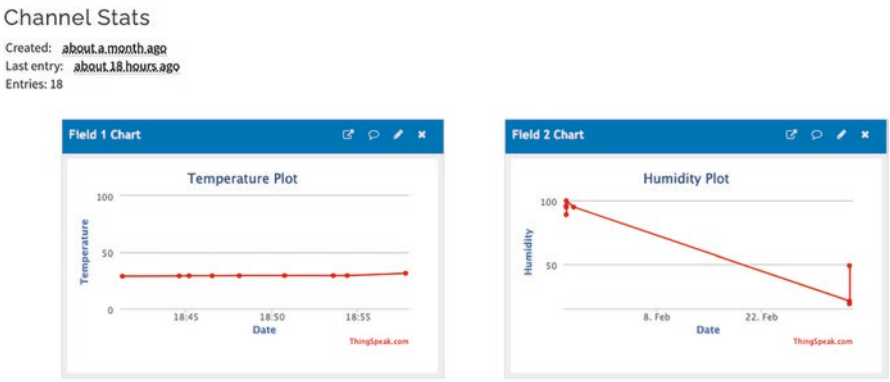


Fig. 8.9 Snapshot of the environmental data: temperature and humidity on the cloud server. (Courtesy: Mathworks Thing speak output)

Table 8.1 Environmental data set as updated periodically in the ThingSpeak cloud server exported in CSV format

Feed			
Created at	entry Id	Field 1 temperature	Humidity
13/05/23 12:18	1	25	
13/05/23 12:20	2	27	
13/05/23 12:21	3	23	
13/05/23 12:22	4	23	89
13/05/23 12:22	5	22	95
13/05/23 12:23	6	26	86
13/05/23 12:24	7	24	100
13/05/23 12:25	8	23.2	86
13/05/23 12:26	9	23.8	95
13/05/23 12:28	10	23.2	21
13/05/23 12:30	11	23.3	20
13/05/23 12:31	12	22.4	20
13/05/23 12:32	13	23.7	20
13/05/23 12:35	14	24.8	19
13/05/23 12:36	15	22.9	19
13/05/23 12:38	16	23	19
13/05/23 12:39	17	23.1	20
13/05/23 12:40	18	23	28

The cloud service providers offer many services to export the data set in many formats like the tabular form in CSV format, JSON, and XML formats for further viewing and processing from anywhere.

Table 8.1 shows the environment temperature and humidity data set in csv format which is written out using the cloud service provided for the registered user who also creates a channel in which the data is stored and displayed.

8.12 Product Design

Once the prototype is acceptable by engineering, marketing, and manufacturing, finalizing the material supply chain, the product is ready for manufacturing in large numbers. It means that the product is ready to be launched in the market. This stage is considered engineering success. From here onwards, it will be production management and marketing activity. The marketing and sales team will get orders and the production team will work and execute the orders. Market success depends on product quality, customer feedback, marketing campaigns, and promotions.

8.12.1 Product Validation and Testing

Prototype and product validation and Testing procedures are identical except that in large product companies, the large-scale product testing is carried out by production teams whereas in small companies it is done by engineering teams. IOT devices and systems are very complex involving many technologies. So, it is essential to test the system at all levels. The complete IOT ecosystem solution for ambient environment monitoring of temperature and humidity in the case study discussed here can be represented in Fig. 8.10.

Most of the validation during development happens at the hardware level, Firmware level for the device and when integrated with the applications on the internet, Integration testing. Once the cloud access is established it is necessary to test the interaction for read and write access of the stored data. Once the system passes the preliminary test at this level, it is necessary to look at overall test framework. The IOT system tests at the product level are classified as follows:

- Functionality test.
- Compatibility test.
- Stress and scalability test.

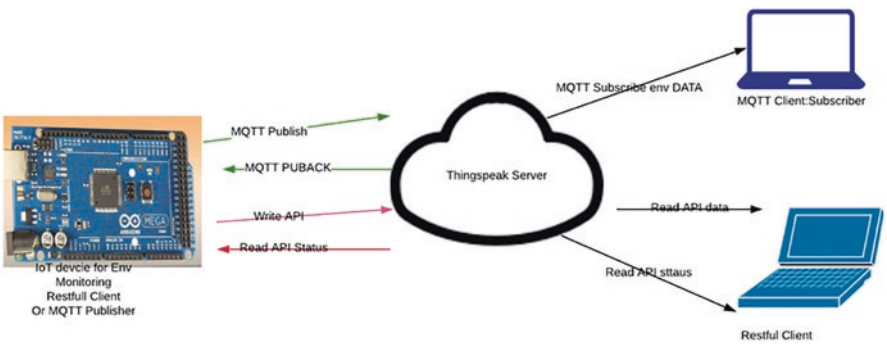


Fig. 8.10 Environment monitoring system for monitoring ambient temperature and humidity

- Data integrity test.
- Security test.
- Performance test.

All the tests mentioned above are explained under the context of the reference design of environmental monitoring device.

8.12.1.1 Functionality Test

The IOT device in reference design is tested for the supported functionalities implemented.

Temperature Measurement: The device should be able to measure the temperature to the accuracy claimed under different conditions. This test requires a golden reference temperature standard.

Initially, measurements are taken for major use-case conditions and calibrated against the standard. Once the device is calibrated, the trend in deviation is observed closely to see whether the device is drifting in any particular direction or on both sides with respect to the standard. These data from the device under test and standard are recorded for a large number of cases, and the data is statistically analyzed by calculating mean difference and standard deviation. Formal analysis is carried out by plotting the Bland Altman chart to check the need for possible skewness in the specified range of temperature. Skewness may be fixed by selective calibration. Now the device is ready for an integrated test. Similarly, functional test and analysis are carried out for humidity also.

8.12.1.2 Compatibility Test

IOT devices get connected to the cloud through a protocol and to other devices through IOT cloud platform. As a result, checking the compatibility of IOT systems with the cloud is important. Major tests include data writing to the cloud, data retrieval from the cloud and connection to mobile smartphone application (if claimed). Data integrity could be affected due to a trivial fact like the wrong interpretation of little or big endianness. That should be tested and confirmed. Also, when the cloud-based web server data is retrieved onto smart mobile applications, data visualization could be a problem as the screen size gets shrunk to a smart mobile display from a large web display. This has to be tested for data visualization integrity.

8.12.1.3 Stress and Scalability Test

Reliability and scalability are important for building an IOT test environment which involves simulation of sensors by utilizing virtualization tools and technologies.

8.12.1.4 Data Integrity Test

It is important to check the Data integrity in the IOT device as it handles a large amount of data. Since an IOT system includes a third-party service like the cloud, it is essential to get error-free information and pass it on to the end user who is trying to access the data.

8.12.1.5 Security Test

In the IOT environment, many users are accessing a massive amount of data from the same cloud. Thus, it is important to validate user authentication and data privacy controls as part of security testing.

8.12.1.6 Performance Test

Performance tests include speed tests, back-to-back data tests, and memory round-off tests, which in turn affect the data integrity also. The entire system must be tested for these scenarios at all interfaces. Performance testing is also important to create a strategic approach for developing and implementing an IOT testing plan.

Table 8.2 shows the relevance of the tests for different components of an IOT system.

8.12.1.7 Safety and Environment Tests

Since the IOT devices are used as consumer and industry applications, they must comply with certain safety and environment safety standards defined by the regulatory authorities, for example, RF radiation tests, leakage current tests, and battery tests. There are exclusive test houses with certification authorization which have test facilities such as special chambers in which the device under test (DUT) is kept and powered externally. The test scenario is created, and the response is observed. TUV Rhineland is one such global test house. There are many Governments owned national accreditation boards for testing and calibration. These test houses in

Table 8.2 Relevance of test types at different components of IOT system

IOT elements testing type	Sensor	Application	Network	ThingSpeak cloud
Functional Testing	True	True	False	False
Usability Testing	True	True	False	False
Security Testing	True	True	True	True
Performance Testing	False	True	True	True
Compatibility Testing	True	True	False	False
Services Testing	False	True	True	True
Operational Testing	True	True	False	False

India offer test facilities and certify the devices for safety and environmental conditions as per the standards in each application domain.

8.12.1.8 IOT Testing on the Internet

Similar to general search engines like Google, Bing etc., there are search engines for checking the servers, devices and IOTs connected on the internet. Some of the examples are www.shodan.io, thingful www.thingful.net. These device search engines provide information about the number of devices connected on the internet, along with their geographies. This information is used for providing customized security to the device users. Once such an application is a device search engine, thingful provides connectivity of the device with the secured interoperability among devices on the internet. This IOT testing tool used to control the data use empowers the device owners to take informed decisions on the data transfers.

8.12.1.9 IOT System for Smart Environment Monitoring Testing

The test conditions relevant for the reference design are listed in Table 8.3.

Table 8.3 Test conditions relevant for IOT for smart environment monitoring system reference design

Test categories	Sample test conditions
Components validation	Device hardware sensor for temperature reading and humidity reading Communicating the read data from sensor onto ThingSpeak cloud through WiFi module Embedded software for cloud access and monitoring the temperature and humidity data Cloud infrastructure for registration, channel creation, and data visualization Network connectivity through WiFi module Third-party software like web application through APIs Sensor testing for sensed data for accuracy and reliability Command testing for MQTT or rest protocol Data format testing for data integrity during communication from IOT device to ThingSpeak cloud and from cloud to any other device. Robustness testing when there is sudden variation in temperature and humidity during rainfall, etc. Safety testing as per the standard requirement.
Function validation	Basic device testing for proper module functions and interconnections Testing between IOT devices (end-to-end tests) Error handling (negative tests, for example, when the temperature is out of range or cloud connection is not established). Negative testing can be even when the IOT device detects temperature changes faster than cloud access. There is a possibility of data loss. Valid calculation for data visualization and data endianness.

(continued)

Table 8.3 (continued)

Test categories	Sample test conditions
Conditioning validation	Manual conditioning different modes like temperature testing and humidity testing modes Automated conditioning when data is written at the frequency cloud data is updated Conditioning profiles data visualization
Performance validation	Data transmit frequency: Continuous data read/writes from device to cloud Multiple request handling stressing the protocol tests Synchronization by having a temporary data buffer if the device is faster (which is usually the case) than the cloud update. Interrupt testing when any critical event of cloud server breakdown or device fault is detected. Device performance Consistency validation
Security and data validation	Validate data packets for registered and unregistered devices Verify data loses or corrupt packets due to frequency mismatches Data encryption/decryption access denials for unauthorized devices Data values for integrity at all stages Users roles and responsibility & its usage pattern User profiling for optimization
Gateway validation	Cloud interface testing for WiFi communication Device to cloud protocol testing WiFi communication Latency testing for performance evaluation for any data loss
Analytics validation	Sensor data analytics checking data visualization IOT system operational analytics System filter analytics selective update of the parameters Rules verification
Communication validation	Interoperability data retrieval from any third-party device on internet M2M or device to device end to end testing Protocol for WiFi and MQTT or HTTP or REst

8.13 Future Scope of IOT SOC Solution

The IOT device of the smart environment monitoring system is realized using the IOT SOC defined in the last chapter. This IOT SOC is Arduino equivalent, WiFi module and sensor interface modules are integrated into a single chip.

As can be seen, from the defined SoC architecture, the application processor, on-chip memories, Wishbone interconnect, and interface cores are integrated on a single chip. It just requires sensors to be connected to the serial interface, a user interface through GPIOs and the battery must be connected to realize a miniaturized IOT device.

There are also SoCs that support multiple wireless protocols like WiFi, Bluetooth, and Zigbee in a single chip and depending on the application, the suitable wireless block can be enabled. A few SoCs also support encryption functionality, which is essential for IOT devices.

Chapter 9
SOC Advanced Architectures



9.1 SOC Advanced Architectures

As the complexity of System on Chip increases, achieving the design goals like Power, Speed performance and Area (PPA) became extremely difficult. It requires architects to consider special advanced techniques while defining architecture. Overdesigning is the term used to define SOC architectures with techniques using special cells in the design or unnecessarily high system clock frequency to meet PPA goals. This could otherwise be achieved using some advanced architectural tweaks or variations. In this chapter, we discuss some of the advanced techniques which can be used to define SOC architectures.

9.2 SOC Accelerators

SOCs which are heavy in signal processing algorithms implemented in hardware require some high speed signal processing functionalities. For such designs, specialized-hardware accelerators are designed in isolation, optimized for the algorithmic performance, with limited consideration of the implications of their integration into a given SOC. However, the interface between these accelerators, the memory hierarchy, and SOC logic are critically important for their individual performance in SOC and the performance of the overall SOC. Today's SOC combines many general-purpose processor cores with a growing number of accelerators, each offering a high-performance specialized hardware implementation of an algorithm or a small class of algorithms. These accelerators are loosely coupled because they are located outside the cores and execute coarse-grain tasks on large datasets independently from them. Accelerators are not complete functionality cores as IP cores but implement a part of functions of an algorithm and

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

V. S. Chakravarthi, S. R. Koteshwar, *System on Chip (SOC) Architecture*,
https://doi.org/10.1007/978-3-031-36242-2_9

require other SOC logic to complete the algorithm functions. Accelerators are also evaluated in isolation as they are also reusable across SOCs. All the latest SOC product families of smartphone SOCs such as Snapdragon from Qualcomm, Apple A series, and Tegra from Nvidea [1] from major SOC vendors use one or more accelerators in their architecture.

Overall SOC performance is determined by the integration logic used in architecture to interface these accelerators with the rest of the data path in SOC and runtime interaction during the SOC operation. This is because accelerators are designed assuming ideal conditions of bandwidth availability of shared resources such as memory and other logic, but the resources are bandwidth limited and often have contentions and interconnect congestions. A major factor in most SOCs with accelerators in influencing the performance is memory hierarchy.

9.3 Scalable Platform Architectures

With the various types of different IP cores getting integrated into SOC, obviously, the resulting heterogeneous SOC architecture will not be of IP cores of regular structures of the same sizes. The integration of these IP cores into the SOC is done by adding different types of glue integration logic in the SOC. This increases SOC design complexity in terms of integration, hardware software interface, and managing shared resources. **Scalable platforms architecture** is a new approach to SOC design and programming. This also provides flexibility in terms of tile-based socketed architecture. Each tile of a platform instance can host a processor, I/O peripherals, system utilities, or accelerators, which are often configurable to the functional requirements. The selection of the tile or mix of these tiles for a target application, is the result of a **design-space exploration**. This can result in dual-core, quad-core, or multiple cores offering scalability in the system designs. Figure 9.1 shows a 16-tile architecture.

9.4 Multiple Clock SOC Architecture

Low-power systems are mandated for any application solution. This is achieved by using multiple clock SOC architectures where different blocks are operated at different clocks of right frequency of operation. The SOC architectures enable the reuse and integration of IP cores, and each is defined to operate at a different clock frequency. Low power consumption is achieved by gating the individual clocks so that the selected IP Core can be shut down when it is not operational in the functional mode of the SOC. Such requirements also arise simply because each block is designed independently and is constrained by different standards (e.g., for external buses, or audio or video I/O), each with its own clocking regime.

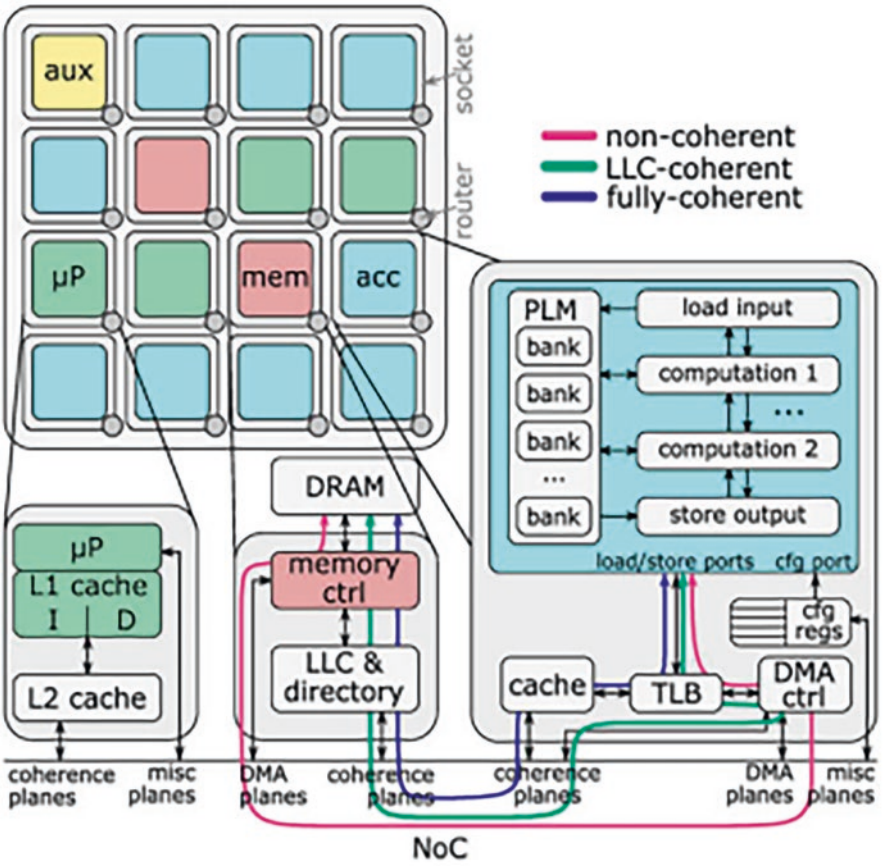


Fig. 9.1 Scalable platform architecture

In such architectures, the signals from one IP Core operating at one clock frequency cross the boundary of its clock domain while interacting with different IP Core operating at different clock frequency. In such cases, normal design conventions of digital logic break down if not taken care at the design level. Special precautions must be taken to ensure that the signals cross clock domains correctly, and care must be taken that there are no accidental crossings which neglect such precautions. It is more likely to get unnoticed while modeling the SOC designs at higher abstraction levels. A good hardware description language (HDL), particularly one which claims to operate at a high level of abstraction, should have features which make it natural for the designer to construct circuits which correctly observe these constraints. The synthesis tool with correct configuration options must be used to generate hardware from such HDL. There are Lint tools available today to check these conditions in design modeling.

Multiple clock system architecture is also helping to interface with the external world components, which can be totally asynchronous in nature. Care should be taken in such scenarios where interface IPs are integrated into the SOC.

9.5 Multiple Voltage Architecture

Power supply has a direct relationship with the dynamic power consumption of the System on Chip, which consists of switching & short-circuit power. Therefore, reducing power naturally enhances power performance. The decreased threshold voltage in the latest process technologies causes an increase in gate delay. Lowering the voltage of SOC blocks is perhaps the first design implementation that is used to meet power performance goals. Figure 9.2 shows the system architecture with different voltage levels. Such architectures are also referred to as multi-VDD designs. In this, the design is partitioned to operate at different power domains and individual power distribution methods are used in the design methodology of individual partitions in the SOC architecture. Care to be taken when the signals cross over different power domains. Such architectures use special power management cells such as isolation cells, level shifters, always on cells and retention cells. These help to manage interactions across blocks of different power domains. The logic is partitioned into different domains called power domains.

The power intent derived using the IEEE standard 1801–2018 Unified Power Format UPF 3.1 is used by many companies to define the power parameters of a chip. The power architect makes use of this technology to create files that describe the power and power control intent of an electric design. Supply sets, power switches, level shifters, and memory retention techniques are all included in the annotation. Power states, transitions, a collection of sim states, the pg (power/ground pin) type and function properties of nets, and the update parameter to assist the gradual refinement of the power intent are all definable descriptions of the potential power applied to the electronic system.

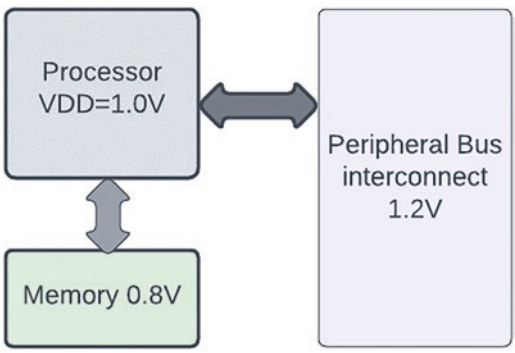


Fig. 9.2 Multi-voltage SOC architecture

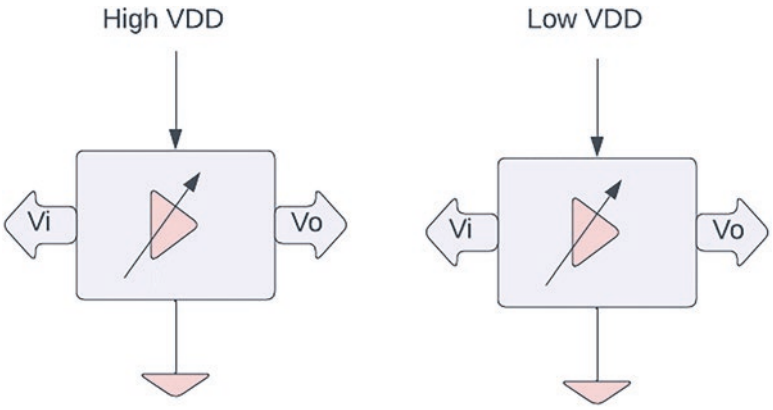


Fig. 9.3 Level shifter

9.6 Requirements to Create a Multi-Voltage Design

9.6.1 Level Shifters

As shown in Fig. 9.3, level shifters will shift the level of voltages to ensure different modules operating at different voltages will operate correctly when LS (Level shifter) circuits are attached. The circuits are implemented in HDL, and they can also be made to implement the drive strength. The figure shows a low-to-high-voltage level shifter (A) and a high-to-low voltage level shifter (B). Vi and Vo are the sources and destinations of different voltage levels in different modules.

9.6.2 Power Gating

The method in Fig. 9.4 is referred to as "disconnecting the power of the gates which are not in use." The figure shows an implementation of such a situation. Power gating is used to reduce leakage power. This step is performed at an architecture level while computing the performance factors of the low power module, or module at a sleep state while other priority modules are ON, or module where power is to be disconnected by software, or while shutting down the power. Power gating is significantly used with modern-day traditional terms like SLEEP/WAKE events of the device. The wakeup and sleep sequence follow certain architectural decisions to enable or disable a sequence of operations that controls the power logic of the chip.

Complex architectures also define power management blocks to manage the data flow to achieve the best performance objectives.

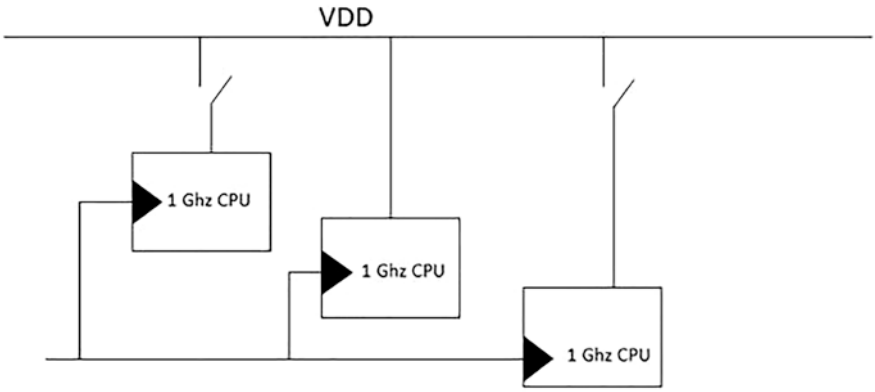


Fig. 9.4 Power gate cell

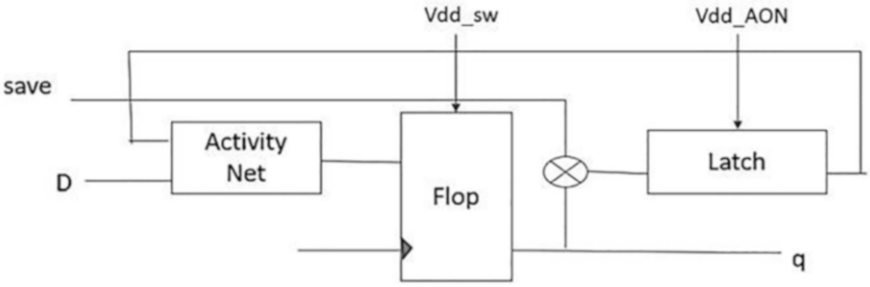


Fig. 9.5 Retention cell

Special care must be taken while implementing power gating as output signals from a power-gated block pose special challenges. This considers isolation and retention strategies at the micro-architecture level while performing the wake-up or sleep sequence. The placement of circuits of retention and isolation strategy in the circuits should not affect the power performance factors.

Retention cells are used to save the state of the chip to use during the wakeup sequence of a module. Fig. 9.5 shows a state being saved upon the assertion of the saving sequence. Vdd_sw (switched supply voltage) is controlled by a switch, and Vdd is always on voltages to power up the circuit. When save (saving sequence) is asserted, the output of the module is latched and is available as feedback.

Figure 9.6 illustrates where isolation cells are introduced when a shutdown or a sleep phase is isolated from the receiving end. Isolation cells keep them turned off and block output to a predefined value. This way, isolation cells are attached to reduce crowbar currents, thereby reducing power leakage.

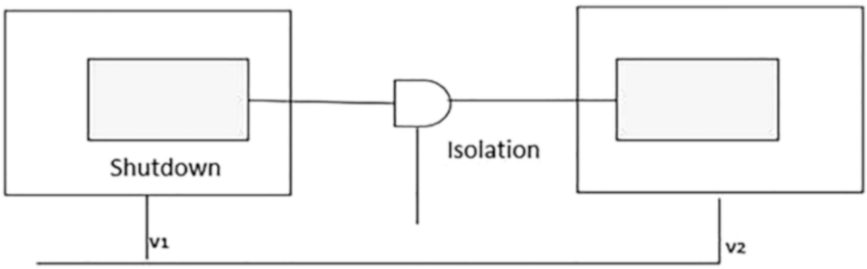


Fig. 9.6 Isolation cell

9.6.3 Clock Gating

This method is referred to as turning off the clock transitions when the circuit does not encounter switching of internal signals when there is no activity to be performed. This controls the frequency of transitions from the CMOS dynamic power equation and hence minimises power consumption in the design elements. Almost all EDA tools identify and support this by inserting special cells called clock gate cells in the design. The complexity of SOCs has increased many folds thus introducing new demands for power management. The supply of the various SOC power domains must be flexible enough to be controlled by developers to control power dissipation and improve battery autonomy. Careful power analysis and knowledge of the special techniques like this and ways to adopt them using EDA tool flow is needed to come with best solution. Power-related crises must be prevented by analyzing power demand as early as possible in the design flow. Achieving power goal for the SOC design is simplified by early analysis and adaptation by the appropriate EDA tool flow. This reduces power consumption of the design to the greate extent. Figure 9.7 shows the clock gate integrated cell (CGIC).

9.7 Near Memory Processing Architectures

Majority of SOC contains approximately 60% or more of an on-chip embedded memory. The data-driven applications like data centers, radio astronomy, health sciences, material science, and chemistry demand large processing of data in the order of terabytes of data, and any amount of on-chip memory can support it. Memory technology has not scaled up as processor technology over the past couple of decades, creating added gaps in meeting the demands. Slowing of **Moore's law** (number of transistors doubling in 2 years), the demise of **Dennard's scaling** (also called MOSFET scaling, which states that as the device scale down, power density remains constant) and **dark silicon** (the part of silicon which is to be inactive to

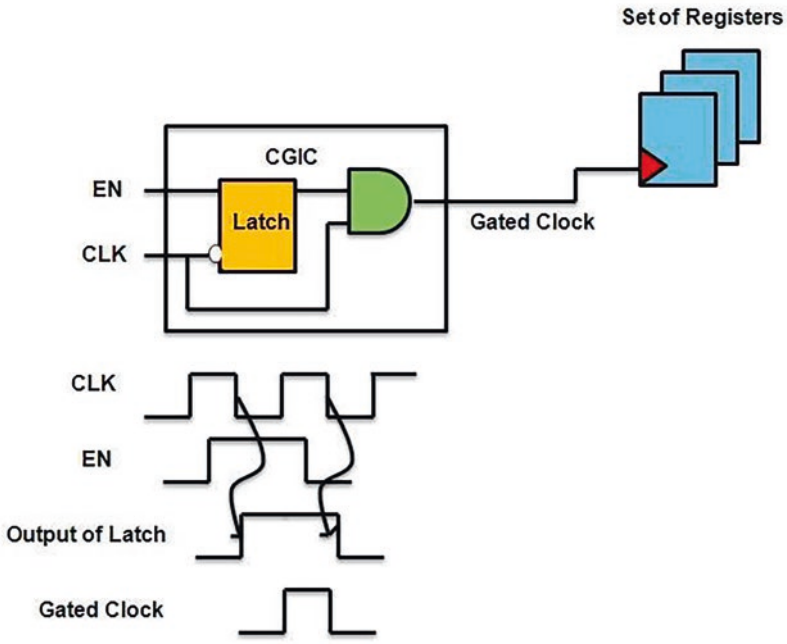


Fig. 9.7 Clock gate integrated cell

maintain the desired performance which otherwise increases power density) has reached a plateau in guaranteeing performance with storage requirements. This has forced researchers to look at alternative architectures to processor-centric architectures, which have been the standard system architectures. The most promising architecture in this endeavor is the near-memory processing architecture. Figure 9.8 shows the evolution of system architectures toward near-memory architectures.

As shown in the figure, the ever-lasting memory requirement, to a great extent, is met by multi-level hierarchical memory systems with multiple levels of on-chip cache memories, dynamic RAM (DRAM), and nonvolatile memory (NVM). New trend in system architectures with 3-D SOCs has raised hope for meeting the storage requirements of systems by near-memory processing data paths. Figure 9.9 shows the processing options for the system solutions following this trend.

As seen in the figure, the system architecture is moving away from processor centric to data-centric architectures and data is processed in close proximity to the memory containing it. This reduces the phenomenal amount of power consumption which would have happened in the data movement from memory to the processing core and vice versa. The 3D chip lets architectures of SOCs with the through silicon via (TSV) promote the solutions with near memory processing architectures of system design.

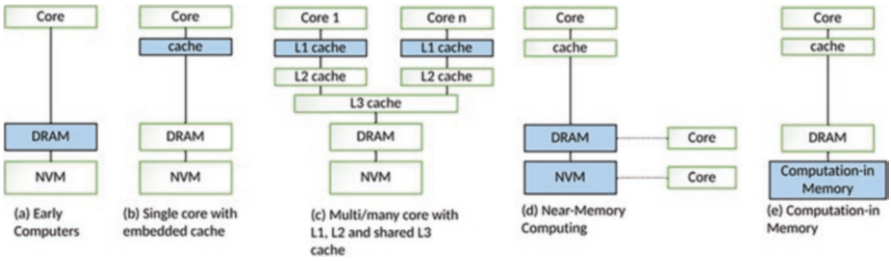


Fig. 9.8 Evolution of near-memory system architectures

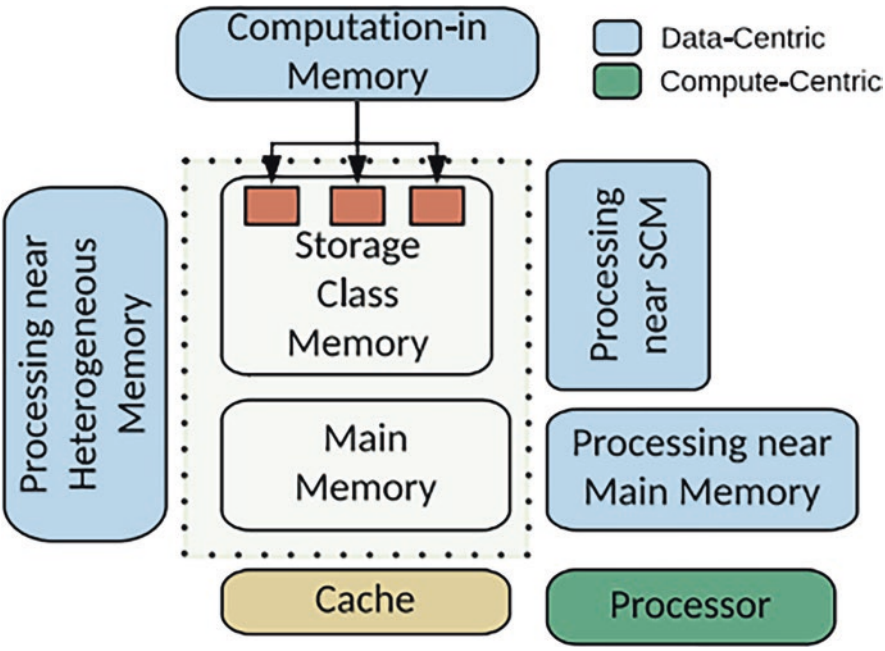


Fig. 9.9 Near-memory processing options

9.7.1 In-Memory Processing Architectures

Processing in-memory system architectures is a new outshoot of near-memory system architecture. It is also called in-storage processing and is another technique to overcome performance and power bottlenecks in SOCs for processing Big data. This is another candidate for 3-D SOC architectures. Figure 9.10 shows the in-memory processing concept.

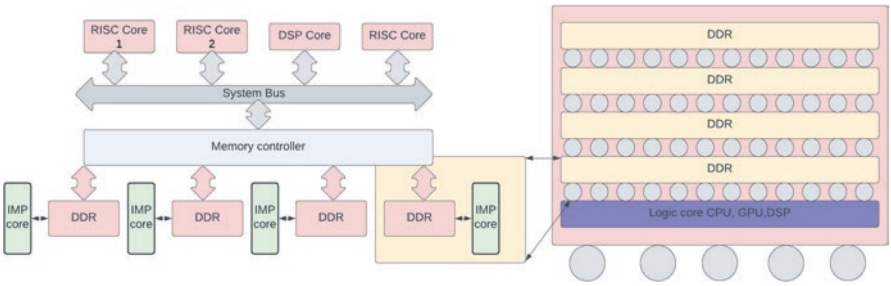


Fig. 9.10 In-memory processing (IMP) concept

To adopt these architectures, it is necessary to understand the system application requirements very well, as this determines the data processing workload. In addition, some of the other considerations are the following:

1. The decision on the type of memory to be used depends on the data processing workload in the system and hence its architecture.
2. Criticality of processor and granularity of processing done on the stored data (which is part of the entire data in the system) in the memory. Availability of modeling and simulation tools and their ability to let designers create near-application scenarios and data streams.
3. Interoperability, which deals with programming models, cache coherence, virtual memory, and efficient data mapping. Interoperability is one of the key enablers for the adoption of any new system.

9.8 Guidelines for a Good SOC Architecture

In this section, we define some general guidelines for defining SOC architecture, which will help architects to address challenges well ahead of time.

- List the application scenarios or use-case scenarios for the proposed SOC
- List all the applicable functional and regulatory standards for the proposed SOC
- Draft a conceptual functional architecture for the listed requirements
- Identify the possibility of parallel and pipeline functionalities
- List the
- After completing the feasibility analysis for SOC design, list all the functional requirements and classify them as follows:
  - Critical functions
  - Type of functions like application processing, core function, protocol, firmware, memory intensive, peripheral core, firmware, software
  - Plan to develop system models for critical functionalities

- Partition the functionality of the SOC into subsystems and partition the functionality into hardware software, and probable processor or memory architectures
  - Logistics of SOC operation such as booting, power management, storage, and interface
  - Applicable RISC, Signal processing or communication and storage functionalities
  - Map the available RISC processors to the functionalities listed and classified
  - Map the functionalities for subsystems conceptually into software hardware, memory, and interface
- Identify the hardware block hierarchy, timing budget, power management strategy, verification strategy for each of the subsystems
- Assemble the subsystems as an architecture for exploration to measure the system timing/performance. Need to accurately model the bottlenecks
- Identify the design Ips including the network on chip (NIC)
- Identify appropriate HW-SW design and verification environment
- List need for functional models for verification of SOC. Identify verification Ips and the software abstraction layers, embedded application software required.
- Plan the HW-SW codesign of the SOC subsystems by defining hardware-software co-synthesis in terms of identifying algorithms, textual /graphical representation of functions
- Plan for detailed design representation of the hardware software blocks as RTL/Gate level models, OS, compilable codes, programs, or OS routines.

Figure 9.11 shows the SOC development stages and the main decisions to be made from the choices.

Further in each of these stages there are number of design decisions are to be made to develop a successful SOC. It is to be remembered that each of the decision taken will affect the SOC development in terms of functionalities, performance, and the cost.

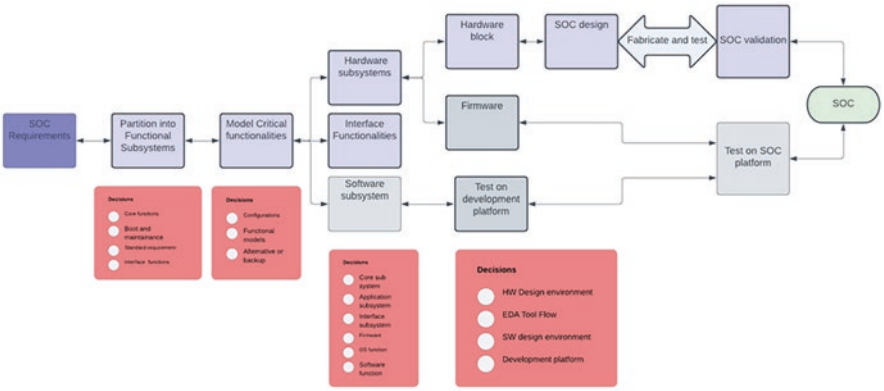


Fig. 9.11 SOC development flow

Reference

1. Theme article from Davide Giri. (n.d.). http://www.cs.columbia.edu/~luca/research/giri_MICRO18.pdf

Chapter 10
Self-Assessment Question Bank



Chapter 1

1. Layer stack is the organization of _____ in a system as layers

A) Functional blocks	B) Macros	C) Memories	D) Physical layers

2. System on Chip architecture is a process of defining the _____ of subsystems, their functionalities, and the process by which each interacts with other subcomponents of the system.

A) POWER distribution	B) Clocking strategies	C) Organization	D) Physical layers

3. Processor core is considered brain of the functional subsystem in System on Chip (SOC)

A) True	B) False

4. Compliance with standards defined by regulatory or professional bodies guarantees the _____ of the SOC developed with other SOCs with the same or peer interface subsystems

A) Correctness	B) Intention	C) Functionality	D) Interoperability

5. Instructions in processors are executed in a core functional block called _____

A) PC	B) Memory	C). CPU	D) ALU

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

V. S. Chakravarthi, S. R. Koteshwar, *System on Chip (SOC) Architecture*,
https://doi.org/10.1007/978-3-031-36242-2_10

6. _____is also called computer architecture.

A) ISA	B) Harvard	C) Van Neuman	D) Data path

7. Instruction decoder transfers a large amount of data from memory to registers

A) True	B) False

8. Many complex instructions are found in the instruction set popularly called complex instruction set computers (CISC)

A) True	B) False

9. _____instructions set is used for applications that do not require complex instructions

A) Complex	B) Reduced	C) Hybrid	D) Small

10. Micro program control unit (mPCU) typically exists in _____architecture

A) RISC	B). CISC	C) EPIC	D) VLIW

11. _____ uses instruction-level parallel operations for executing special instructions

A) RISC	B) CISC	C) EPIC	D) VLIW and EPIC

12. Depending on the number of processor cores used in an architecture, systems are classified as _____ and multiprocessor systems.

A) Uniprocessor	B) Many processors	C) Large processor	D) Analog processor

13. Depending on the type of physical structure, systems are classified as _____ and System on Chip

A) Hybrid systems	B) Embedded systems	C) Multiprocessor systems	D) Uniprocessor systems

14. Depending on the function performed by the systems, they are classified as Processing systems and _____

A) Computational systems	B) Multiprocessor systems	C) Server systems	D) Large systems

15. _____systems operate on signals and are widely used in multimedia applications.

A) Analog	B) Large	C) Data Server	D) Signal Processing

Chapter 2

16. _____Technology is used to integrate many circuits on a semiconductor wafer to make electronic systems

A) CMOS Fabrication	B) Soldering	C) Assembly	D) Printing

17. _____is used in PCB fabrication technology.

A) Silicon	B) Metal	C) FR4	D) Glass

18. Phenomenal growth in integrating many devices in CMOS is due to _____phenomenon.

A) Good designs	B) Scaling	C) EDA tools	D) Designers

19. High-performance processor System on Chips have instruction-driven architectures

A) True	B) False.

20. _____driven system architectures are used when a set of applications are targeted to be implemented on them

A) instruction	B) Platform	C). Data	D) Memory

21. Most times, System on Chips forms complete solutions

A) True	B) False

22. _____ form a large set of functional subsystems in many applications in SOC development.

A) Standard cells	B) IP Cores	C) Macros	D) Memory cores

23. A _____ is a functionally accurate and/or timing-accurate component realization

A) System-level models	B) Standard cells	C) Designs	D) Technology files

24. _____is predominantly known to be a system modelling language

A) Verilog	B) SystemC	C) TcL	D) C++

25. Virtual platforms are not high-level abstractions of IP models

A) True	B) False

26. Virtual platforms are configurable platforms that can run real operating systems software

A) True	B) False

27. In emulation systems virtual platforms are connected to real external peripheral interfaces such as memories, interface components

A) True	B) False

28. Physical platforms normally take the form of a development board containing the components in a chip on an FPGA

A) True	B) False

29. The key component of the hardware platform is the ____.___ _____ and _____

A) FPGA, SOC, Antenna	B) Clock system, Power supply and Oscillator	C) System controller like computer, Physical Interface, development board	D) Memory controller, Processor, SRAM

30. No one-size-fits-all applications are seen in _____architectures

A) RISC	B) Server	C) Mobile system	D) IOT

31. System on Chips does not typically require software.

A) True	B) False

Chapter 3

32. System architecture is an abstract structure of system elements implemented through technologies meeting a set of traceable requirements and supported by life cycle concepts

A) True	B) False

33. System architecture is defined based on principles, concepts, and theoretical practices.

A) True	B) False

34. Conceptualization helps system architects in characterizing the _____, identifying potential solutions.

A) Competition	B) System	C) Solution	D) Problem space

35. The _____ of an architecture model is determined relative to its intended use

A) Validation	B) Start	C) Time to market	D) Completeness

36. The design process provides feedback to the architecture processes to consolidate or confirm the concepts and properties of the architecture.

A) True	B) False

37. Market research leads to market requirement document with identified problems and probable solutions.

A) True	B) False

38. Drivers are often called “drivable computers.”

A) True	B) False

39. _____are often referred as drivable computers.

A) Vehicles	B) Drivers	C) Conveyers	D) Computers

40. _____and reliability are major requirements of automotive SOCs.

A) Engine	B) Safety	C) Speed	D) Fuel efficiency

41. Multiple cycle datapaths require additional memory and _____ components

A) ALU	B) Additional datapath	C) CPUs	D) buffer memories and associated Multiplexers

42. In multi-cycle data paths, fetch, decode, and execute operations are executed in the same cycle

A) True	B) False

43. Input processing in the SOC architecture always use analog to digital converters irrespective of type of input signals to be processed.

A) True	B) False

44. The speed of a circuit depends on the number of gates in the longest computational path.

A) True	B) False

45. The speed of the standard cell in VLSI technology library depends on degree of _____

A) clock	B) VDD	C) fan-out	D) fan-in

46. Latency of the logic operation does not depend on the implementation.

A) True	B) False

47. The performance of ALU depends on the degree of parallelism and the operand arrival latencies at the ALU blocks in the implementation

A) True	B) False

48. _____affect performance, just like better algorithms improves efficiency in software.

A) Designers	B) Resources	C) Architectural organization	D) EDA tools

49. Control is hardware that controls the flow of data and operations on them along the datapath

A) True	B) False

50. _____architecture is typically used in processor architectures

A) Register file	B) Load-store	C) Platform	D) Simple

51. Simplest architecture is _____architecture.

A) Register file	B) Load-Store	C) Hybrid	D) Platform

52. ALU is the basic building block of computing SOC.

A) True	B) False

53. Compliance to standard is not considered in architecture definition.

A) True	B) False

54. Many processing cores are used in compute-rich architectures.

A) True	B) False

55. Single instruction processor does not process any instruction.

A) True	B) False

56. Pipelining is facilitated by _____data paths in processor SOCs.

A) Single cycle	B) Multi cycle	C) Two cycle	D) One-and-half

57. Datapaths are always of type single cycle datapaths in all SOCs.

A) True	B) False

Chapter 4

58. _____ demand deferent performance from SOC designs.

A) Processor core	B) Applications	C) Clock	D) Implementation

59. Pipelining concept is used to increase _____of the design

A) Power	B) Speed	C) Area	D) Processing

60. The choice of processors in a SOC architecture depends on the _____

A) Process technology	B) Good design	C) Data processing requirements	D). Implementation

61. _____ are large class of devices used in most applications.

A) Embedded processors	B) Serial interfaces	C) USB Cores	D) Oscillators

62. The performance of the processor is determined by number of instructions processor supports and_____

A) Clock frequency	B) The clock cycles each of the instructions take to get executed	C) Setup time	D) Hold time

63. _____is an implementation technique to overlap the different phases of execution of instruction in a processor

A) Pipelining	B) Parallelism	C) Clock Tree	D). Multiple datapaths

64. Pipelining improves performance by increasing _____

A) Datapath throughput	B) Instruction throughput	Instruction latency	D) Time of execution

65. Pipelining reduces execution time of individual instructions.

A) True	B) False

66. All instructions will not have equal time of execution in processors

A) True	B) False

67. _____is a real metric to determine how good a processor core is.

A) Instruction throughput	B) Clock frequency	C) Number of instructions	D) Logic count

68. _____is the condition in processor when next phase of instruction in pipeline cannot be executed.

A) Lock	B) Time Out	C) Pipeline hazard	D) Race condition

69. _____translates high-level program to instruction specific assembly instructions in a processor core

A) Compiler	B) Assembler	C) Debugger	D) A,B,C

70. The data transfer instruction that copies data from memory to a register is traditionally called _____

A) Load	B) Store	C) Move	D) load-store

71. The most used architectures in processor design are _____

A) Stack architecture	B) VLIW	C) Register file architecture	D) Load-store architecture

72. There are three types of hazards: *structural hazards*, *data hazards,* and *instruction hazards*.

A) True	B) False

73. _____are unplanned and unscheduled event that occurs due to data overflow or underflow while executing the program

A) Interrupts	B) Status update	C) Exceptions	D). Critical interrupts

74. In _____the control of execution is transferred to the address of a register file or instruction memory depending on the cause of interrupts.

A) Any interrupts	B) Exception	C) Vectored interrupts	D) Critical interrupts

75. Pipelining is executing different stages of instruction in parallel

A) True	B) False

76. Architecture exploration selects a set of processing elements and maps the computation behavior of the specification onto the _____.

A) Processing elements (PE)s	B) Registers	C) Memory sections

77. Instructions executed in parallel in processor cores are termed as instruction-level parallelism.

A) True	B) False

78. Many instructions executed in parallel is called pipelining.

A) True	B) False

79. Single instruction processor is not much used in real applications.

A) True	B) False

80. Interrupt interface to hardware is through instructions or commands from software.

A) True	B) False

81. Software interface to hardware is through instructions or commands from software

A) True	B) False

Chapter 5

82. The multi-processor core systems invariably use _____ architectures with defined memory organization on chip for optimized system performance.

A) Instruction	B) Chip multi-processor (CMP)	C) Single data path instruction	D) Multi path

83. Off-chip memory is slower than on-chip memory in terms of access times.

A) True	B) False

84. Level 1 cache memory is faster accessible memory by processor.

A) True	B) False

85. Embedded memories in processor cores are organized in terms of _____

A) Size	B) Frequency	C) Speed	D) Levels of access to processor

86. Storing a set of packet lengths of data packets is typically organized as _____

A) Cache	B) Registers	C) Buffers	D) external memory

87. _____is an on-chip temporary storage for faster access of data by the processor

A) Internal stack memory	B) Higher levels of cache	C) Cache	D) Registers

88. Cache memory is a high speed on-chip memory

A) True	B) False

89. If data is found in the cache memory in a first attempt it is called _____

A) Cached	B) Cache miss	C) Cache hit	D) Stacked

90. _____is unavailability of required data in the cache memory when processor accesses it.

A) Cache frequency	B) Cached	C) Cache hit	D) Cache miss

91. _____is the high-speed temporary storage which stores data and programs which are frequently accessed by the processor elements.

A) RAM	B) Flash	C) Cache memory	D) External memory

92. _____memory is typically storage of large size.

A) Level 1 Cache	B) Level 2 Cache	C). On-chip	D) External memory

93. The data processed recently has a greater likelihood of being used again by the processor for further processing. This property is called _____

A) Spatial locality	B) Temporal locality	C) Bad data	D) Similarity

94. Data is fetched into the cache memory from the main memory in blocks called _____

A) Cachable data	B) Cache Dataline	C) Cache miss	D). Cache hit

95. In most cases algorithms use data from closer address proximity. This property is called_____

A) Temporal locality	B) Spatial locality	C) Redundant data	D) data miss

96. Temporal locality in processor system is fetching the data from the same address location.

A) True	B) False

97. Smaller the number of cache miss, better the processor performance.

A) True	B) False

98. _____is the Cache data accessed by a processor task which is updated by other process.

A) Cache Hit	B) Cache miss	C) Cache Coherence	D) Cache non-coherence

99. Most used protocol used to avoid Cache coherence issue in processor core is _____

A) Cache	B) MESI	C) CRC	D) FCS

100. TLBs hold logical or virtual addresses mapped to actual addresses of the physical memory

A) True	B) False

Chapter 6

101. Requirement to solution for a defined problem are both explicit and implicit in nature. Scalability and flexibility to use it as solution to other related problems are examples of _____requirements.

A) Implicit	B) Explicit	C) Compliance	D) Optional

102. Solution architecture translates technical needs into practical solutions while establishing rules and instructions for proper implementation and delivery.

A) True	B) False

103. The library of functional blocks from the standard available library in standard IDE is called _____

A) Standard library	B) Base Library	C) Shield	D) IP cores

104. _____of the solution can be achieved by adding another processing node if the solution is made modular.

A) Hardware and Software	B) Processor and analog sensors	C) Scalability	D) Expandability

105. There are several ways a solution can be implemented based on cost, level of automation, user interface, and technical feasibility.

A) True	B) False

106. In the case study of environmental monitoring system, the remote access of cloud server environmental data is optional.

A) True	B) False

107. _____ Partition is the major activity of System architecture definition.

A) Processor and Memory	B) Design elements	C) Logic gates	D) Hardware and Software

108. _____ is the process of dividing the probable computations of a system into a part that executes as sequential instructions on a processor and a part that runs as concurrent logic on a SOC

A) Hardware Software partition	B) IP core selection	C) Power supply choice	D) Technology library decision

109. _____is typically followed in analyzing solution architecture.

A) Software first	B) Hardware first	C) Outsourcing	D) Buying third-party IP solutions

110. _____ is high in case of any mistake hardware first approach of architecture definition

A) Time to market	B) Project cost	C) Performance	D) Project impact

111. IDE is not a necessary requirement for the proposed solution for environment monitoring.

A) True	B) False

112. _____is the main objective of design partitioning

A) High frequency of operation	B) Latency	C) Cost	D) Power reduction

113. Type of functional specification _____ is the basis on which the software-oriented partitioning and hardware-oriented modeling are carried out.

A) System architect	B) Manager	C) Model	D) Marketing manager

Chapter 7

114. IP Cores in any form can be integrated into SOC design.

A) True	B) False

115. Hard IP cores are more flexible

A) True	B) False

Chapter 8

116. SDLC consists of planning and explaining each phase of the development process.

A) True	B) False

117. Software requirement document of the last step in application development life cycle.

A) True	B) False

118. _____outlines the functionality of proposed Software modules in the software architecture

A) PRD	B) MRD	C) HLD	D) LLD

119. The architecture in which the data begins at the top layer and works its way down until it reaches the bottom, which is typically a database is called _____

A) Instruction Set Architecture	B) Data path architecture	C) Simple architecture	D) Layered architecture

120. The architecture where in system blocks waits for some event to occur to process the data is called _____

A) Instruction Set Architecture	B) Data path architecture	C) Event-driven architecture	D) Layered architecture

Answers

Chapter 1

1. D)
2. C)
3. A)
4. D)
5. C)
6. A)
7. B)

8. A)
9. B)
10. B)
11. D)
12. A)
13. B)
14. A)
15. D)

Chapter 2

16. A)
17. C)
18. B)
19. A)
20. B)
21. B)
22. B)
23. A)
24. B)
25. B)
26. B)
27. A)
28. A)
29. C)
30. D)
31. B)

Chapter 3

32. A)
33. A)
34. D)
35. D)
36. A)
37. A)
38. B)
39. A)
40. B)
41. D)
42. B)
43. B)
44. A)
45. D)
46. B)
47. A)
48. C)

49. A)
50. B)
51. A)
52. A)
53. B)
54. A)
55. B)
56. B)
57. B)
58. B)

Chapter 4

59. B)
60. C)
61. A)
62. B)
63. A)
64. B)
65. B)
66. A)
67. A)
68. C)
69. A)
70. A)
71. D)
72. A)
73. C)
74. C)
75. A)
76. A)
77. A)
78. B)
79. A)
80. B)
81. A)

Chapter 5

82. A)
83. A)
84. A)
85. D)
86. C)
87. C)
88. A)
89. C)

90. D)
91. C)
92. C)
93. D)
94. B)
95. B)
96. B)
97. A)
98. D)
99. B)
100. A)

Chapter 6

101. A)
102. A)
103. C)
104. C) & D)
105. A)
106. B)
107. D)
108. A)
109. B)
110. D)
111. B)
112. A)
113. C)

Chapter 7

114. A)
115. B)

Chapter 8

116. A)
117. B)
118. C)
119. D)
120. C)

Index

A
Accelerators, 6, 62, 93, 127–128
Analog front end (AFE), 93, 100
Application development life cycle, 105, 152
Application-specific SOCs, 49–63
Architecture processes, 37–40, 143

C
Cache management, 69
Cache memory, 23, 65–71, 80, 84, 134, 148, 149
Clock gates, 133, 134
Cluster, 65
Cockpit, 43
Code memory, 55
Computer architecture, 9
Computer-driven architectures, 9, 140
Conceptual models, 3, 38, 75, 76, 109
Control path, 6, 8, 71, 100

D
Data cache (DCache), 81, 102
Data-driven architecture, 19
Data memory, 32, 33, 47
Datapath, 45–48, 100, 143–146
DDR controller, 22, 93
Development board, 22
Design-space analysis, 128

E
Embedded processor subsystems, 50–53
Embedded RAM, 91
Environment monitoring, 75–89, 118, 121, 124–125, 151
Exceptions, 56–57, 63

F
Feasibility study, 40

G
Gateway, 25–28, 125

H
Hard IP, 94–95, 152
Hardware-software interface, 53–55, 128
Hardware-software partition, 85–91
Harvard architecture, 30
High level design (HLD), 107
Hits, 69, 71, 148

I
In-memory processing (IMP), 135–136
Instruction cache (ICache), 81, 102
Instruction-driven architectures, 18, 141
Instructions, 8–11, 18, 30–32, 34, 35, 45–47, 50–57, 63, 66, 77, 81, 85, 102, 103, 139–141, 145–148, 150–152
Interface cores, 5, 8, 14, 17, 24, 48, 62, 80, 84, 85, 93, 95, 103, 111, 125
Internet of Things (IOT), 10, 13, 15, 40, 42, 62, 75, 78, 79, 81, 85, 110
Interrupts, 6, 20, 22, 23, 56–57, 88, 125, 147
IOT SOC, 25, 29, 42, 43, 76–90, 94, 111

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2023

V. S. Chakravarthi, S. R. Koteshwar, *System on Chip (SOC) Architecture*,
https://doi.org/10.1007/978-3-031-36242-2

IOT software, 110
Isolation cells, 130, 132, 133

L
L1, 23, 68
L2 cache, 23, 68
Level shifters, 130, 131
Low-level design (LLD), 107–108

M
Memory management units (MMU), 71
Message queuing telemetry transport (MQTT), 81, 119, 124, 125
Misses, 20, 68–71, 148, 149
Multiple clock architectures, 128–130
Multi-voltage architectures, 130–131

N
Near-memory processing, 73, 133–136
Network on chip (NOC), 5, 80

O
On-chip cache, 80, 134
Open SystemC Initiative, 21

P
Parallelism, 45, 51, 53, 144, 146, 147
  throughput, 53
Phase locked loop (PLL), 61, 93, 94, 103
Pipelining, 47, 51–53, 145–147
Platform architectures, 20, 129
Power gates, 132
Printed circuit board (PCB), 1, 3, 141
Processes, 3, 4, 6, 8, 9, 17–19, 23–26, 29, 32, 37–40, 42, 43, 46, 48, 49, 57, 58, 60, 62, 71, 77, 78, 80, 81, 85, 86, 89, 94, 95, 99, 100, 105, 107, 108, 110, 130, 139, 143, 145, 149, 151, 152
Processing, 2, 5–14, 17–19, 23, 24, 27, 29, 30, 32, 34, 35, 41, 42, 51, 57, 60, 62, 65–68, 72, 73, 77, 78, 80, 85, 88, 89, 93, 100, 101, 104, 110, 112, 120, 127, 133–136, 140, 144, 145, 147, 149, 150
  ALU, 9
Processing elements (PEs), 57–59, 95, 97, 100, 104, 147
Processor subsystems, 8
Proof of concept system, 78–81
Protocol block, 8

Q
Quality assurance (QA), 105, 108
Quality of results (QOR), 15

R
Read only memory (ROM), 4, 5, 23, 41, 72, 91
Real-time operating system (RTOS), 3, 21, 27, 103, 111
Register set, 63, 66
Representational State Transfer (REST), 81, 119
Retention cells, 130, 132
Redundancy in memory, 73
RISC architectures, 10
RISC V, 21, 55, 83, 84, 102

S
Scratchpad, 66, 88
Sensors, 8
Signals, 2, 3, 5, 6, 8, 12, 13, 17, 29–35, 42, 46, 47, 62, 72, 81, 82, 84, 92, 93, 100, 127, 129, 130, 132, 133, 141, 144
Silicon dies, 8
SOC accelerators, 93, 127, 128
SOC architecture, 4, 6, 9–14, 37–48, 65, 75–104, 125, 127–136, 144, 145
Soft IP, 21, 22, 91, 94, 95, 99
Software development life cycle (SDLC), 105–108, 152
Software model, 89
Solution architecture, 28, 40, 76–77, 150, 151
  automotive safety, 43
Storage in SOCs, 65–73
Stored program concepts, 55
System architectures, 3, 7, 8, 10, 11, 13, 14, 17, 20, 38, 40–42, 50, 51, 57–59, 63, 66–71, 73, 78, 101, 104, 107, 130, 134, 135, 141–143, 151
System modelling, 57–58, 142
System on chips (SoCs), 3–15, 17–35, 37–63, 65–73, 75–95, 100, 102–125, 127–130, 133–135, 139, 141–145, 151
  DMA, 9
  EPIC, 11
  VLIW, 11

T
Tasks, 27, 30, 40, 41, 45, 56, 57, 107, 109, 127, 149
TCP/IP, 81
Threads, 41, 69, 70
3D ICs, 25

V
Virtual memory, 136
Virtual platform, 22
Von Neumann architecture, 30